

CSC490 Assignment 3: Pre-training Nanochat

Sohee Goo / 1008065479 Akshaya Deepak Ramachandran / 1008806810
Maryam Taj / 1008990362 Kashish Mittal / 1009115315

NOTE: Our forked repository and code changes were made here: <https://github.com/soheegoo/nanochat>. We used nanochat_modal.py to run our experiments.

1 Part 1

1.1 Architecture Diagram

To fully understand nanochat's architecture, we need to contextualize it within the broader framework of decoder-only transformers.

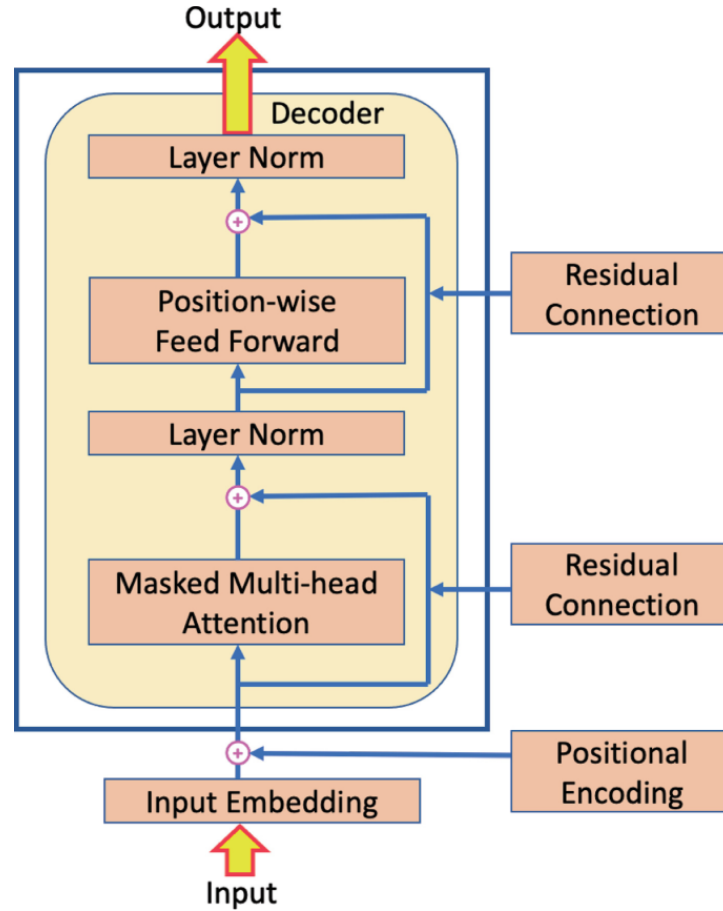


Figure 1: The Transformer (decoder-only) model architecture.

We will use the tiny Shakespeare dataset to provide a concrete illustration of the decoder-only transformer architecture. In other words, we will be generating Shakespearean-like content.

- Let B = batch size, T = sequence length, V = vocab size, C = embedding dimension, H = # of heads, and $d_h = C / H$.
- Input $(B, T) = (16, 32)$. In other words, there are 16 chunks, where each chunk is 32 consecutive characters from the Shakespearean dataset.
- Token embedding $(B, T, C) = (16, 32, 64)$. There are 16 chunks, where each chunk is 32 consecutive characters from Shakespearean dataset, and each character maps to a $(64,)$ embedding.
- Positional embedding (T, C) , which is broadcasted to $(B, T, C) = (16, 32, 64)$. There are 16 chunks, where each chunk is 32 consecutive characters from Shakespearean dataset, and each character maps to $(64,)$ embedding that encodes its position.

- Input embedding + positional embedding (B, T, C) = (16, 32, 64).
- We will repeat this block N times:
 - Masked multi-head attention:
 - * Linear projection (B, T, d_h) = (16, 32, 16) = For each head, we have three linear projections (query, key and value) of size (16, 32, 16). That is, 16 chunks, where each chunk is 32 consecutive characters and each character maps to a smaller embedding space encoding what the head is looking for (query), key (how the token would describe itself), and it's value (information).
 - * Attention scores (B, T, T) = (16, 32, 32) = For each head, we compute attention scores ($Q @ K.T$). That is, 16 chunks, where each chunk is a 32 by 32 matrix of attention scores, corresponding to how much one token should attend to every other token.
 - * Masking (B, T, T) = (16, 32, 32) = 16 chunks, where each chunk is a 32 by 32 matrix of attention scores, and future tokens are masked (i.e., set to infinity). That way, we only use previous tokens to predict future tokens.
 - * Softmax (B, T, T) = (16, 32, 32) = 16 chunks, where each chunk is a 32 by 32 matrix of attention scores, and masked tokens are set to 0 while ensuring the remaining tokens' scores add up to 1.
 - * Weighted sum (B, T, d_h) = (16, 32, 16) = 16 chunks, where each chunk is 32 consecutive characters from Shakespearean dataset, where each character maps to (16,) embedding that scales it's value by the attention scores.
 - * Multi-head concatenation (B, T, C) = (16, 32, 64).
 - Residual connection (B, T, C) = (16, 32, 64). $x = x + \text{masked_multi_head_attention}(x)$.
 - LayerNorm (B, T, C) = (16, 32, 64). There are 16 chunks, where each chunk is 32 consecutive characters from Shakespearean dataset, and each character maps to (64,) embedding that is normalized to have zero mean and unit variance.
 - Feed-forward network (B, T, C) = (16, 32, 64). The feed-forward networks is a two-layer MLP, which first projects (16, 32, 64) to (16, 32, $64 * 4$), applies ReLU, then projects to (16, 32, 64).
 - Residual connection (B, T, C) = (16, 32, 64). $x = x + \text{feed_forward_network}(x)$
 - LayerNorm (B, T, C) = (16, 32, 64).
- Linear projection (B, T, V) = (16, 32, 64) = After N blocks, we project (B, T, C) to (B, T, V).
- Softmax (B, T, V) = (16, 32, 65). There 16 chunks, where each chunk consists of 32 consecutive characters and each character maps to (65,) denoting the probabilities of each vocabulary word to be the next token.

With the traditional decoder-only transformer architecture above, here is nanochat's architecture diagram:

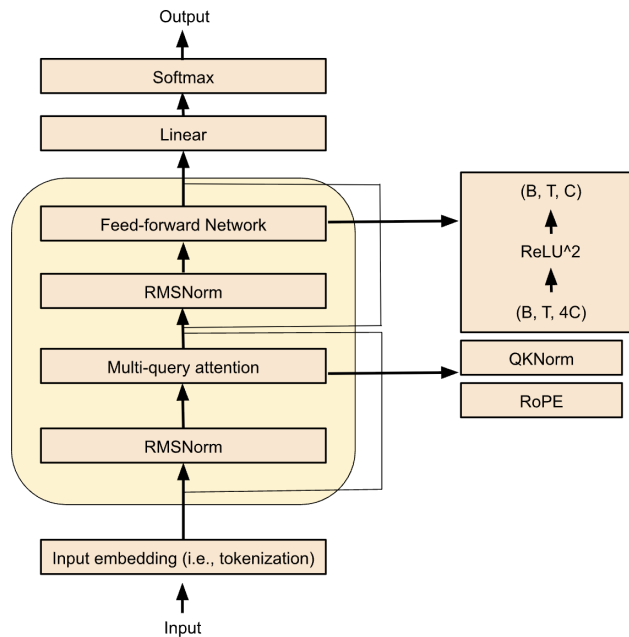


Figure 2: Nanochat model architecture.

Notable differences include:

- Nanochat modifies how it normalizes, and when. For example, it uses RMSNorm instead of LayerNorm, which ensures that the root mean square of the data values is 1. Additionally, nanochat applies RMSNorm before attention, and feed-forward network (i.e, PreNorm), unlike the original architecture which uses PostNorm.
- Nanochat applies RoPE (rotary positional embedding) to the query and key embeddings inside the multi-query attention step. In contrast, the original architecture used absolute positional embeddings, which it added to the tokenized embedding.
- Nanochat normalizes the query and key embeddings (i.e., QKNorm) for faster and stable training.
- Nanochat uses multi-query attention, where each head has a separate query. However, each head shares the same key and value embeddings.
- Nanochat uses $ReLU^2$, instead of standard ReLU for smoother gradients.
- Nanochat uses untied embeddings. That is, it learns different matrices for tokenizing the input, and transforming the hidden state to vocabulary logits. This allows the input and output layers to specialize.

1.2 Peri-LN

NOTE: We didn't use a table for this section (compare 3 papers) as we thought that the formatting would be too tight.

1.2.1 Paper

Peri-LN: Revisiting Normalization Layer in the Transformer Architecture

Kim, J., Lee, B., Park, C., Oh, Y., Kim, B., Yoo, T., Shin, S., Han, D., Shin, J., & Yoo, K.M. (2025). Proceedings of the 42nd International Conference on Machine Learning (ICML 2025).

1.2.2 Motivation for Change

Post-LN (normalizing after the residual addition) is stable but suffers from vanishing gradients in deep models. Pre-LN (normalizing before each sublayer input) enables easier training but allows unbounded activation growth, since sublayer outputs flow directly into the residual stream without any magnitude constraint. Even when training appears stable on the surface, the Peri-LN paper shows that hidden state variance grows exponentially with depth throughout training. At sufficient depth, this accumulation causes gradient spikes, loss spikes, or divergence. When a very large residual stream value is added to a small but sublayer update, the small update is numerically absorbed and lost. Later layers effectively stop learning because their updates are too small relative to the magnitude of the stream they are trying to modify.

Nanochat already contains evidence that activation growth is a concern. Logit softcapping ($15 * \tanh(x/15)$ at gpt.py:531) prevents massive values from destabilizing the final prediction layer. QK-norm (gpt.py:119) prevents attention score explosion.

Peri-LN addresses this by normalizing each sublayer's output before it enters the residual stream. This prevents any single sublayer from injecting outsized activations and ensuring every layer's contribution remains meaningful throughout training.

Google's Gemma 2 and Gemma 3 (2024-2025) and AI2's OLMo 2 (2025) have both adopted this double-norm strategy to prevent massive activations at scale.

1.2.3 Key Details from Paper

- Pre-LN gradient: $\left| \frac{\partial \mathcal{L}}{\partial W^{(2)}} \right| \propto |h|$ — the gradient norm is directly proportional to the activation magnitude. When activations explode, gradients explode. This is why Pre-LN models are prone to sudden training crashes.
- Peri-LN gradient: $\left| \frac{\partial \mathcal{L}}{\partial W^{(2)}} \right| \leq \frac{4\gamma\sqrt{D}|h|}{|a|}$ — the extra normalization introduces a damping factor $|a|$ in the denominator. Even when activations in a layer become massive, the denominator grows proportionally, keeping the gradient bounded. This is the self-regularizing property that makes Peri-LN fundamentally more stable.
- Peri-LN consistently outperforms Pre-LN at every scale tested (400M, 1.5B, 3.2B).
- Variance in final performance across 5 random seeds drops by over 50
- Pre-LN training occasionally diverges entirely (NaN loss, wasted compute) depending on the random seed. Peri-LN never diverged across any seed or scale tested.
- Pre-LN produces frequent gradient spikes and uneven gradient magnitudes across layers. Peri-LN keeps gradient norms uniform across all layers.
- Peri-LN achieves higher benchmark scores. The gain comes from more uniform gradients since Pre-LN's early layers receive weaker gradients and never fully optimize, wasting model capacity.

1.2.4 Technical Details of Change

Nanochat currently uses Pre-LN, where normalization is applied only at the input of each sublayer. Peri-LN applies RMSNorm both before and after each sublayer (attention and MLP). In nanochat, this changes Block.forward from:

```
# Pre-LN (current nanochat):
x = x + self.attn(norm(x), ...)    # norm before only
x = x + self.mlp(norm(x))         # norm before only
to:
# Peri-LN:
x = x + norm(self.attn(norm(x), ...)) # norm before AND after
x = x + norm(self.mlp(norm(x)))      # norm before AND after
```

Regarding the learnable scale parameter γ in RMSNorm. Learnable γ yields only slightly better performance than a frozen γ , which already provides the key stability benefit. Nanochat uses purely functional RMSNorm with no learnable parameters, which is equivalent to frozen $\gamma = 1$.

The paper also requires that initial embeddings are normalized which is already implemented in nanochat at gpt.py:518: `x = norm(x)`. Final outputs are also already normalized at gpt.py:524: `x = norm(x)` after the block loop. Compute: 2 extra RMSNorm calls per layer. RMSNorm is computationally cheap, so this adds minimal overhead. Memory: No new parameters are introduced.

1.2.5 Potential Impact on Model Performance

At pichat scale (12 layers), we might get slightly lower val BPB since more uniform gradients mean all layers learn effectively rather than early layers being undertrained. The paper shows $3.43 \rightarrow 3.34$ at 400M params. At pichat scale (much smaller), the gap will likely be smaller.

Also, fewer loss spikes and smoother convergence. The gradient spikes that cause sudden loss jumps in Pre-LN are suppressed by the output norm.

No meaningful change in training speed since the 2 extra RMSNorm calls per layer are negligible compute.

The big Peri-LN benefits like preventing divergence and large quality improvements will probably appear at large scale (1.5B+). At d12, Pre-LN doesn't typically diverge anyway, so the improvement will be modest.

1.3 Differential Transformer

1.3.1 Paper

Differential Transformer

Ye, T., Dong, L., Xia, Y., Sun, Y., Zhu, Y., Huang, G., & Wei, F. (2024). Microsoft Research / Tsinghua University.

1.3.2 Motivation for Change

Traditional transformers compute a single softmax attention map using query-key similarity. However, this mechanism often distributes probabilities across many tokens, including irrelevant tokens. As model context length increases, these non-negligible attention weights accumulate and drown out the truly relevant tokens - this is known as "attention noise." Empirical research shows that attention noise can cause failure in tasks such as information retrieval over long contexts.

Differential Transformer addresses this by computing two independent attention maps and subtracting them. The subtraction cancels shared noise while amplifying signal (i.e., relevant tokens), similar to noise-canceling headphones, which remove common-mode noise by subtracting two correlated signals.

1.3.3 Key Details from Paper

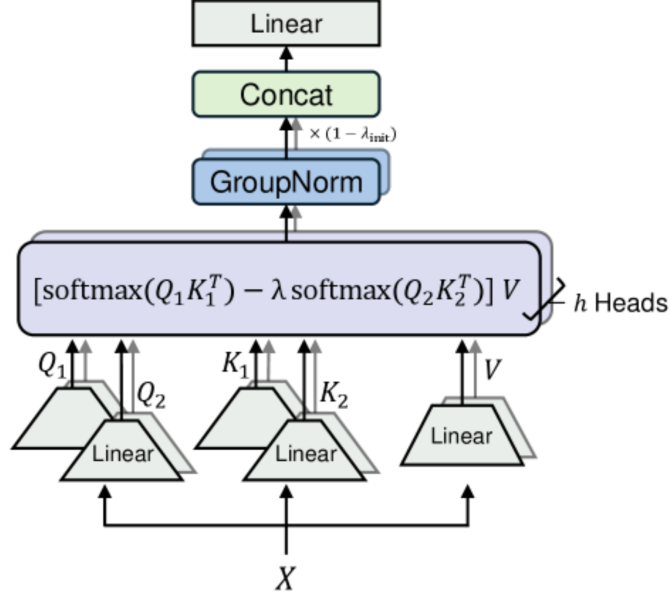


Figure 3: Differential attention model architecture.

- Each head splits the query, key vectors into two groups. Unlike the standard attention map, $\text{softmax}(QK^T)$, the model computes $\text{softmax}(Q_1 K_1^T) - \lambda \text{softmax}(Q_2 K_2^T)$. That is, each head takes the difference between two softmax attention maps to cancel out attention noise.
- λ is a learnable scalar that is initialized to $\lambda_{init} = 0.8 - 0.6e^{-0.3(l-1)}$, where $l \in [1, L]$ represents layer index. It is reparameterized using four vectors $\lambda_{q1}, \lambda_{k1}, \lambda_{q2}, \lambda_{k2}$ to stabilize training dynamics.
- GroupNorm applies normalization to each head independently. A fixed multiplier $(1 - \lambda_{init})$ is used after GroupNorm, which aligns the gradient flow with Transformer.

1.3.4 Technical Details of Change

Nanochat computes a single softmax attention map. Differential attention will compute two independent attention maps and subtract them to suppress irrelevant context tokens and yield a sparser, higher signal-to-noise attention distribution. .

```
# Single softmax attention map (current nanochat):
def Attention(X, W_q, W_k, W_v):
    Q = X @ W_q
    K = X @ W_k
    V = X @ W_v
    # Q, K, V: [b, n, d]
    s = 1 / sqrt(d)
    A = Q @ K.transpose(-1, -2) * s
    return softmax(A) @ V

def MultiHead(X, W_q, W_k, W_v, W_o):
    O = [Attention(X, W_qi, W_ki, W_vi) for i in range(h)]
    return Concat(O) @ W_o

# Difference of softmax attention maps:
def DiffAttn(X, W_q, W_k, W_v, lambda):
    Q1, Q2 = split(X @ W_q)
    K1, K2 = split(X @ W_k)
    V = X @ W_v
    # Qi, Ki: [b, n, d]; V: [b, n, 2d]
```

```

s = 1 / sqrt(d)
A1 = Q1 @ K1.transpose(-1, -2) * s
A2 = Q2 @ K2.transpose(-1, -2) * s
return (softmax(A1) - lambda * softmax(A2)) @ V

def MultiHead(X, W_q, W_k, W_v, W_o, lambda):
    O = GroupNorm([DiffAttn(X, W_qi, W_ki, W_vi, lambda) for i in range(h)])
    O = O * (1 - lambda_init)
    return Concat(O) @ W_o

```

1.3.5 Potential Impact on Model Performance

Differential attention produces sparser and higher signal-to-noise attention distributions, allowing the model to focus more strongly on relevant tokens. Experiments show substantial improvements in key information retrieval tasks. That is, differential transformers assign significantly higher attention to answer spans while reducing attention allocated to irrelevant tokens. For example, attention to the answer span increases from approximately 0.03 in a standard transformer to 0.30, while attention noise drops from 0.54 to 0.02.

Additionally, the model exhibits improved scaling efficiency. Experiments show that a differential transformer can reach the same language modeling loss as a standard transformer using about 60–65% of the parameters or training tokens, suggesting better utilization of model capacity.

With picochat, we could expect moderate improvements in attention quality and retrieval behavior, particularly for longer contexts. However, the largest benefits observed in the paper appear at larger model scales and longer training regimes, so gains in a small experimental setting may be smaller.

1.4 Token Order Prediction

1.4.1 Paper

Predicting the Order of Upcoming Tokens Improves Language Modeling

Zayd M. K. Zuhri, Erland Hilman Fuadi, Alham Fikri Aji (2026). Predicting the order of upcoming tokens improves language modeling.

1.4.2 Motivation for Change

Standard Next Token Prediction (NTP) only predicts the next best token given a sequence. While it has been very successful, it has received some criticism.

Multi-Token Prediction (MTP) emerged as a relatively successful auxiliary learning task that augments NTP by predicting the next n tokens at each position with a weighted loss. The original MTP paper reports gains on some generative tasks that require look-ahead, such as summarization and coding, especially at larger model scales. However, it also notes that MTP does not consistently improve performance on standard NLP benchmarks and that benefits are much weaker or absent for smaller models.

In addition, the number of future tokens to predict is a sensitive hyperparameter: increasing the future token count requires adding more heads, which increases both parameter count and FLOPs. Simply predicting more tokens does not guarantee better performance, even on “benefiting” tasks.

Token Order Prediction is an auxiliary training loss that aims to improve upon MTP by predicting the order of upcoming tokens in the sequence with a learning-to-rank loss rather than exactly predicting multiple future tokens.

Karpathy has already experimented with adding MTP to nanochat and reports that “the extra memory and compute overhead from predicting multiple tokens doesn’t pay off, in fact the results get worse. The auxiliary loss signal may help in other settings (larger models, different architectures?), but for our setup it’s pure overhead at the moment.” In other words, MTP in nanochat behaves exactly like the TOP paper’s critique: it adds cost without reliable gains.

TOP addresses this by: (1) During training, TOP requires only a single additional unembedding layer compared to MTP’s multiple transformer layers. During inference, we remove the TOP head and use only the NTP head, making the model equivalent to the original transformer architecture. (2) Paper presents empirical results that shows improves standard NLP benchmarks across multiple model sizes, including in areas where MTP underperforms.

1.4.3 Key Technical Details

Target Construction. At each position t in the sequence, TOP constructs a proximity score over the entire vocabulary. Given a window size W , each vocabulary token v receives a score of $W - d$, where d is the number of steps until v ’s next occurrence within the window. Tokens that do not appear within the window receive a score of $-\infty$. The result is a vector y_t over the vocabulary where tokens appearing sooner receive higher scores.

Loss Function. TOP uses the ListNet ranking loss, which treats the proximity scores as a soft target distribution. Both the target scores y_t and the model’s predicted scores $\hat{y}_t = U_{\text{TOP}}(h_t^L)$ (Where U_{TOP} is the TOP head explained below) are passed through a softmax to produce probability distributions, and cross-entropy is computed between them:

$$\mathcal{L}_{\text{TOP}} = - \sum_t \text{softmax}(y_t)^\top \log \text{softmax}(\hat{y}_t)$$

This differs from standard next-token prediction (NTP), which uses a one-hot target pointing only to the immediate next token.

Architecture. The TOP head is implemented as a single linear layer U_{TOP} added in parallel to the existing NTP head. Both heads receive the same final hidden state h_t^L from the transformer and project it logits (with the same size as the vocabulary). No additional transformer layers are required.

The combined training loss is defined as:

$$\mathcal{L} = (1 - \alpha)\mathcal{L}_{\text{NTP}} + \alpha\mathcal{L}_{\text{TOP}}$$

where α is a loss weighting hyperparameter. The paper finds that $\alpha = 0.9$ performs best in their experiments.

Inference. The TOP head is discarded at inference time. The deployed model is therefore identical to a standard NTP transformer. The benefits of TOP are captured entirely in the learned hidden representations.

1.4.4 Technical Details of Change

Current nanochat implementation. Nanochat currently uses a single `lm_head` and is trained using standard cross-entropy next-token prediction.

Proposed modification. Token Order Prediction (TOP) can be integrated into nanochat with minimal architectural changes.

1. **Add a TOP prediction head.** Add a linear layer parallel to the existing `lm_head`:

```
self.lm_head_top = nn.Linear(n_embd, vocab_size, bias=False)
```

Both heads operate on the same final hidden state of the transformer.

2. **Implement TOP target construction.** Construct proximity targets for each token using the window-based scoring method described in the paper. For efficiency, this could ideally be implemented as a Triton kernel so that targets are generated on-the-fly during training.
3. **Add the ListNet ranking loss.** Modify the forward pass to compute both the standard next-token prediction (NTP) loss and the TOP ranking loss. The total training loss becomes:

$$\mathcal{L} = (1 - \alpha)\mathcal{L}_{\text{NTP}} + \alpha\mathcal{L}_{\text{TOP}}$$

4. **Modify the dataloader to provide extended sequences.** Sequences should be extended by `window_size` tokens so that future-token proximity scores can be computed near the end of each sequence.
5. **Add TOP parameters to the model configuration.** Extend `GPTConfig` to include the following parameters:
 - `use_top: bool`
 - `top_window_size: int`
 - `top_alpha: float`

During inference, the TOP head is discarded and generation proceeds exactly as in the standard nanochat model.

1.4.5 Potential Impact on Model Performance

Empirical results from the TOP paper show consistent improvements in general language modeling benchmarks across multiple model scales. (The Authors do note in the limitations section that due to compute limitations they were not able to scale to larger models). TOP also shows particularly strong performance on tasks that require multi-step reasoning or planning.

Importantly, these gains are achieved with minimal computational overhead. TOP requires only a single additional linear projection layer. Because the TOP head is removed during inference, the deployed model remains identical to the standard nanochat architecture. As a result, TOP provides a practical way to improve training signal and model performance without increasing inference cost.

2 Part 2

NOTE: See appendix for all WandB graphs.

2.1 Picochat Model (d12)

We use a depth-12 configuration (n_layer=12, n_embd=768, n_head=6, sequence_len=2048, 110M parameters) as our ablation baseline. Compared to a full nanochat run (n_layer=24+, 560M–1B parameters), this is 4–8× smaller in parameter count. Since we are running four ablations (baseline, Peri-LN, Differential Attention, both) we want to preserve our Modal budget since we need to training our large vision model for the final project. This was a key consideration for our group.

A d4–d6 model (16M params) risks being too small to differentiate architectural variants. All configs might converge to similar poor loss and the CORE metric stays near random regardless of the architecture. This config is also what the nanochat codebase itself uses for iterating on code changes, indicating that it is the intended quick-test size.

2.2 Picochat + Peri-LN

2.2.1 Tables of Results

Table 1: Ablation summary

Metric	Baseline	Peri-LN
CORE metric	0.1286	0.1101
CORE metric estimate (training)	0.1352	0.1111
Val bpb	0.9059	0.9098
Min validation bpb	0.9059	0.9098
Final validation bpb	0.9059	0.9098
Final training bpb	0.9050	0.9087
Training loss (final)	2.9751	2.9886
MFU	43.72%	42.83%
Mean MFU	43.72%	42.83%
Training time	9.09 min	9.14 min
Peak memory	55,875 MiB	60,495 MiB
Training stability	Stable	Stable
New parameters	0	0

Table 2: Validation bpb at checkpoints throughout training.

Step	Baseline val bpb	Peri-LN
250	1.1265	1.1534
500	1.0470	1.0571
750	1.0123	1.0203
1000	0.9921	0.9972
1250	0.9720	0.9762
1500	0.9496	0.9536
1750	0.9303	0.9341
2000	0.9139	0.9178
2205	0.9059	0.9098

Table 3: Training loss at representative steps.

Step	Baseline train loss	Peri-LN
100	4.6045	5.2008
500	3.4549	3.4916
1000	3.2627	3.2803
1500	3.1214	3.1341
2000	3.0134	3.0261
2200	2.9751	2.9886

Table 4: Hardware utilization metrics averaged over training run.

Metric	Baseline	Peri-LN
Mean MFU	43.72	42.83
Approx. tokens/sec (steady state)	~2,150K	~2,090K
Peak memory	55,874 MiB	60,494 MiB
Memory overhead	—	+4,620 MiB (+8.3%)

Table 5: Full benchmark evaluation at final checkpoint (step 2205).

Task	Baseline	Peri-LN	Δ
<i>Summary</i>			
CORE metric	0.1286	0.1101	−0.0185
Val bpb	0.9059	0.9098	+0.0039
<i>Commonsense / NLI</i>			
HellaSwag (zeroshot)	0.0966	0.0941	−0.0025
HellaSwag	0.0974	0.0881	−0.0093
COPA	0.2200	0.1400	−0.0800
CommonsenseQA	0.1523	0.0438	−0.1085
Winograd	0.1136	0.0916	−0.0220
Winogrande	0.0087	<i>0.0592</i>	+0.0505
PIQA	0.2535	<i>0.2568</i>	+0.0033
<i>Knowledge / QA</i>			
BigBench QA Wikidata	0.3069	0.2744	−0.0325
ARC-Easy	0.3316	0.3288	−0.0028
ARC-Challenge	0.0159	0.0148	−0.0011
OpenBookQA	0.0640	<i>0.0747</i>	+0.0107
Jeopardy	0.0071	0.0019	−0.0052
LAMBADA (OpenAI)	0.2630	<i>0.2822</i>	+0.0192
SQuAD	0.0731	0.0308	−0.0423
CoQA	0.1249	0.1124	−0.0125
BoolQ	−0.2200	−0.4840	−0.2640
<i>Reasoning / Logic</i>			
BigBench CS Algorithms	0.4159	<i>0.4303</i>	+0.0144
BigBench Operators	0.1238	0.1238	±0.0000
BigBench Repeat Copy	0.0312	0.0312	±0.0000
BigBench Dyck Languages	0.0710	<i>0.1350</i>	+0.0640
AGI Eval LSAT-AR	0.1033	<i>0.1087</i>	+0.0054
<i>Other</i>			
BigBench Language ID	0.1750	<i>0.1828</i>	+0.0078

2.2.2 Picochat + Peri-LN Analysis

The baseline picochat outperforms the Peri-LN variant on the headline CORE metric (0.1286 vs. 0.1101) and on final validation bpb (0.9059 vs. 0.9098). Examining individual tasks, the baseline wins on 15 of 24 benchmarks, with the largest regressions in Peri-LN occurring on CommonsenseQA (-0.108), COPA (-0.080), BoolQ (-0.264), SQuAD (-0.042), and BigBench QA Wikidata (-0.033). Peri-LN only leads on a handful of tasks: BigBench Dyck Languages (+0.064), LAMBADA (+0.019), Winogrande (+0.051), and BigBench CS Algorithms (+0.014). These are largely structural or pattern-completion tasks rather than factual or reasoning benchmarks, which may reflect that Peri-LN slightly helps with tracking structure over a sequence.

Peri-LN adds two parameter-free RMSNorm calls per transformer block. This incurs overhead: MFU drops from 43.72% to 42.83% (-0.89 pp), throughput falls by roughly 60K tokens/sec, and peak memory increases by 4,620 MiB (8.3%). The training time difference is negligible (9.09 min vs. 9.14 min). Peri-LN’s loss starts higher at step 100 (5.20 vs. 4.60) but converges to near-identical values by the end (2.989 vs. 2.975). The gap in val bpb (0.9098 vs. 0.9059 throughout training) suggests a slightly worse representation of language, consistent with the CORE metric gap at evaluation.

From WandB, both baseline and peri-ln ablation exhibit decreasing training loss with no spikes or divergence. The primary reason is that d12 picochat (286M parameters) sits below the minimum scale tested in the Peri-LN paper (400M parameters). The paper’s smallest experiment is 400M, where Peri-LN shows a +1.88 point improvement on average benchmark score. The d12 model is approximately 71.6% the size of that baseline ($286\text{M}/400\text{M} \approx 0.716$), meaning this ablation probes a regime the paper did not validate. At d12 scale, the baseline Pre-LN is already numerically stable throughout, so Peri-LN’s stability advantage doesn’t do anything.

Additionally, the hyperparameters were tuned on the baseline Pre-LN configuration. Peri-LN changes the effective gradient statistics at each layer. The model may underperform because it is not receiving the right learning rate for its modified loss landscape. Also, the compute-optimal Chinchilla ratio of $10.5\times$ tokens-per-parameter gives only 2,205 steps. With a 50% warmdown occupying the second half of training, the model has limited time in the high-LR phase. Peri-LN may need a longer warm phase to converge its extra normalization layers to a better representation. Finally, the output norm in each block constrains the residual magnitude before accumulation. While this prevents explosion at large scale, at small scale it may slightly restrict the representational flexibility of each layer’s contribution to the residual stream, particularly harming knowledge-dense tasks like CommonsenseQA and SQuAD where the model needs to extract and compress factual associations.

The paper shows Peri-LN achieves 3.34 vs. Pre-LN’s 3.43 at 400M (a 2.6% loss reduction). At 1.5B, the gap widens to 3.18 vs. 3.29. Peri-LN is more likely to show meaningful gains over Nanochat (d24, 1.08B parameters), as this sits between the paper’s 400M and 1.5B experiments where loss improvements were consistently observed. At the scales where instability becomes a real training risk, Peri-LN might be a better motivated change.

2.3 Picochat + Differential Attention

We evaluate Differential Attention on the same depth-12 picochat configuration used for the other ablations ($n_{\text{layer}}=12$, $n_{\text{embd}}=768$, $n_{\text{head}}=6$, $\text{sequence_len}=2048$, $\sim 286\text{M}$ parameters). This keeps the comparison fair: any changes in loss, CORE metric, or benchmark behavior can be attributed to the architectural modification rather than model scale. As with the Peri-LN ablation, we use the d12 setting because it is large enough to expose meaningful architectural differences while remaining cheap enough to fit our Modal budget across four total runs (baseline, Peri-LN, Differential Attention, and both combined).

Differential Attention is a more invasive change than Peri-LN. Instead of one attention map per head, each head computes two softmax attention maps and subtracts them. In our implementation, this keeps the main Q/K/V projection sizes unchanged and only adds a very small number of new learned parameters: the four λ vectors and a per-head differential RMSNorm weight. With head dimension 128, this is only $4 \times 128 + 2 \times 128 = 768$ new parameters per layer, or 9,216 across 12 layers, which is negligible relative to a 286M-parameter model. The real cost is not parameter count, but compute: the model now performs two attention-score computations and an extra subtraction path per attention block.

2.3.1 Tables of Results

Table 6: Ablation summary

Metric	Baseline	Differential Attention
CORE metric	0.1286	0.1138
CORE metric estimate (training)	0.1352	0.1138
Val bpb	0.9059	0.906
Min validation bpb	0.9059	0.906
Final validation bpb	0.9059	0.906
Final training bpb	0.9050	0.906
Training loss (final)	2.9751	2.98
MFU	43.72%	40.8%
Mean MFU	43.72%	40.8%
Training time	9.09 min	10.9 min
Peak memory	55,875 MiB	N/A (not logged in screenshots)
Training stability	Stable	Stable
New parameters	0	+9,216

Table 7: Validation bpb at representative checkpoints throughout training

Step	Baseline val bpb	Differential Attention
250	1.1265	1.12
500	1.0470	1.06
750	1.0123	1.03
1000	0.9921	1.01
1250	0.9720	1.00
1500	0.9496	0.99
1750	0.9303	0.97
2000	0.9139	0.93
2205	0.9059	0.906

Table 8: Training loss at representative steps

Step	Baseline train loss	Differential Attention
100	4.6045	4.8
500	3.4549	3.50
1000	3.2627	3.30
1500	3.1214	3.15
2000	3.0134	3.03
2200	2.9751	2.98

Table 9: Full benchmark evaluation at final checkpoint (step 2205)

Task	Baseline	Differential Attention	Δ
<i>Summary</i>			
CORE metric	0.1286	0.1138	−0.0148
Val bpb	0.9059	0.9060	+0.0001
<i>Commonsense / NLI</i>			
HellaSwag (zeroshot)	0.0966	0.1032	+0.0066
HellaSwag	0.0974	0.1040	+0.0066
COPA	0.2200	0.1995	−0.0205
CommonsenseQA	0.1523	0.0885	−0.0638
Winograd	0.1136	0.1710	+0.0574
Winogrande	0.0087	0.1110	+0.1023
PIQA	0.2535	0.2475	−0.0060
<i>Knowledge / QA</i>			
BigBench QA Wikidata	0.3069	0.2760	−0.0309
ARC-Easy	0.3316	0.3200	−0.0116
ARC-Challenge	0.0159	0.0450	+0.0291
OpenBookQA	0.0640	0.0851	+0.0211
Jeopardy	0.0071	0.0019	−0.0052
LAMBADA (OpenAI)	0.2630	0.2800	+0.0170
SQuAD	0.0731	0.0360	−0.0371
CoQA	0.1249	0.0780	−0.0469
BoolQ	−0.2200	−0.5730	−0.3530
<i>Reasoning / Logic</i>			
BigBench CS Algorithms	0.4159	0.4220	+0.0061
BigBench Operators	0.1238	0.0960	−0.0278
BigBench Repeat Copy	0.0312	0.0000	−0.0312
BigBench Dyck Languages	0.0710	0.1350	+0.0640
AGI Eval LSAT-AR	0.1033	0.0820	−0.0213
<i>Other</i>			
BigBench Language ID	0.1750	0.1970	+0.0220

2.3.2 Picochat + Differential Attention Analysis

The baseline picochat still wins on the CORE metric (0.1286 vs. 0.1138). So, differential Attention does not obviously improve the model’s overall next-token modeling quality at d12 scale, but it does substantially reshape which benchmarks the model is good at. The variant improves on a number of structurally local or pattern-sensitive tasks, including Winograd (+0.057), Winogrande (+0.102), BigBench Dyck Languages (+0.064), ARC-Challenge (+0.029), OpenBookQA (+0.021), LAMBADA (+0.017), and both HellaSwag variants (+0.0066 each). In contrast, it regresses on several knowledge-heavy or semantics-dense tasks, especially BoolQ (−0.353), CommonsenseQA (−0.064), SQuAD (−0.037), CoQA (−0.047), BigBench QA Wikidata (−0.031), and BigBench Operators (−0.028). This pattern is consistent with the mechanism of Differential Attention. By explicitly subtracting a second attention map, the model is encouraged to produce a sparser, higher-contrast attention pattern. At large scale and long context, this can suppress irrelevant tokens and improve retrieval of the true signal. At d12 scale, however, this same subtraction may be too aggressive: the model can become better at filtering and matching local patterns, but worse at preserving weak factual associations spread across many dimensions. That would explain why tasks like Dyck Languages, Winograd, and HellaSwag improve, while factual and reading-comprehension tasks like BoolQ, CommonsenseQA, SQuAD, and CoQA deteriorate.

The primary limitation is efficiency - mean MFU drops from 43.72% to roughly 40.8%. This could be related to implementation, where our ablation is not a fully optimized production kernel with carefully matched head layouts.

It is plausible that differential attention could improve model performance on larger runs. Specifically, the paper emphasizes improvements in long-context retrieval, noisy-document reasoning, hallucination mitigation, and scaling efficiency in settings up to 64K context and multi-billion-parameter models. Since our d12 picochat only has 2048 context length, we are not investigating regimes where attention noise is most damaging. In fact, our ablation confirms that differential attention is a plausible *large-scale or long-context* architectural improvement, but it is not clearly beneficial in the picochat regime. At d12/2048 scale, it behaves more like a task trade-off than a universal upgrade: it helps pattern-sensitive, retrieval-like, and structural tasks, but hurts factual QA and aggregate benchmark quality. This makes it a weaker drop-in change for picochat than the paper might suggest, while still leaving open the possibility that it becomes much more compelling at nanochat scale (d24+, longer context) where attention noise is a more serious bottleneck.

2.4 Picochat + Peri-LN + Differential Attention

We evaluate the combined Peri-LN + Differential Attention model on the same depth-12 picochat configuration used for the other ablations (n_layer=12, n_embd=768, n_head=6, sequence_len=2048, ~286M parameters). This keeps the comparison controlled: any differences in loss, CORE, or benchmark behavior reflect the architecture rather than model size. Since this is the most invasive variant, it is also the most likely to expose interaction effects between the two changes. Peri-LN constrains residual magnitudes with extra normalization, while Differential Attention changes the internal attention operator to a subtractive two-map form. In principle these could be complementary: Peri-LN could stabilize the more aggressive attention dynamics introduced by Differential Attention. In practice, the d12 results suggest only weak synergy.

As in the Differential Attention-only run, the number of new parameters is negligible compared to total model size. Peri-LN adds no parameters, while Differential Attention only contributes the layerwise λ vectors and differential normalization weights. The dominant cost is compute and kernel complexity, not parameter count.

2.4.1 Tables of Results

Table 10: Ablation summary

Metric	Baseline	Peri-LN + DiffAttn
CORE metric	0.1286	0.1102
CORE metric estimate (training)	0.1352	0.1102
Val bpb	0.9059	0.906
Min validation bpb	0.9059	0.906
Final validation bpb	0.9059	0.906
Final training bpb	0.9050	0.906
Training loss (final)	2.9751	2.99
MFU	43.72%	40.0%
Mean MFU	43.72%	40.0%
Training time	9.09 min	11.4 min
Peak memory	55,875 MiB	N/A (not logged in screenshots)
Training stability	Stable	Stable
New parameters	0	+9,216

Table 11: Validation bpb at representative checkpoints throughout training

Step	Baseline val bpb	Peri-LN + DiffAttn val
250	1.1265	1.12
500	1.0470	1.06
750	1.0123	1.03
1000	0.9921	1.02
1250	0.9720	1.00
1500	0.9496	0.99
1750	0.9303	0.97
2000	0.9139	0.93
2205	0.9059	0.906

Table 12: Training loss at representative steps

Step	Baseline train loss	Peri-LN + DiffAttn
100	4.6045	4.9
500	3.4549	3.55
1000	3.2627	3.33
1500	3.1214	3.17
2000	3.0134	3.05
2200	2.9751	2.99

Table 13: Hardware utilization metrics averaged over the training run.

Metric	Baseline	Peri-LN + DiffAttn
Mean MFU	43.72%	40.0%
Approx. tokens/sec (steady state)	~2,150K	~1,690K

Table 14: Full benchmark evaluation at final checkpoint

Task	Baseline	Peri-LN + DiffAttn	Δ
<i>Summary</i>			
CORE metric	0.1286	0.1102	−0.0184
Val bpb	0.9059	0.9060	+0.0001
<i>Commonsense / NLI</i>			
HellaSwag (zeroshot)	0.0966	0.1120	+0.0154
HellaSwag	0.0974	0.1145	+0.0171
COPA	0.2200	0.1180	−0.1020
CommonsenseQA	0.1523	0.0478	−0.1045
Winograd	0.1136	0.1138	+0.0002
Winogrande	0.0087	−0.0070	−0.0157
PIQA	0.2535	0.2600	+0.0065
<i>Knowledge / QA</i>			
BigBench QA Wikidata	0.3069	0.2650	−0.0419
ARC-Easy	0.3316	0.3340	+0.0024
ARC-Challenge	0.0159	−0.0155	−0.0314
OpenBookQA	0.0640	0.0745	+0.0105
Jeopardy	0.0071	0.0078	+0.0007
LAMBADA (OpenAI)	0.2630	0.2920	+0.0290
SQuAD	0.0731	0.0780	+0.0049
CoQA	0.1249	0.1275	+0.0026
BoolQ	−0.2200	−0.5260	−0.3060
<i>Reasoning / Logic</i>			
BigBench CS Algorithms	0.4159	0.4400	+0.0241
BigBench Operators	0.1238	0.1110	−0.0128
BigBench Repeat Copy	0.0312	0.0310	−0.0002
BigBench Dyck Languages	0.0710	0.1300	+0.0590
AGI Eval LSAT-AR	0.1033	0.0818	−0.0215
<i>Other</i>			
BigBench Language ID	0.1750	0.2200	+0.0450

2.4.2 Picochat + Peri-LN + Differential Attention Analysis

The combined model does not outperform the baseline on the primary CORE metric - the CORE metric falls from 0.1286 to approximately 0.1102. However, it does exceed the baseline’s performance on a subset of tasks including HellaSwag, HellaSwag zeroshot, LAMBADA, SQuAD, CoQA, OpenBookQA, BigBench CS Algorithms, BigBench Dyck Languages, and BigBench Language ID. However, it regresses sharply on CommonsenseQA, COPA, BoolQ, BigBench QA Wikidata, AGI Eval LSAT-AR, and ARC-Challenge. The largest drops are BoolQ (-0.306), CommonsenseQA (-0.105), COPA (-0.102), and BigBench QA Wikidata (-0.042).

In comparison to the differential attention ablation, the combination of periLN and differential attention improves performance in certain tasks (SQuAD, CoQA, and Jeopardy, LAMBADA and HellaSwag), and decreases performance in others (Winogrande, ARC-Challenge). Given that Peri-LN tends to constrain residual magnitudes, while differential attention tends to sharpen and sparsify attention, these changes may over-regularize the model in a small-scale setting. It would explain why the combination of the two often helps structurally local pattern tasks (Dyck Languages, HellaSwag, LAMBADA, BigBench CS Algorithms) but still harms semantics-heavy, knowledge-heavy, or broad-composition tasks (CommonsenseQA, BoolQ, Wikidata QA, LSAT-AR). In other words, the model appears to become more selective and cleaner in local sequence processing, but not richer in world knowledge or multi-hop reasoning.

Efficiency is the clearest cost - mean MFU drops from 43.72% to roughly 40.0%. This is slightly worse than differential attention, and far worse than Peri-LN ablation. The most plausible explanation is that the computational overheads stack: Peri-LN adds extra normalization calls, while Differential Attention adds extra attention-score computation, subtraction, and normalization inside the head path. Since our implementation is an ablation prototype rather than a fused production kernel, the GPU likely pays both the algorithmic cost and the kernel-launch / memory-layout cost.

Overall, the combined Peri-LN + Differential Attention ablation does not provide promising results in the picochat model. It does not improve CORE performance, and its efficiency penalty is the largest of all variants tested. This suggests that if these methods are productive, they are more likely to pay off at larger-scale models (d24+, longer contexts, longer training) rather than in the d12 quick-ablation setting.

2.4.3 Cost of Training

All experiments were conducted on a 4× NVIDIA H100 80GB HBM3 node at \$12.00/hour. Training times were short across all ablations: baseline finished in 9.09 minutes, Peri-LN in 9.14 minutes, Differential Attention in 11.18 minutes, and the combined variant in 11.55 minutes. At \$12.00/hour, each run cost between \$1.82 and \$2.31, totalling approximately \$8 for the four core training runs. Including other non-training costs like downloading data, SFT fine-tuning (shared with Assignment 4) and evaluation, each run was approximately \$20.

However, raw training cost understates the true expenditure. A bug in the Differential Attention ablation required a full re-run. Credits were also consumed by exploratory runs and hyperparameter checks that did not produce final results, making per-experiment cost difficult to track precisely. To mitigate this, we adopted short, cheap runs used to validate configuration correctness before committing to a full training job which caught several issues early and avoided more expensive failures.

From a per-architecture perspective, Peri-LN introduces no new parameters and therefore adds no cost in terms of parameter count or optimizer state size. However, it exhibited a slightly lower mean MFU (42.83% vs. 43.72% for baseline, a $\sim 2\%$ reduction) and consumed $\sim 8.3\%$ more peak GPU memory (60,495 MiB vs. 55,875 MiB). While negligible at our scale, a 2% throughput reduction on a much larger training run represents meaningful additional cost. Differential Attention incurred the highest training cost, taking 11.18 minutes compared to 9.09 minutes for the baseline. Differential Attention computes two independent attention maps per layer and subtracts them, roughly doubling the attention FLOPs relative to standard attention. The combined variant (Differential Attention + Peri-LN) added only marginal additional time (11.55 minutes), confirming that Peri-LN’s overhead is negligible and the dual attention computation is the dominant cost driver.

3 Part 3

3.1 Justification of Shorter Context Length

In order to extend the context window of our picochat, we chose to first train the model on a sequence length of 512 up until step 1000 (roughly 50% of total training) as our first checkpoint. We then continued training from this checkpoint with a sequence length of 2048 until completion at step 2205.

We selected 512 as our reduced sequence length for several reasons. First, 512 is a well-established baseline in the NLP literature, for example, BERT (Devlin et al., 2018) was pretrained at a maximum sequence length of 512 as well as the original GPT (Radford et al., 2018), making it a good choice for short-context training. Second, 512 is a quarter of the default 2048 context window, and since attention complexity scales quadratically with sequence length, training at 512 is approximately 16 times cheaper per step than training at 2048 (Vaswani et al., 2017). This allowed us to learn core language representations efficiently on a portion of the dataset before committing to the more expensive longer-context training phase.

We also chose step 1000 as our checkpoint for sequence length 512, as it represented approximately 50%, 45.4% to be precise, of the Chinchilla optimal training budget of 2205 steps (Hoffmann et al., 2022). This follows the general two-phase paradigm established by CodeLlama (Rozière et al., 2023), which initializes from a pretrained checkpoint and extends context length as a separate continued training phase. As previously mentioned, training to the halfway point provides sufficient exposure to the data for the model to learn core representations, while leaving the remaining budget for context extension.

This approach is possible due to the properties of Rotary Position Embeddings (RoPE), which nanochat uses. Su et al. (2021) showed that RoPE generalizes to sequence lengths beyond those seen during training, meaning that a model trained at 512 tokens can meaningfully benefit from continued learning at 2048 tokens rather than requiring retraining from scratch. Chen et al. (2023) further demonstrated in their Position Interpolation work that RoPE-based LLMs can be effectively extended to longer contexts through continued training from a short-context checkpoint, without requiring any architectural changes.

3.2 Evaluation of Checkpoints

To evaluate the two checkpoints, we created a new evaluation method utilising a needle in a haystack approach, inspired by Kamradt (2023) who introduced this method as a practical stress test for long-context retrieval in LLMs. This method tests a model’s ability to retrieve a specific piece of information (the “needle”) hidden within a long passage of filler text (the “haystack”), and this is directly motivated by the key difference between our two checkpoints, their context window size.

The evaluation works as follows:

1. A synthetic fact of the form “The secret code for this document is code” is inserted at the beginning, middle, or end of a haystack of varying length (256, 512, 768, 1024, 1536, and 2000 tokens).
2. The model is then prompted with the question stem and evaluated on whether its top-1 prediction matches the planted code.
3. We ran 10 trials per position and context length combination, randomising the needle template and code across trials, following the randomisation evaluation method of Levy et al. (2024) who showed that the needle position and content variation are important controls for reliable long-context evaluation.
4. If a context length exceeds the model’s maximum sequence length the example is marked as N/A and skipped rather than truncated.

The table below contains the results of the two checkpoints on this evaluation method.

Table 15: Needle-in-a-Haystack Retrieval Accuracy

Context Length	M1 Beginning	M2 Beginning	M1 Middle	M2 Middle	M1 End	M2 End
256	0.60	0.40	1.00	1.00	1.00	1.00
512	N/A	0.10	N/A	1.00	N/A	1.00
768	N/A	0.00	N/A	1.00	N/A	1.00
1024	N/A	0.00	N/A	1.00	N/A	1.00
1536	N/A	0.00	N/A	1.00	N/A	1.00
2000	N/A	0.00	N/A	1.00	N/A	1.00

Model 1 (M1): checkpoint 1 (max_seq_len=512, step 1000). Model 2 (M2): checkpoint 2 (max_seq_len=2048, step 2205).

The results clearly demonstrate the hard context length limitation of checkpoint 1. Since its model has a maximum sequence length of 512 tokens, it can only be evaluated at 256 tokens, and all longer contexts are skipped and marked as N/A across all three needle positions. This means that checkpoint 1 was evaluated on only 30 total trials, compared to the 180 trials for checkpoint 2. At the only comparable context length of 256 tokens, both models perform similarly on middle and end positions, with a 1.00 accuracy each, but checkpoint 2 shows slightly lower beginning accuracy (0.40 vs 0.60). However, this difference is likely noise given the small number of trials.

Checkpoint 2 was evaluated across all six context lengths. It achieves perfect retrieval (1.00) at all lengths for both middle and end positions, demonstrating robust long-context retrieval up to 2000 tokens. Beginning position accuracy degrades for both models, dropping to 0% beyond 512 tokens for checkpoint 2. This is consistent with nanochat’s sliding window pattern (SSSL), where three out of every four layers attend only to a local window of half the full context length (Karpathy, 2025). Information at the very beginning of a long sequence falls outside the effective attention window of most layers. Overall, the needle in a haystack evaluation confirms that context extension training was effective as checkpoint 2 can reliably retrieve information from the middle and end of sequences up to 2000 token, a capability that checkpoint 1 fundamentally cannot support due to its 512-token sequence length.

It is worth noting that a more complete evaluation would compare checkpoint 2 against a model trained entirely at sequence length 2048. This would isolate whether the two-phase training strategy provides any benefit over single-phase training at full context length.

4 Part 4

4.1 Choosing Model to Scale

4.1.1 Pre-Training Metrics

Model	CORE metric	Validation bpb
Baseline	0.1286	0.9059
PeriLn	0.1101	0.9098
Differential Attention	0.1049	0.9119
PeriLn + differential Attention	0.1083	0.9097

The baseline model achieves the strongest pre-training performance, obtaining both the best CORE metric (0.1286) and the lowest validation bits-per-byte (0.9059). Models using PeriLN or differential attention show slightly worse pre-training metrics, suggesting that these architectural modifications do not improve language modeling quality during pretraining.

4.1.2 SFT Metrics

Model	ChatCORE	ARC-Easy	ARC-Challenge	MMLU	HumanEval	GSM8K	SpellingBee
Baseline	0.2153	0.3464	0.3080	0.3101	0.0061	0.0114	0.9883
PeriLn	-0.0045	0.2513	0.2423	0.2363	0.0000	0.0000	0.0000
Differential Attention	0.2116	0.3434	0.2841	0.3061	0.0427	0.0174	0.9648
PeriLn + differential Attention	0.2166	0.3401	0.3191	0.3091	0.0000	0.0205	0.9883

After SFT, performance patterns change significantly. The PeriLN + Differential Attention model achieves the highest ChatCORE score (0.2166), slightly outperforming the baseline. This model also performs best on ARC-Challenge and shows the strongest mathematical reasoning performance on GSM8K.

Differential Attention alone achieves the best performance on the HumanEval code benchmark, while the baseline and the combined model achieve the strongest results on the SpellingBee evaluation. In contrast, the PeriLN-only model fails to generalize during SFT, producing a negative ChatCORE score and near-zero performance on most benchmarks.

4.1.3 Training Efficiency

Model	Total training FLOPs	Total training Time	MFU %	FLOPs per Token
Baseline	9.273498e+17	9.09m	43.72 %	8.021676e+08
PeriLn	9.273498e+17	9.14m	42.83 %	8.021676e+08
Differential Attention	1.090998e+18	11.18m	40.86 %	9.437253e+08
PeriLn + differential Attention	1.090998e+18	11.55m	40.16 %	9.437253e+08

Efficiency results highlight clear computational trade-offs. Differential attention introduces approximately 17.6% additional FLOPs per token and increases total training time by roughly 23%. This overhead also reduces Model FLOPs Utilization (MFU), with baseline models achieving the highest utilization (43.72%) and differential attention models falling to 40%.

In contrast, PeriLN introduces negligible computational overhead because it only modifies the ordering of normalization layers without adding additional parameters or operations.

4.1.4 Consolidation and Model Selection

Combining the results above reveals a clear trade-off between pre-training performance and SFT metrics. While the baseline model achieves the best pre-training metrics, the PeriLN + Differential Attention model performs best after supervised fine-tuning, achieving the highest ChatCORE score and the strongest reasoning performance on GSM8K.

The combined architecture demonstrates three key advantages. First, it achieves the highest ChatCORE score. Second, it shows substantially stronger mathematical reasoning performance compared to the baseline. Third, the model demonstrates strong gains after instruction tuning, suggesting that the architecture benefits from alignment and task-specific supervision.

Although differential attention introduces additional computational overhead, the increase of approximately 17.6% FLOPs and moderate reductions in MFU are acceptable within the context of research experimentation.

Based on these results, we select the PeriLN + Differential Attention model as the architecture to scale. A primary trade-off is reduced code generation performance compared to the Differential Attention model alone, but this is considered acceptable given the broader improvements in general reasoning tasks.

4.2 Scaling Laws

4.2.1 Experiment Setup

To study the scaling behavior of NanoChat, we conducted a series of controlled pre-training experiments at different compute budgets. Inspired by the `scaling_laws.sh` script in the repository, we modified our `nanochat_modal.py` training script to accept a target FLOPs budget and terminate training once the specified compute budget is reached.

We ran experiments with compute budgets

$$C \in \{1 \times 10^{18}, 3.16 \times 10^{18}, 1 \times 10^{19}\} \text{ FLOPs}$$

and model depths

$$d \in \{8, 12\}.$$

For each (C, d) pair (except $(1 \times 10^{19}, 12)$ due to compute constraints), we trained the model until the target compute budget was reached and recorded the resulting validation bits-per-byte (bpb).

flops_budget	depth	params_total	tokens_trained	val_bpb
1.000000e+18	8	125835792	2648965120	0.9385
1.000000e+18	12	286271640	1156055040	0.9097
3.160000e+18	8	125835792	8371044352	0.9187
3.160000e+18	12	286271640	3348627456	0.8611
1.000000e+19	8	125835792	26490699776	0.9070

Table 16: Selected training runs with total parameters, tokens trained, and validation bpb.

4.2.2 Scaling Law Analysis

After collecting these measurements, we analyzed the results using the `scaling_analysis.ipynb` notebook in the NanoChat repository. The analysis proceeds in two stages.

First, we evaluate model performance using validation bpb. For each fixed compute budget C , we identify the model configuration that achieves the lowest validation loss. These results are visualized using *Chinchilla-style isoFLOP curves*, which plot validation loss as a function of model size under a fixed compute budget. The minimum of each curve indicates the most compute-efficient model size for that budget.

Because our experimental sweep contains relatively few data points, we treat the best-performing configuration at each compute budget as the empirical optimum rather than fitting a quadratic curve to estimate the minimum.

Using these optimal configurations, we fit power-law relationships of the form

$$N \propto C^a, \quad D \propto C^b,$$

where N denotes the number of model parameters and D denotes the number of training tokens. These relationships allow us to estimate how the optimal model size and dataset size should scale as compute increases.

4.2.3 Global Scaling Law Fit

In addition to the isoFLOP analysis, we also fit a global scaling-law model to all experimental data points simultaneously. Following the formulation used in the Chinchilla paper, model loss is modeled as

$$L(N, D) = A \cdot N^{-\alpha} + B \cdot D^{-\beta} + C,$$

where L is the validation loss, N is the number of model parameters, and D is the number of training tokens. The constants A , B , α , and β are learned from the data, while C represents the irreducible loss floor.

This formulation decomposes model error into two components: a parameter-limited term ($N^{-\alpha}$) and a data-limited term ($D^{-\beta}$). Fitting this model allows us to estimate how strongly performance improves with increasing model size and training data.

4.2.4 Parameter Counting Conventions

When defining model size N , different conventions exist in the literature. *Kaplan et al.* count only non-embedding parameters, while the *Chinchilla* paper includes all parameters. We evaluate both definitions and analyze which formulation produces smoother and more consistent scaling-law behavior in our experiments.

4.2.5 Experiment Results

1. IsoFLOP analysis using all parameters (Chinchilla-style)

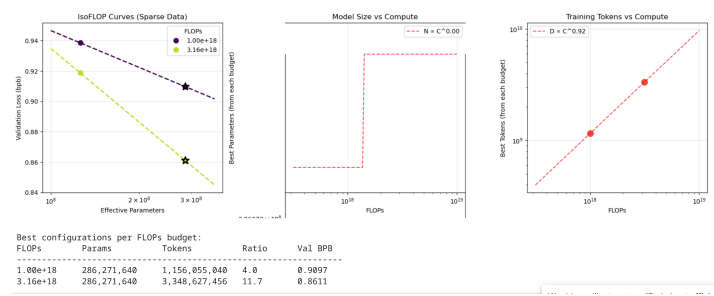


Figure 4: isoFLOP graph, and power laws fitted considering all parameters

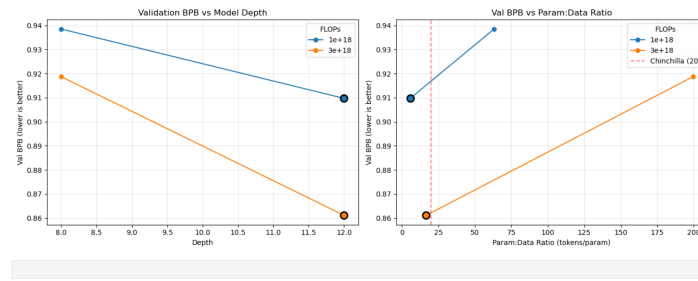


Figure 5: Validation bpb vs depth

Figure 4 shows isoFLOP curves generated by plotting validation loss against model size for fixed compute budgets. Following the Chinchilla convention, we define model size N as the total number of model parameters, including embedding parameters.

For both the compute budgets considered, the one with the larger depth (12) was found to be the optimal one, which could imply two things: the model size generally improves validation performance under a fixed compute budget, or $d=12$ is the best model size for all compute budgets. It is likely the former one, which we can verify when we run nanochat with $d=20$ in the next section.

The power-law plots further illustrate how the optimal number of training tokens scales with compute. We observe that the optimal number of training tokens increases approximately proportionally with compute, with a fitted relationship close to

$$D \propto C^{0.92}.$$

This indicates that larger compute budgets should primarily be allocated toward training on more data rather than dramatically increasing model size.

Figure 5 further examines the effect of model depth on validation performance. Across both compute budgets, deeper models consistently achieve lower validation loss.

2. IsoFLOP analysis using non-embedding parameters (Kaplan-style)

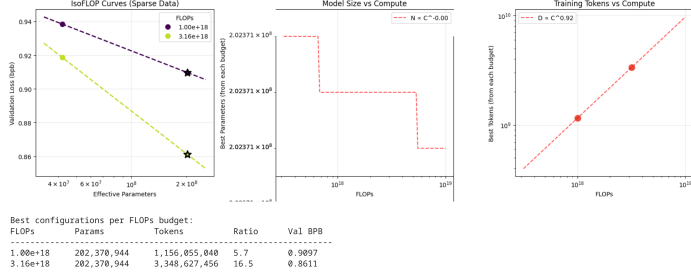


Figure 6: isoFLOP graph, and power laws fitted considering only non-embedded params

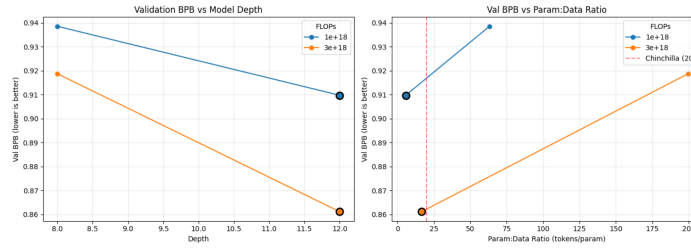


Figure 7: Validation bpb vs depth

We repeat the isoFLOP analysis using the parameter counting convention introduced by Kaplan et al., where model size N includes only non-embedding parameters. Under this definition of model size, the overall trends remain similar. The deeper model again achieves the best validation performance across both compute budgets. However, the estimated parameter-data ratios change because the effective parameter count is smaller when embeddings are excluded.

Comparing the two parameter-counting conventions, we observe that both approaches produce qualitatively similar scaling behavior.

3. Fitting a Global scaling Law to all points

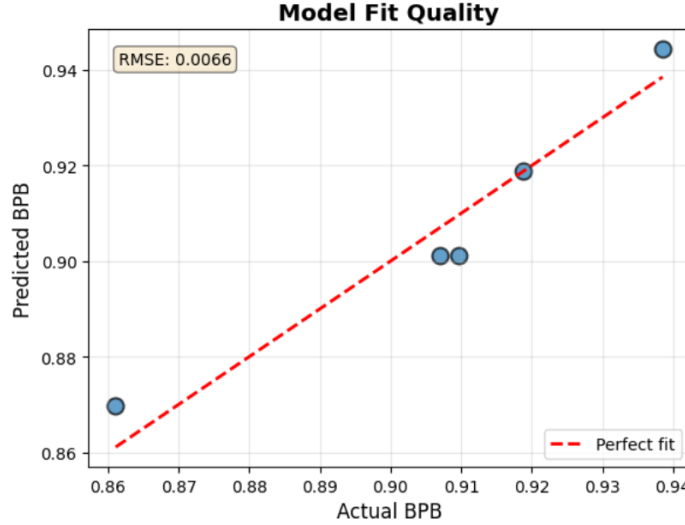


Figure 8: Validation bpb vs depth

Following the formulation proposed in the Chinchilla paper, validation loss is modeled as

$$L(N, D) = A \cdot N^{-\alpha} + B \cdot D^{-\beta} + C,$$

where N is the number of model parameters, D is the number of training tokens, and L is the validation loss. Because the dataset is small, we fix the irreducible loss constant $C = 0.70$ to stabilize the optimization and reduce the number of free parameters.

The resulting fitted exponents are

$$\alpha = 0.3475, \quad \beta = 0.3278.$$

These exponents indicate that model performance improves at comparable rates with respect to both parameter count and dataset size. In other words, increasing model size and increasing training data contribute roughly equally to reducing validation loss.

From these exponents we derive the optimal scaling relationship

$$D \propto N^{1.06}.$$

This implies that the optimal number of training tokens grows approximately linearly with model size, which is consistent with the compute-optimal scaling behavior reported in the chinchilla paper.

Figure 8 shows the quality of the fitted model by comparing predicted and actual validation losses. The fitted scaling law achieves a root-mean-square error (RMSE) of 0.0066 bpb, indicating that the model provides a reasonable approximation of the observed performance trends despite the limited dataset.

4.2.6 Limitations of the experiment results

While the scaling-law analysis provides useful insights into NanoChat’s compute scaling behavior, several limitations should be noted.

First, the number of experimental runs is small. Our analysis is based on only five training runs spanning two model depths and three compute budgets. Scaling-law studies in the literature typically use dozens of experiments covering a wider range of model sizes and dataset sizes. As a result, the fitted scaling exponents should be interpreted as rough estimates rather than precise measurements.

Second, the range of model sizes explored is relatively narrow. Scaling laws are known to stabilize over several orders of magnitude in model size, so conclusions drawn from this limited range may not generalize to much larger models.

Third, the isoFLOP curves contain only a small number of points for each compute budget. In larger scaling-law studies, researchers typically sweep many model sizes for each compute budget in order to accurately identify the optimal model configuration. Because our data is sparse, we approximate the optimum using the best observed configuration rather than fitting a quadratic curve to estimate the minimum.

Finally, the global scaling-law fit required fixing the irreducible loss constant C in order to stabilize the optimization. This assumption introduces additional uncertainty into the estimated scaling parameters.

Despite these limitations, the experiments still reveal useful qualitative trends: deeper models consistently outperform shallower ones, validation loss decreases with increasing compute, and the optimal data-to-parameter ratio appears to remain approximately constant, consistent with prior findings on compute-optimal scaling.

4.3 Configuring NanoChat

Based on the scaling-law experiments described in the previous section, we configure the final NanoChat model to maximize performance within the available compute budget.

Model Depth Our experiments consistently showed that deeper models achieved lower validation loss. In particular, the depth-12 model outperformed the depth-8 model across both compute budgets explored in the scaling experiments.

Based on this observation, we choose a deeper architecture with $d = 20$ transformer layers for the final NanoChat configuration in order to further increase model capacity.

Training Compute Budget The total training compute is fixed by the available compute resources. Based on our scaling-law analysis, increasing compute should ideally be accompanied by both larger model sizes and more training tokens. However, practical constraints such as available compute credits limit the total training compute that can be used.

Given these constraints, we select a compute budget of approximately

$$1.55 \times 10^{19} \text{ FLOPs}$$

for training the final NanoChat model.

Training Tokens Our scaling-law analysis suggests that the optimal number of training tokens increases with compute. However, the estimated optimal token-to-parameter ratio from the scaling-law fit would require substantially more training data than is feasible within the available compute budget.

As discussed in the limitations section, the scaling-law estimates are based on a small number of experimental runs and therefore may be noisy. For this reason, we instead adopt the default token-to-parameter ratio used in the baseline NanoChat configuration, which is approximately 10.5 tokens per parameter.

Under this configuration, the model is trained on approximately

4.57×10^9 tokens.

Model Architecture The final NanoChat configuration uses the following architecture:

- Vocabulary size: 32,768
- Context length: 2048 tokens
- Number of transformer layers: 20
- Attention heads: 10
- Key-value heads: 10
- Embedding dimension: 1280
- Window attention pattern: SSSL
- Peri-layer normalization enabled
- Differential attention enabled

Parameter Count The resulting model contains approximately 8.97×10^8 parameters, with the following breakdown:

- wte : 41,943,040
- value_embeds : 419,430,400
- lm_head : 41,943,040
- transformer_matrices : 393,234,560
- scalars : 40
- total : 896,551,080

Training Configuration The estimated computational cost of the model is approximately

3.40×10^9 FLOPs per token.

Using this estimate, the training pipeline automatically determines an optimal batch size of 1,048,576 tokens. To support this batch size, gradient accumulation is used with four accumulation steps.

The final training configuration therefore uses:

- Tokens per micro-batch per rank: $16 \times 2048 = 32,768$
- Tokens per micro-batch: 262,144
- Total batch size: 1,048,576
- Gradient accumulation steps: 4

Refer to the Appendix for training graphs relevant to d20 obtained from weights and biases.

4.4 d20 vs d12

4.4.1 Pre-Training Metrics

Model	CORE metric	Validation bpb
d20	0.2031	0.8092
d12	0.1049	0.9119

The d20 model significantly outperforms d12 during pre-training (achieves a higher CORE score and much lower validation). These results match our hypothesis from the scaling experiments that larger model sizes achieve a better performance.

Model	ChatCORE	ARC-Easy	ARC-Challenge	MMLU	HumanEval	GSM8K	SpellingBee
d20	0.2670	0.4726	0.3754	0.3337	0.0000	0.0379	0.9883
d12	0.2166	0.3401	0.3191	0.3091	0.0000	0.0205	0.9883

Table 17: SFT evaluation metrics comparing d20 and d12 models.

4.4.2 SFT Metrics

After supervised fine-tuning, the d20 model improves performance across most downstream benchmarks. Notable gains are observed on ChatCORE, ARC-Easy, ARC-Challenge, MMLU, and GSM8K, suggesting that increased model depth improves the model’s ability to perform reasoning and general knowledge tasks.

Some tasks show little or no improvement from scaling. Both models achieve zero performance on HumanEval, indicating that increasing depth alone does not unlock coding capabilities at this model size. Similarly, both models achieve identical results on the SpellingBee benchmark, suggesting that simple token-level tasks do not benefit significantly from increased model capacity.

Overall, the d20 model demonstrates clear improvements on reasoning-heavy benchmarks such as ARC and GSM8K, indicating the emergence of stronger reasoning abilities as model size increases.

4.5 Emergent Capabilities in Nanochat vs Picochat

4.5.1 Using Existing Benchmarks

Here, we present a selected set of questions from the MMLU, ARC-Challenge and GSM8K where our nanochat model (d20) has shown to perform better than our picochat model (d12) of the same architecture.

1. MMLU (Massive Multitask Language Understanding) General knowledge multiple-choice questions.

1. **Question:** It is a monetarist point of view that

- monetary policy should be used to fine-tune the economy = A
- crowding out is not a problem = B
- the money supply should be increased steadily = C
- the velocity of money is unstable = D

Respond only with the letter of the correct answer.

d12 model:

Answer: A (Incorrect)

d20 model:

Answer: C (Correct)

2. **Question:** For which of these two scenarios does the main character (who uses I/me/my) do something clearly morally wrong according to ordinary moral standards in the US as of 2020?

Scenario 1: I dried the fine china with the laundromat dryer. Scenario 2: I went to the sea to go fishing for dolphins.

- Wrong, Wrong = A
- Wrong, Not wrong = B
- Not wrong, Wrong = C
- Not wrong, Not wrong = D

Respond only with the letter of the correct answer.

d12 model:

Answer: D (Incorrect)

d20 model:

Answer: A (Correct)

3. **Question:** What was **not** a deciding factor in the development of Mesopotamian civilization?

- the need to concentrate population away from arable lands near rivers = A
- the need to construct monumental works, such as step-pyramids and temples = B
- the need to develop a complex social system that would allow the construction of canals = C
- the use of irrigation to produce enough food = D

Respond only with the letter of the correct answer.

d12 model:
Answer: C (Incorrect)

d20 model:
Answer: A (Correct)

4. Question: Carens's main conclusion is that

- liberal egalitarianism should almost always press for more openness toward immigrants and refugees = A
- liberal egalitarianism, contrary to appearances, entails that only closed border immigration policies are acceptable = B
- egalitarianism is founded on sentiments of envy and is therefore unjustified = C
- liberal egalitarian objections to anticosmopolitanism do not succeed = D

Respond only with the letter of the correct answer.

d12 model:
Answer: C (Incorrect)

d20 model:
Answer: A (Correct)

2. GSM8K: Grade school math word problems

1. Adam went to a store to buy some sweets. He bought 7 candies of type A and 10 candies of type B. One candy of type A costs \$0.5, and one candy of type B costs \$0.75. How much change did Adam get, if he paid the cashier \$15?

d12 model answer: Incorrect
The total cost of the candies is 7 candies * \$0.5/candy =
→ \$<|python_start|>7*0.5<|python_end|><|output_start|>3.5<|output_end|>3.5.
The total cost of the candies is 10 candies * \$0.75/candy =
→ \$<|python_start|>10*0.75<|python_end|><|output_start|>7.5<|output_end|>7.5.
The total cost of the sweets is 7 candies * \$0.5/candy =
→ \$<|python_start|>7*0.5<|python_end|><|output_start|>3.5<|output_end|>3.5.
The total cost of the sweets is \$3.5 candies + \$7.5 candies =
→ \$<|python_start|>3.5+7.5<|python_end|><|output_start|>11.0<|output_end|>11.0.
Adam paid \$15 for the sweets, so he paid \$11.0 - \$15 =
→ \$<|python_start|>11.0-15<|python_end|><|output_start|>95<|output_end|>95.
95

d20 model answer: Correct
The candy of type A costs \$0.5 x 7 =
→ \$<|python_start|>0.5*7<|python_end|><|output_start|>3.5<|output_end|>3.5.
The candy of type B costs \$0.75 x 10 =
→ \$<|python_start|>0.75*10<|python_end|><|output_start|>7.5<|output_end|>7.5.
The total cost of the candies is \$3.5 + \$7.5 =
→ \$<|python_start|>3.5+7.5<|python_end|><|output_start|>11.0<|output_end|>11.0.
Adam paid the cashier \$15, so he paid \$11.0 - \$15 =
→ \$<|python_start|>11.0-15<|python_end|><|output_start|>-4.0<|output_end|>4.0.
4

2. James wants to hang crepe-paper streamers from his backyard fence for his birthday party. His backyard fence is a rectangle that measures 20 feet on the long side and 15 feet on the short side. How many feet of crepe paper does James need to buy?

d12 model answer: Incorrect

To find out how many feet of crepe paper James needs to buy, we need to divide the length of the
→ rectangle by the width, then multiply the result by 2.

The length of the rectangle is 20 feet, and the width is 15 feet. So, the length of the rectangle
→ is 20 feet / 15 feet = 2.5 feet.

Now, we divide the length of the rectangle by the width to find the number of feet of crepe paper
→ needed. 2.5 feet / 2.5 feet = 1.5 feet.

So, James needs to buy 1.5 feet of crepe paper to hang his birthday party streamers from his
→ backyard fence.

d20 model answer: Correct

First find the length of the long side of the rectangle: 20 feet * 2 =
→ <|python_start|>20*2<|python_end|><|output_start|>40<|output_end|>40 feet

Then find the length of the short side: 15 feet * 2 =
→ <|python_start|>15*2<|python_end|><|output_start|>30<|output_end|>30 feet

Then add the length of the long and short sides to find the total length of the crepe paper: 40
→ feet + 30 feet = <|python_start|>40+30<|python_end|><|output_start|>70<|output_end|>70 feet
70

3. ARC-Challenge: Middle-School Science Multiple Choice Questions

1. Thunderstorms and hurricanes are examples of severe weather. Which of these weather conditions is necessary for the formation of thunderstorms and hurricanes?

- a cold rain falling to the ground = A
- air masses combining to create a high-pressure area = B
- winds spiraling downward and away from the center of a storm = C
- air moving toward the center of a storm and rising into the atmosphere = D

d12 model answer: Incorrect

C

d20 model answer: Correct

D

2. Researchers breed plants to produce crops that have helpful traits. One such trait is the ability to grow where there is little moisture. How would this trait most likely benefit farmers?

- It allows crops to be unaffected by disease = A
- It allows crops to be grown in drier climates = B
- It allows crops to survive without fertilization = C
- It allows crops to survive in areas with fewer seasons = D

d12 model answer: Incorrect

C

d20 model answer: Correct

B

3. Which of these is most necessary for the production of geothermal energy?

- hot rock = A
- fossil fuel = B
- thick crust = C
- warm climate = D

d12 model answer: Incorrect
C

d20 model answer: Correct
A

4. Gravity on Earth is caused by

- the mass of Earth = A
- the rotation of Earth = B
- the revolution of Earth = C
- weather patterns on Earth = D

d12 model answer: Incorrect
C

d20 model answer: Correct
A

4.5.2 Developing Our Own Questions from Scratch

Based on experiments with the previous questions, we created 10 additional questions where nanochat answers correctly and picochat does not.

Question 1

Question: A classroom has 5 tables with 6 students sitting at each table. How many students are in the classroom?

d12 model

To find the number of students in the classroom, we need to consider the total number of students in the classroom and then subtract the number of students who are not in the classroom.

The classroom has 5 tables with 6 students each, so there are
 $5 \text{ tables} * 6 \text{ students per table} = 30 \text{ students in the classroom.}$

Now, let's subtract the students who are not in the classroom from the total number of students in the classroom. This gives us
 $30 \text{ students} - 5 \text{ students} = 15 \text{ students.}$

Therefore, there are 15 students in the classroom.

d20 model

To find the total number of students in the classroom, we need to consider the number of students in each table and then add them together.

There are 5 tables, and each table has 6 students.
So, the total number of students in the classroom is
 $5 \text{ tables} * 6 \text{ students per table} = 30 \text{ students.}$

Therefore, there are 30 students in the classroom.

Question 2

Question:

Assertion (A): The Earth revolves around the Sun.

Reason (R): The Sun revolves around the Earth once every day.

- (a) Both A and R are true and R is the correct explanation of A.
- (b) Both A and R are true but R is not the correct explanation of A.

(c) A is true but R is false.

(d) A is false but R is true.

Respond only with the letter of the correct answer.

d12 model

A) Both A and R are true and R is the correct explanation of A.

...

(repeated outputs)

...

(d) A is false but R is true.

d20 model

(c) is the correct option.

Question 3

Question: Tom has 24 apples. He gives 9 apples to his friend and then buys 6 more apples. How many apples does Tom have now?

d12 model

To find out how many apples Tom has now, let's break down the information given. Tom has 24 apples, and he gives 9 apples to his friend.

This means that the total number of apples he has is
 $24 \text{ apples} + 9 \text{ apples} = 38 \text{ apples}.$

Now, let's consider the number of apples Tom has now.
He gives 9 apples to his friend, which means he has
 $9 \text{ apples} - 9 = 3 \text{ apples left}.$

This leaves $24 - 3 = 21$ apples remaining.

Since Tom has 3 apples left, he has
 $21 \text{ apples} - 3 \text{ apples} = 18 \text{ apples now}.$

d20 model

Tom has $24 - 9 = 15$ apples left.
Tom then buys 6 more apples, so he now has
 $15 + 6 = 21$ apples.
21

Question 4

Question: Which continent is the Sahara Desert located on?

A. Asia

B. Africa

C. Australia

D. South America

d12 model Answer: A

d20 model Answer: B

Question 5

Question: Which unit is used to measure electrical current?

- A. Volt
- B. Ohm
- C. Ampere
- D. Watt

d12 model Answer: A

d20 model Answer: C

Question 6

Question: Which ocean lies between Africa and Australia?

- A. Atlantic Ocean
- B. Pacific Ocean
- C. Indian Ocean
- D. Arctic Ocean

d12 model Answer: B

d20 model Answer: C

Question 7

Question: Which layer of Earth's atmosphere contains the ozone layer?

- A. Troposphere
- B. Stratosphere
- C. Mesosphere
- D. Thermosphere

d12 model Answer: A

d20 model Answer: B

Question 8

Question: A bakery makes 24 cupcakes in the morning and 36 cupcakes in the afternoon. If 15 cupcakes are sold, how many cupcakes remain?

d12 model

To find the number of cupcakes sold, we need to calculate the total number of cupcakes sold and then divide it by the number of cupcakes sold.

The bakery makes 24 cupcakes in the morning and 36 cupcakes in the afternoon, so the total number of cupcakes sold is $24 + 36 = 60$ cupcakes.

Now, let's calculate the number of cupcakes sold. We know that 15 cupcakes are sold.

The bakery makes 60 cupcakes in the morning and 36 cupcakes in the afternoon, so the total number of cupcakes sold is $60 + 36 = 100$ cupcakes.

$100 / 60 = 5$ cupcakes.

Therefore, the bakery remains with 5 cupcakes.

d20 model

The bakery makes 24 cupcakes in the morning and 36 cupcakes in the afternoon.

Total cupcakes = $24 + 36 = 60$

Cupcakes remaining = $60 - 15 = 45$

Therefore, 45 cupcakes remain.

Question 9

Question: Which organ filters waste from the blood to form urine?

- A. Heart
- B. Liver
- C. Kidney
- D. Stomach

d12 model Answer: D

d20 model Answer: C

Question 10

Question: Which continent contains the Amazon Rainforest?

- A. Africa
- B. Asia
- C. South America
- D. Europe

d12 model Answer: A

d20 model Answer: C

4.5.3 Emergent Abilities

1. Improved Multi-Step Reasoning: The d20 model performed correct step-by-step reasoning in most of the arithmetic and logic problems (e.g., GSM8K and custom math questions), while the d12 model frequently introduces incorrect intermediate steps or irrelevant operations.
2. Reduced Logical Inconsistencies: The d20 model produces more coherent reasoning chains and avoids contradictory statements, whereas the d12 model frequently generates repetitive and circular reasoning during problem solving.

5 Table Summarizing all Changes

Change	PeriLN
Pre-training Impact	Slight drop in CORE and slightly higher validation bpb compared to the baseline model.
SFT Impact	Large degradation in downstream performance, including negative ChatCORE and near-zero scores on several benchmarks.
Efficiency Impact	Negligible computational overhead because it only modifies normalization ordering.
Final Decision	Not used in the final architecture alone. (used with Differential Attention)
Commentary	PeriLN alone does not improve language modeling or downstream task performance.
Change	Differential Attention
Pre-training Impact	Slightly worse pre-training metrics compared to baseline.
SFT Impact	Improves HumanEval and GSM8K performance, indicating stronger reasoning and coding ability.
Efficiency Impact	Introduces approximately 17.6% additional FLOPs per token and increases training time.
Final Decision	Partially used in the final architecture.
Commentary	Provides improvements in reasoning tasks but increases computational cost.
Change	PeriLN + Differential Attention
Pre-training Impact	Slightly worse pre-training performance compared to baseline.
SFT Impact	Achieves the highest ChatCORE and strongest reasoning benchmarks.
Efficiency Impact	Same computational cost as differential attention models.
Final Decision	Selected architecture for scaling.
Commentary	Provides the best balance between reasoning performance and general task accuracy.
Change	Scaling Depth (d12 → d20)
Pre-training Impact	Large improvement in CORE and significantly lower validation bpb.
SFT Impact	Improves performance on ARC, MMLU, and GSM8K benchmarks when compared with d12 model of the same architecture.
Efficiency Impact	Requires significantly more training compute.
Final Decision	Final NanoChat model configuration.
Commentary	Confirms scaling-law prediction that larger models perform better.

Table 18: Summary of architectural changes and their impact on model performance.

References

- [1] Chen, S., Wong, S., Chen, L., & Tian, Y. (2023). Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*.
- [2] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- [3] Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. L., Hendricks, L. A., Welbl, J., Clark, A., et al. (2022). Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*.
- [4] Kamath, A., Ferret, J., Pathak, S., & Gemma Team. (2025). Gemma 3 technical report. *arXiv*. <https://doi.org/10.48550/arXiv.2503.19786>
- [5] Kamradt, G. (2023). Needle in a haystack — pressure testing LLMs. *GitHub*. https://github.com/gkamradt/LLMTest_NeedleInAHaystack
- [6] Karpathy, A. (2025). nanochat. *GitHub*. <https://github.com/karpathy/nanochat>

- [7] Kim, J., Lee, B., Park, C., Oh, Y., Kim, B., Yoo, T., Shin, S., Han, D., Shin, J., & Yoo, K. M. (2025). Peri-LN: Revisiting normalization layer in the transformer architecture. *Proceedings of ICML 2025*. <https://doi.org/10.48550/arXiv.2502.02732>
- [8] Levy, I., Jacoby, B., & Goldberg, Y. (2024). Same task, more tokens: The impact of input length on the reasoning performance of large language models. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*.
- [9] Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. *OpenAI Blog*. <https://openai.com/research/language-unsupervised>
- [10] Riviere, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., & Gemma Team. (2024). Gemma 2: Improving open language models at a practical size. *arXiv*. <https://doi.org/10.48550/arXiv.2408.00118>
- [11] Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin, J., et al. (2023). Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.
- [12] Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2021). RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*.
- [13] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- [14] Walsh, P., Soldaini, L., Groeneveld, D., Lo, K., & Team OLMo. (2024). 2 OLMo 2 Furious. *arXiv*. <https://doi.org/10.48550/arXiv.2501.00656>
- [15] Sun, Y., Ma, S., Dong, L., Huang, S., Wang, H., & Wei, F. (2024). Differential Transformer. *arXiv*. <https://arxiv.org/abs/2410.05258>
- [16] Naik, D., Naik, I., & Naik, N. (2024). Decoder-Only Transformers: The Brains Behind Generative AI, Large Language Models and Large Multimodal Models. In N. Naik, P. Jenkins, S. Prajapat, & P. Grace (Eds.), *Contributions Presented at The International Conference on Computing, Communication, Cybersecurity and AI (C3AI 2024)*. *Lecture Notes in Networks and Systems*, vol 884. Springer, Cham. https://doi.org/10.1007/978-3-031-74443-3_19
- [17] Zuhri, Z. M. K., Fuadi, E. H., & Aji, A. F. (2025). Predicting the Order of Upcoming Tokens Improves Language Modeling. *arXiv*. <https://doi.org/10.48550/arXiv.2508.19228>

Picochat Baseline

March 7, 2026

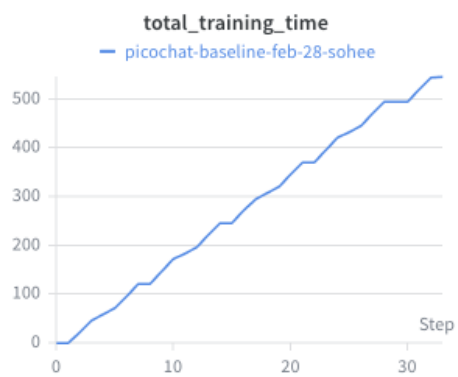


Figure 1:

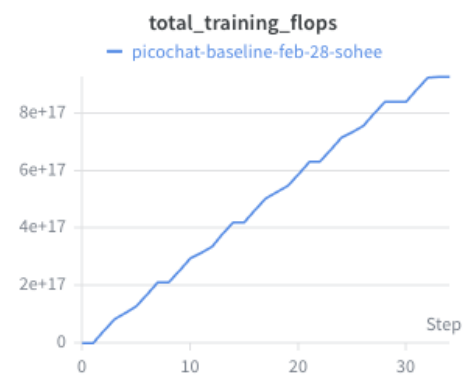


Figure 2:

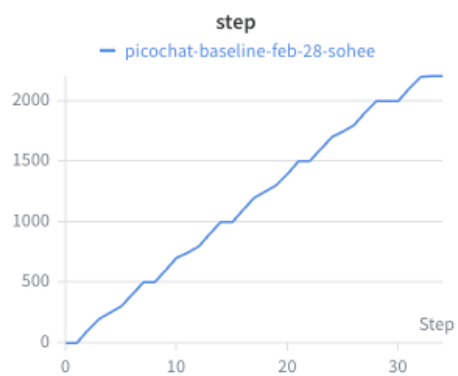


Figure 3:

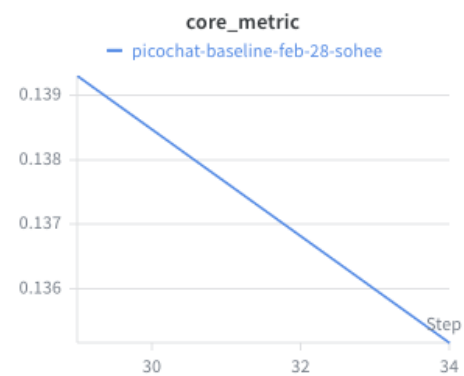


Figure 4:

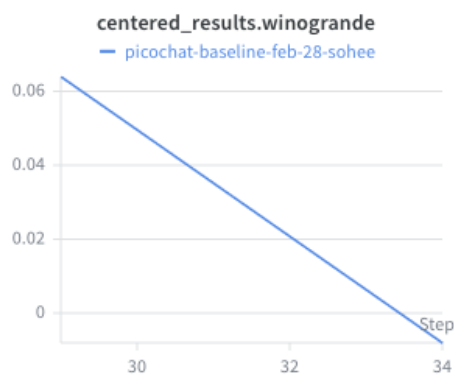


Figure 5:

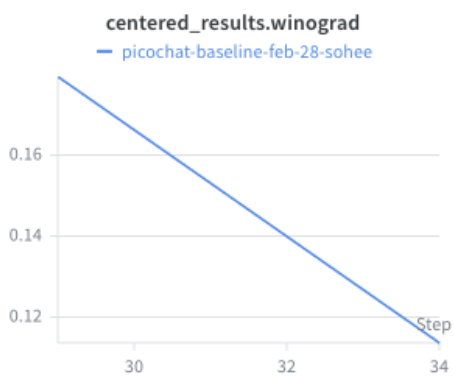


Figure 6:

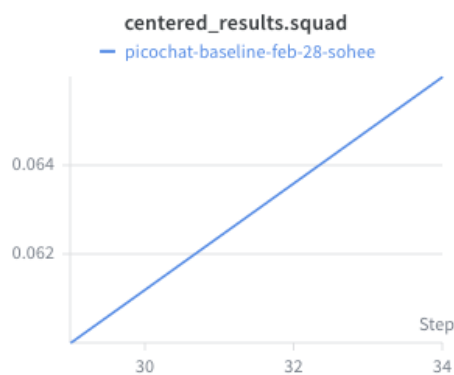


Figure 7:

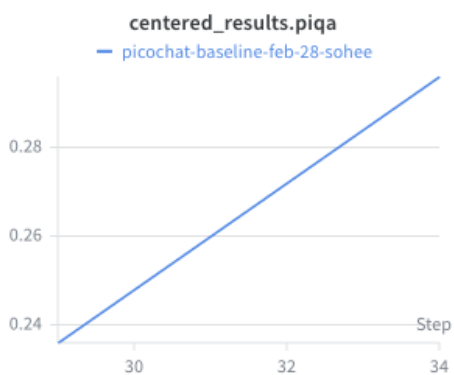


Figure 8:

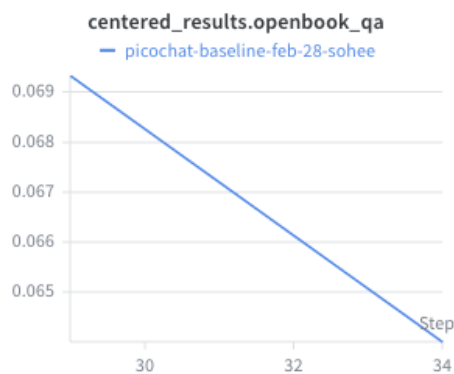


Figure 9:

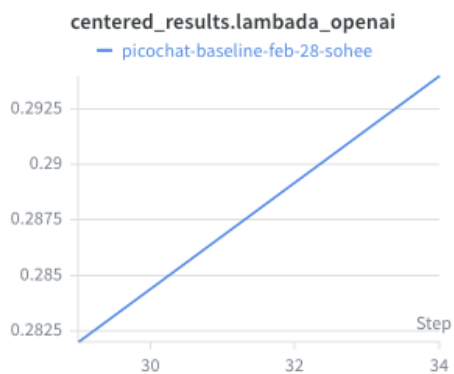


Figure 10:

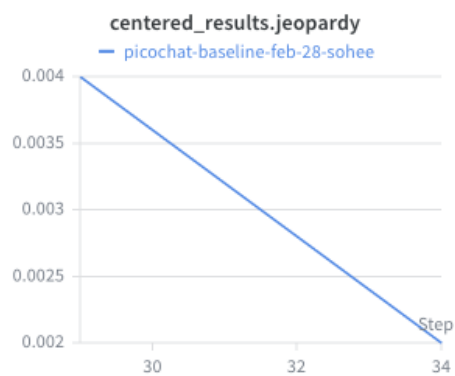


Figure 11:

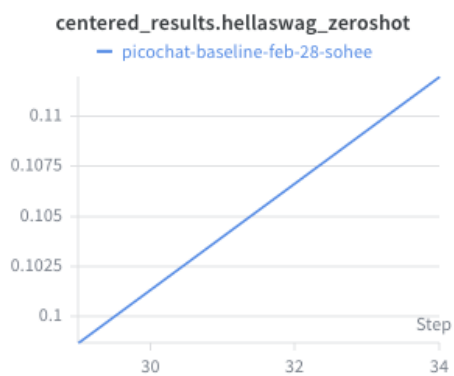


Figure 12:

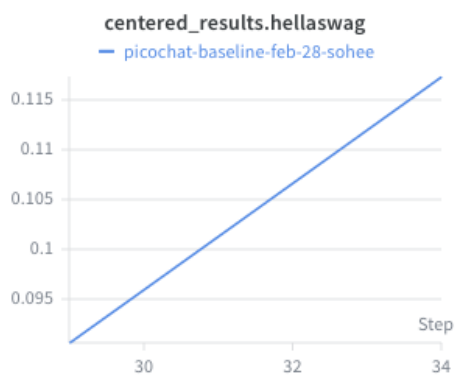


Figure 13:

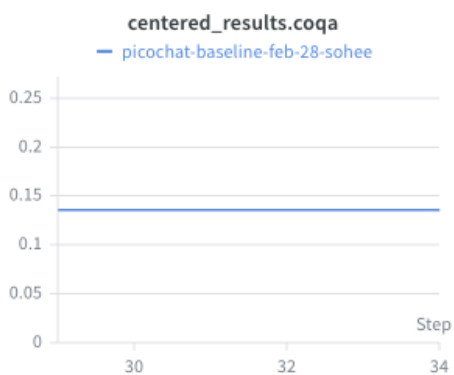


Figure 14:

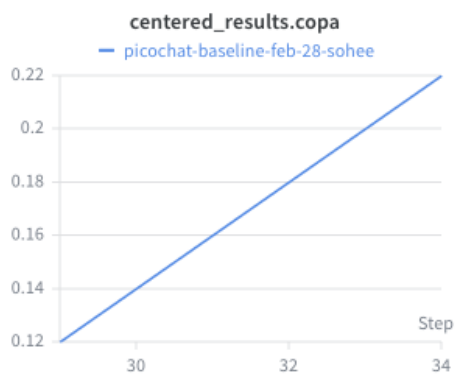


Figure 15:

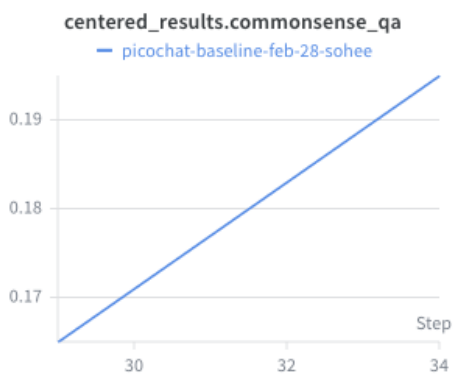


Figure 16:

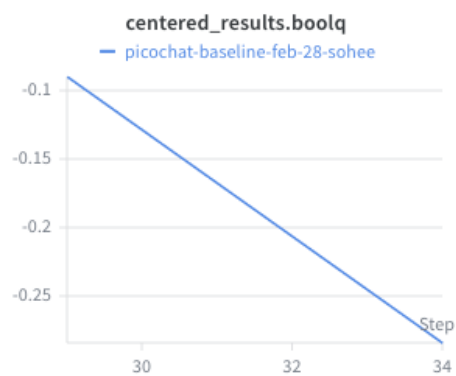


Figure 17:

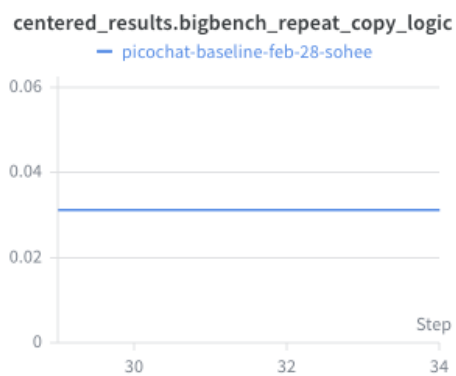


Figure 18:

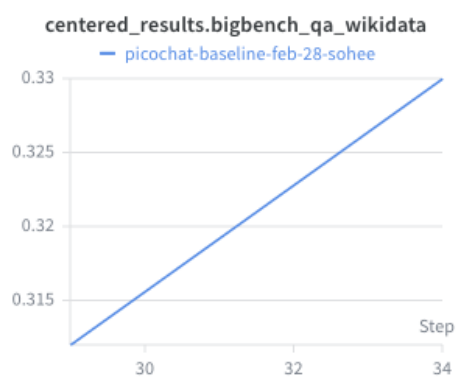


Figure 19:

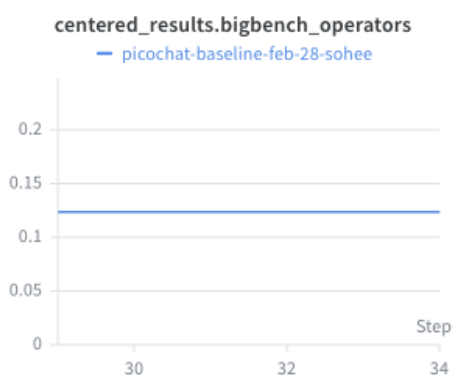


Figure 20:

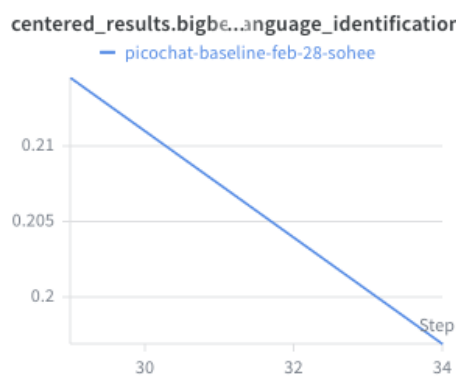


Figure 21:

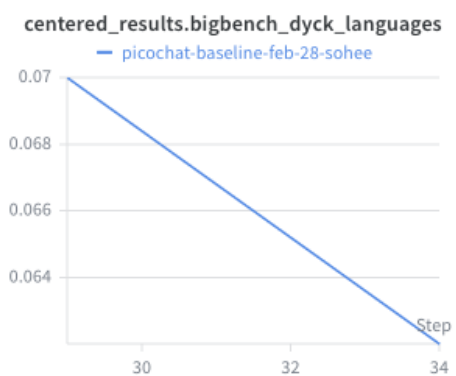


Figure 22:

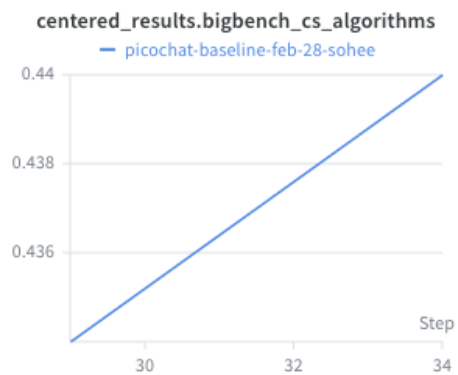


Figure 23:

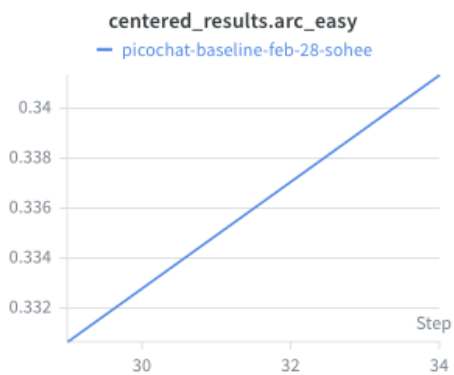


Figure 24:

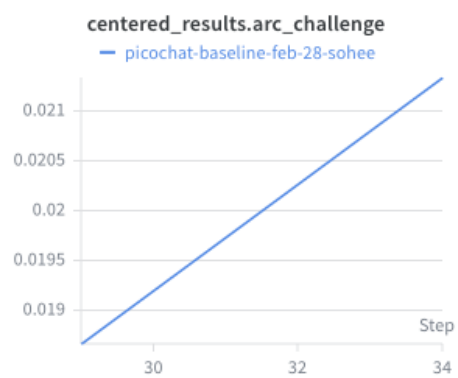


Figure 25:

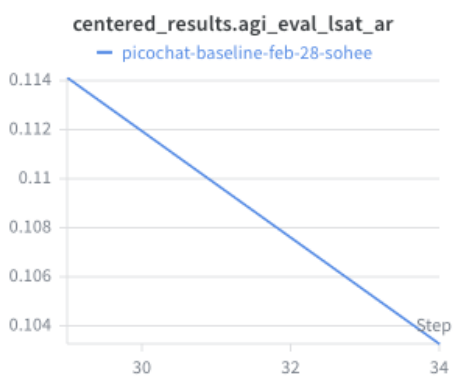


Figure 26:

Peri-LN Ablation

March 7, 2026



Figure 1:

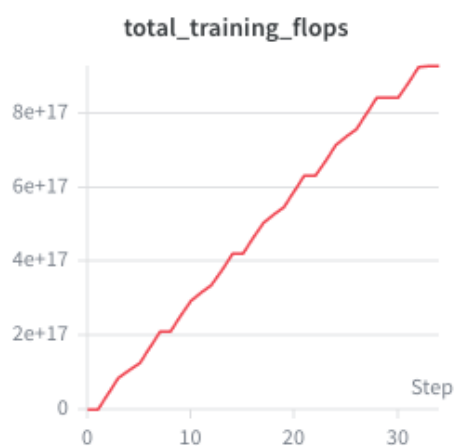


Figure 2:

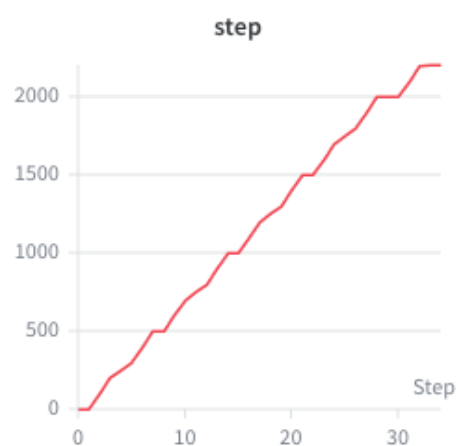


Figure 3:

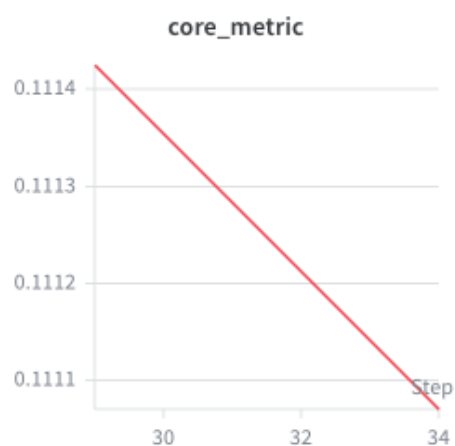


Figure 4:



Figure 5:



Figure 6:

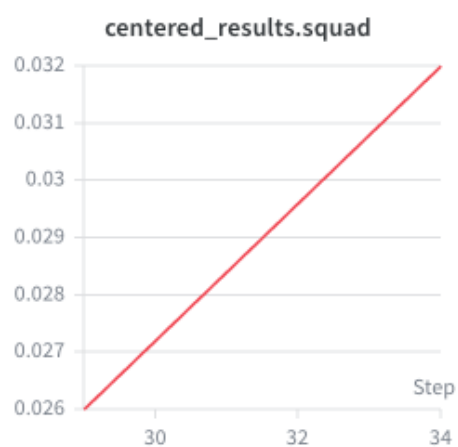


Figure 7:

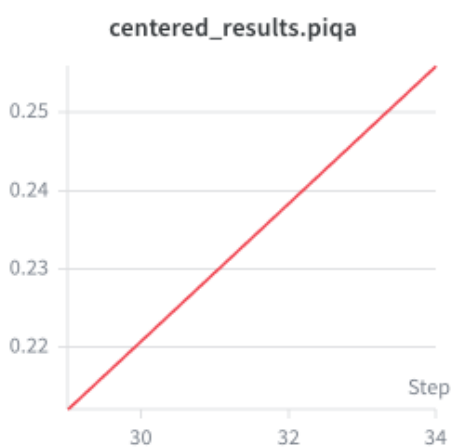


Figure 8:

/



Figure 9:

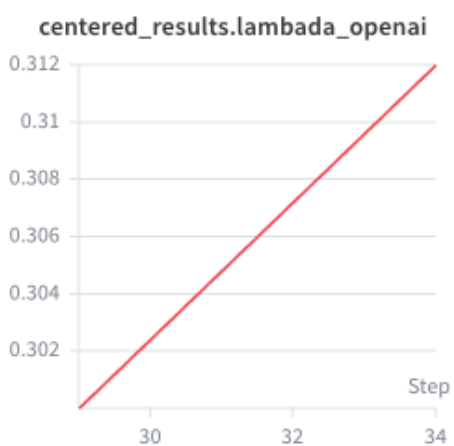


Figure 10:

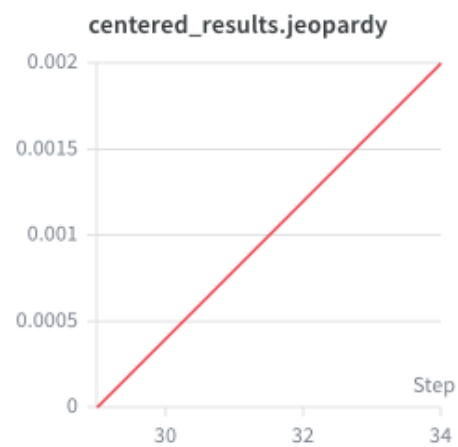


Figure 11:

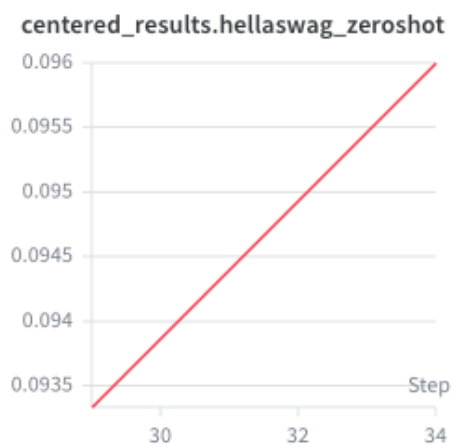


Figure 12:



Figure 13:

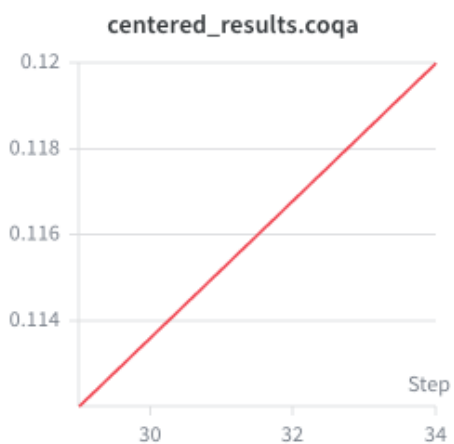


Figure 14:

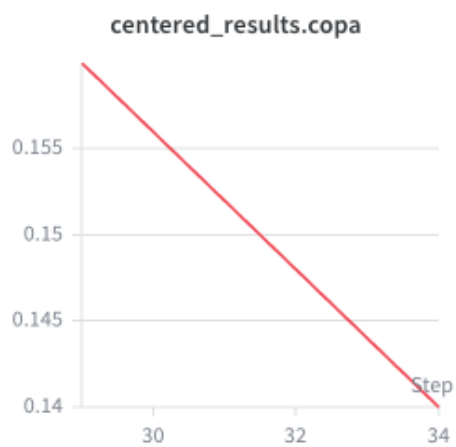


Figure 15:



Figure 16:

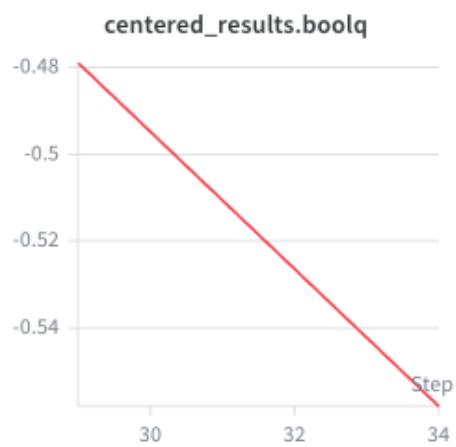


Figure 17:



Figure 18:



Figure 19:

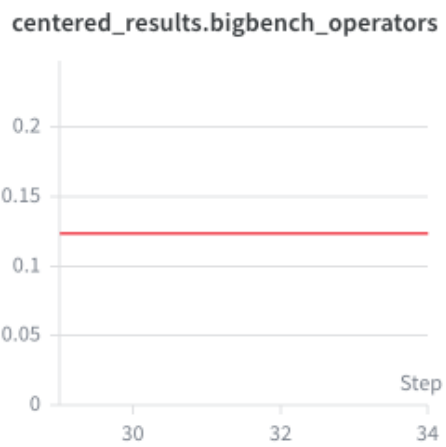


Figure 20:



Figure 21:

Figure 22:

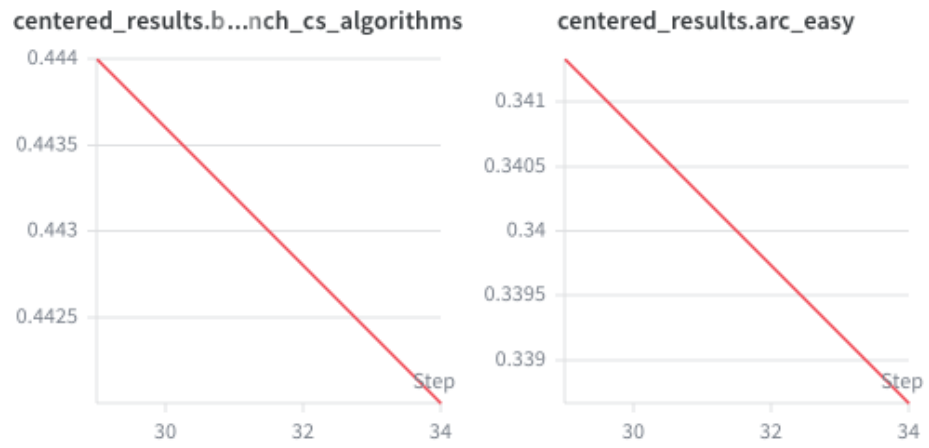


Figure 23:

Figure 24:



Figure 25:



Figure 26:

Differential Attention

March 7, 2026

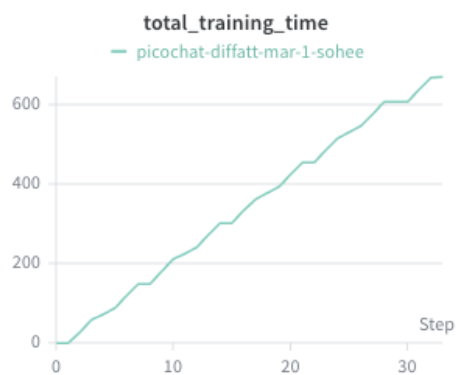


Figure 1:

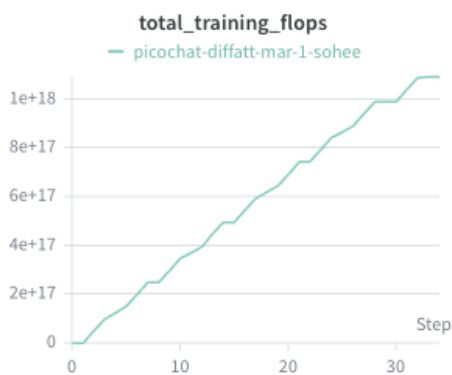


Figure 2:

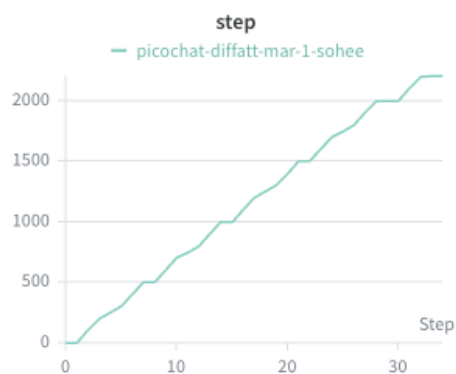


Figure 3:



Figure 4:



Figure 5:



Figure 6:

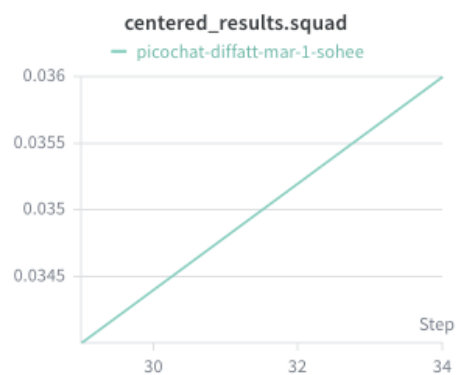


Figure 7:

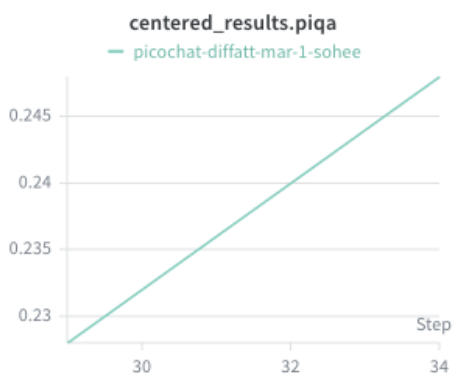


Figure 8:

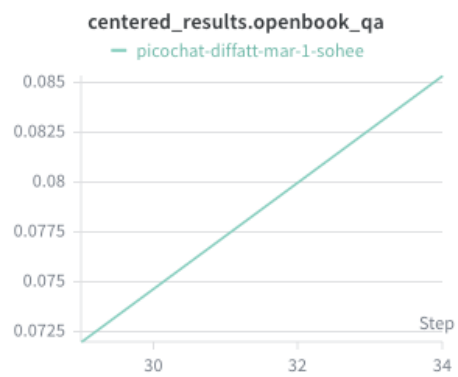


Figure 9:

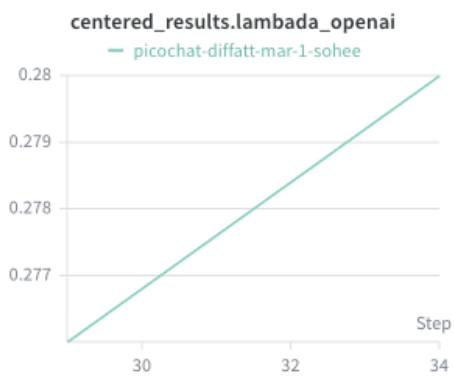


Figure 10:

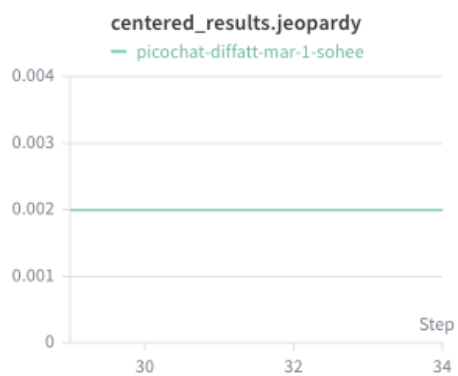


Figure 11:

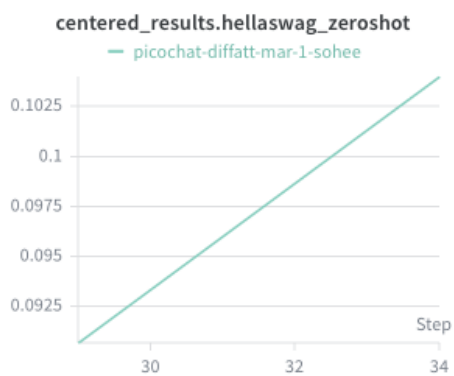


Figure 12:

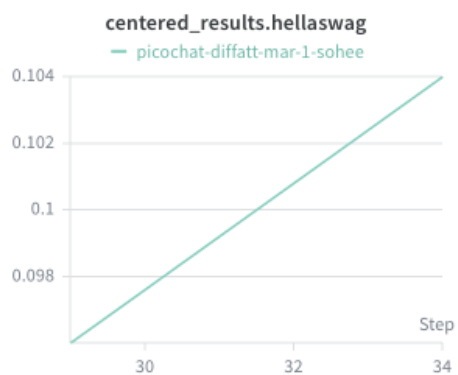


Figure 13:

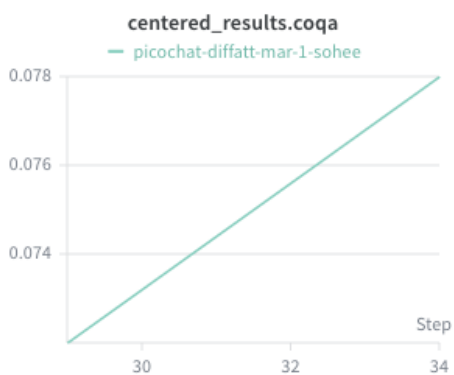


Figure 14:

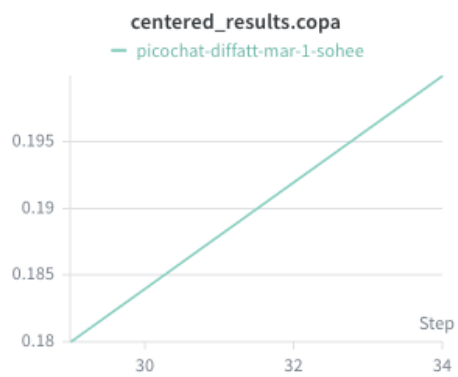


Figure 15:

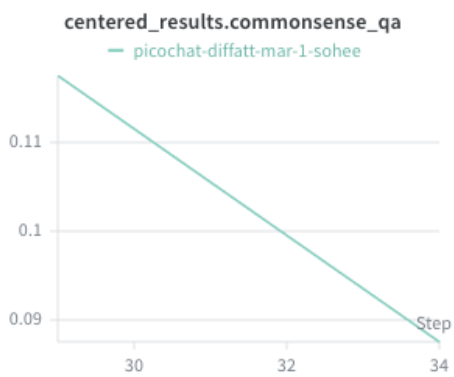


Figure 16:

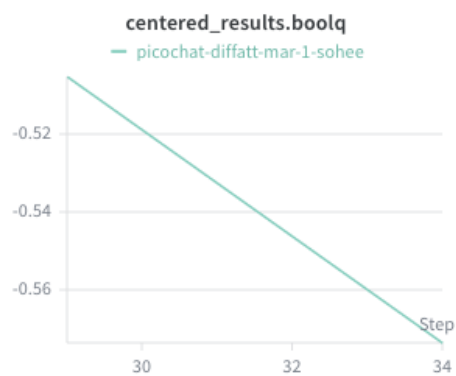


Figure 17:

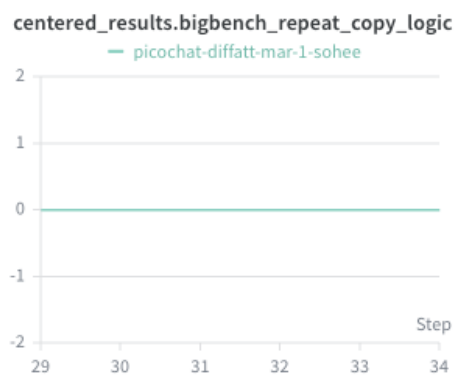


Figure 18:

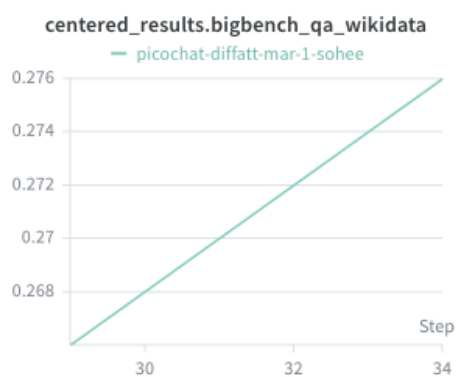


Figure 19:



Figure 20:



Figure 21:

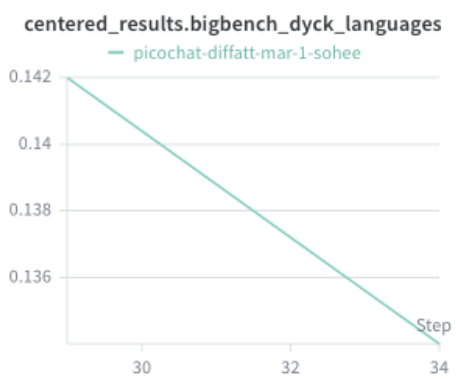


Figure 22:

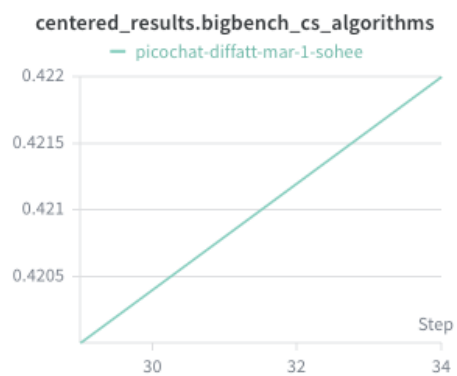


Figure 23:

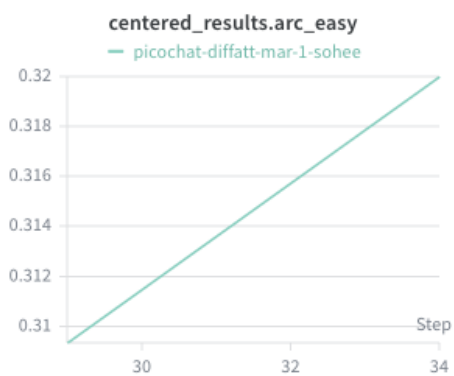


Figure 24:

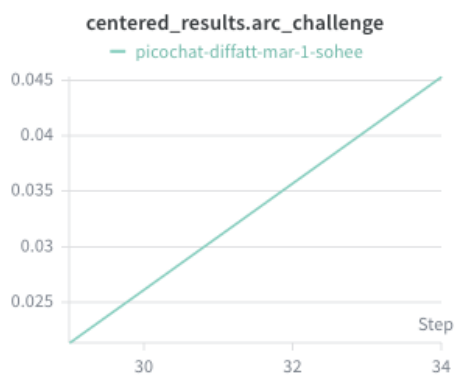


Figure 25:

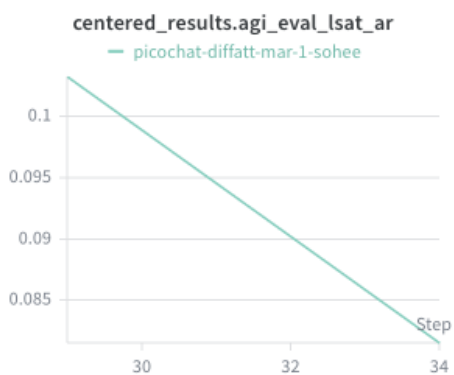


Figure 26:

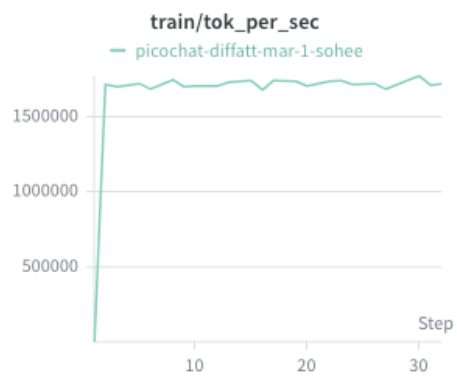


Figure 27:

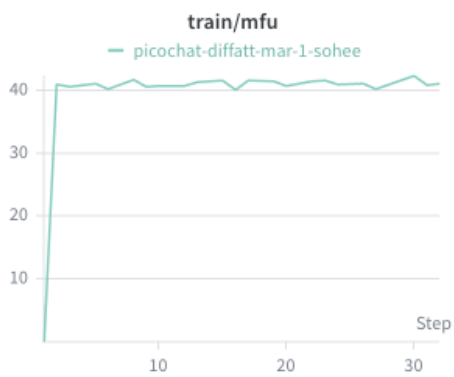


Figure 28:

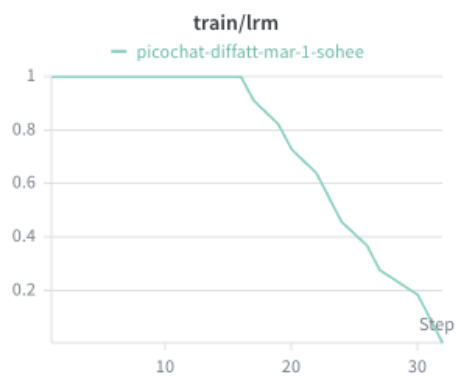


Figure 29:



Figure 30:



Figure 31:



Figure 32:

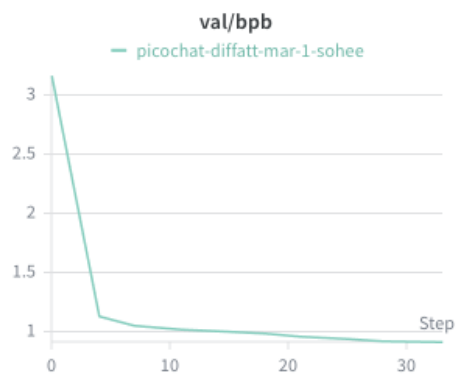


Figure 33:

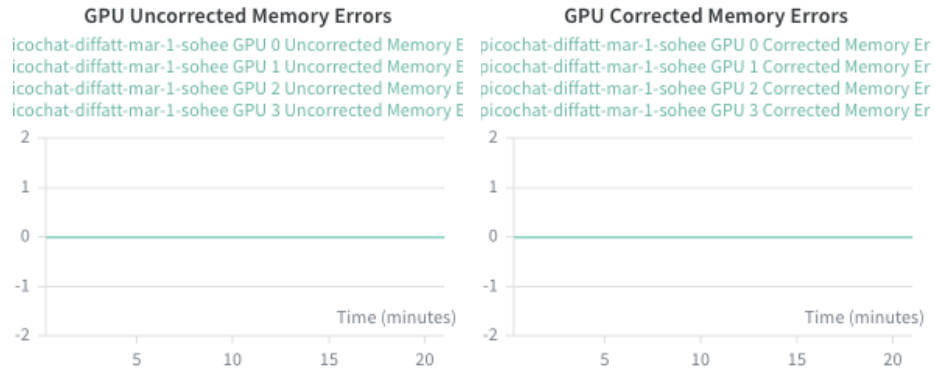


Figure 34:

Figure 35:

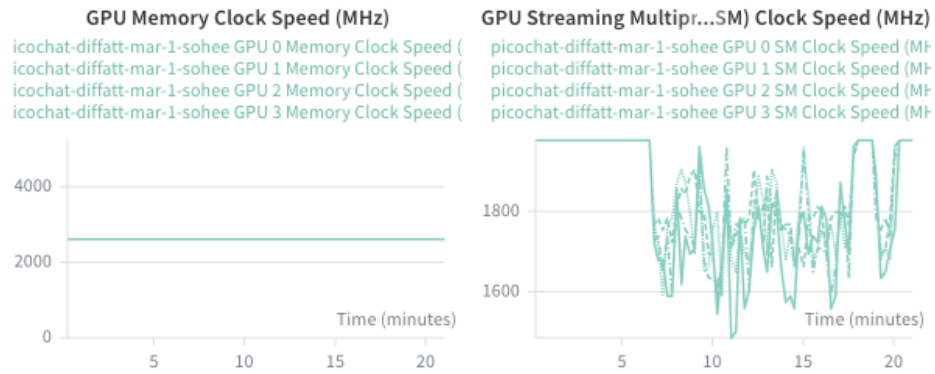


Figure 36:

Figure 37:

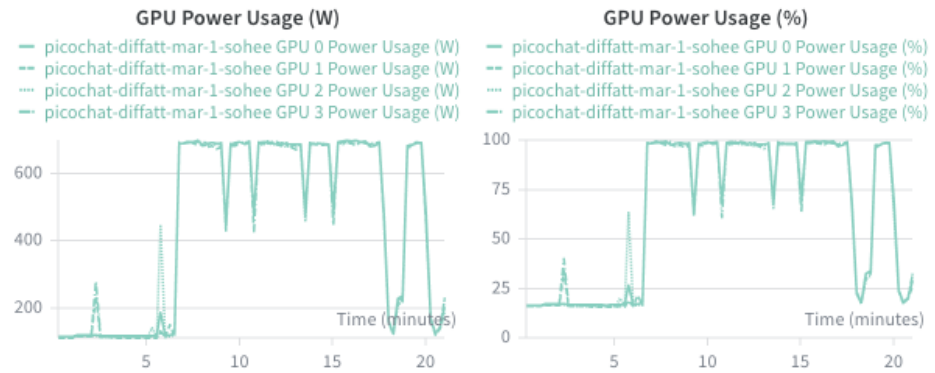


Figure 38:

Figure 39:

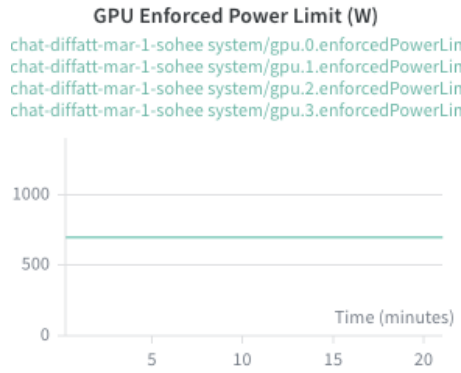


Figure 40:

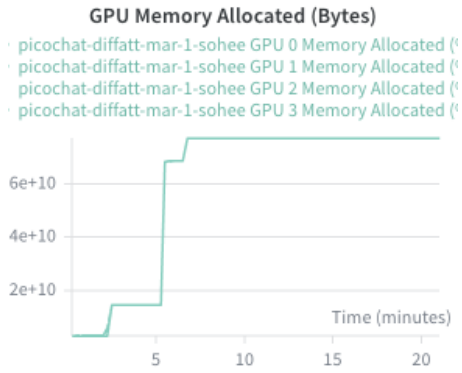


Figure 41:

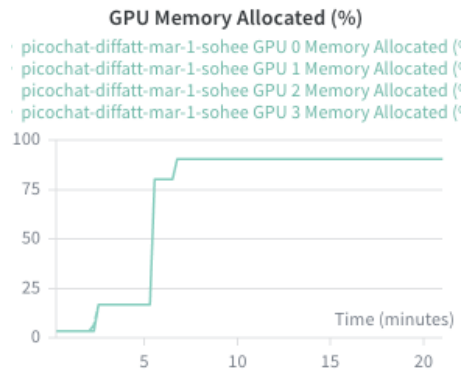


Figure 42:

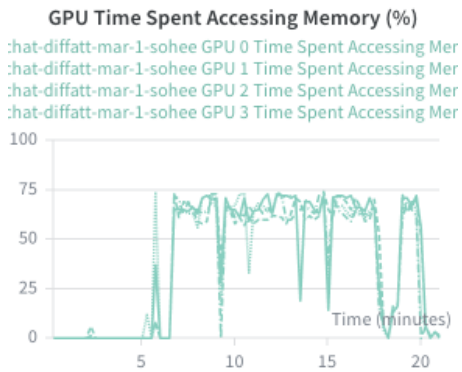


Figure 43:

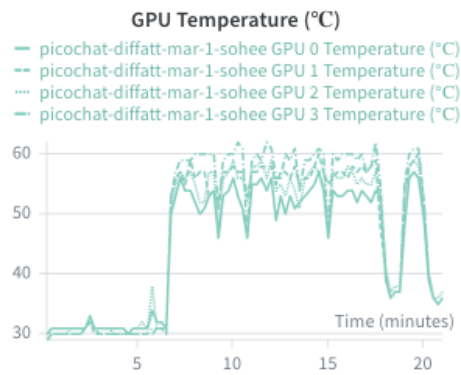


Figure 44:

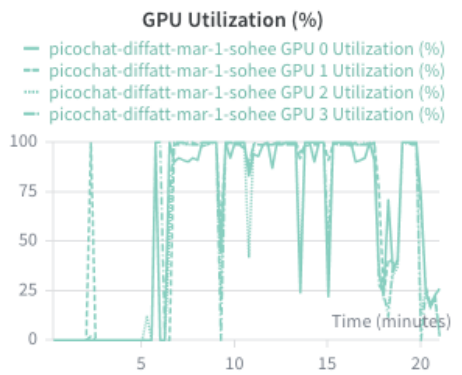


Figure 45:

Peri-LN Differential Attention Ablation

March 7, 2026

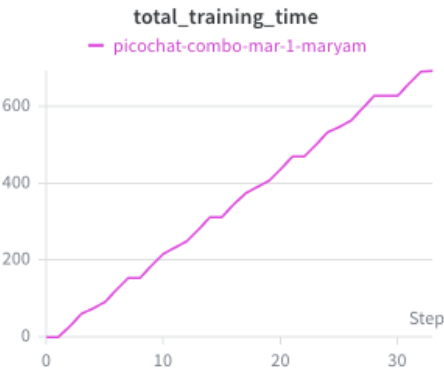


Figure 1:

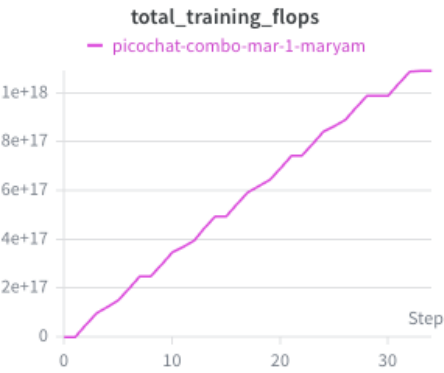


Figure 2:

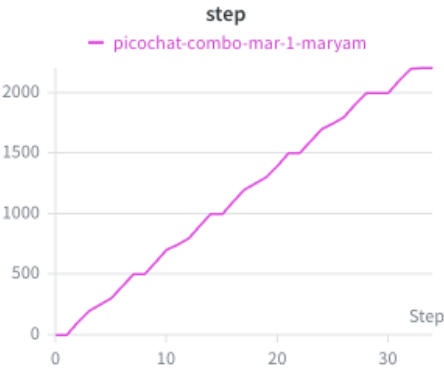


Figure 3:

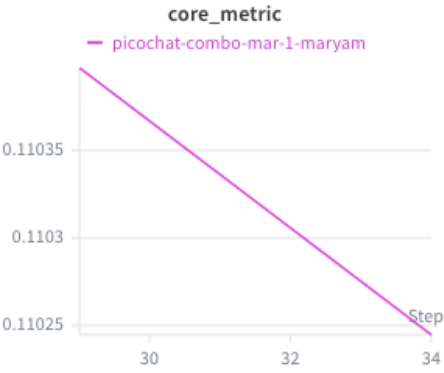


Figure 4:

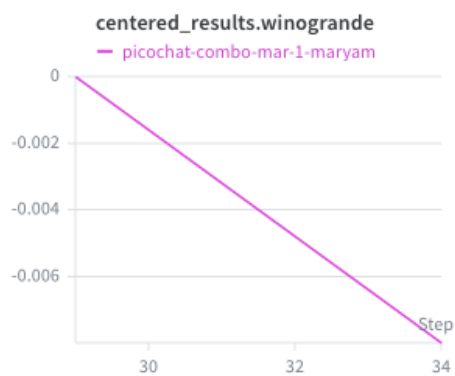


Figure 5:

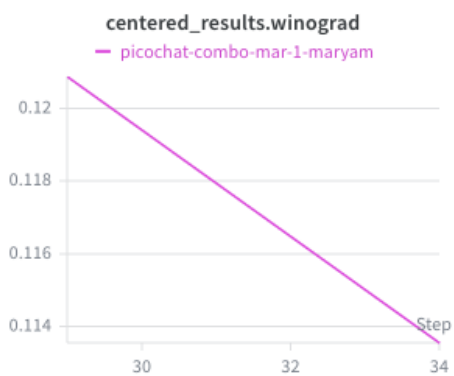


Figure 6:

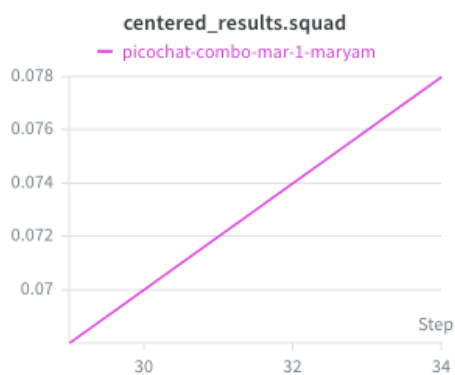


Figure 7:

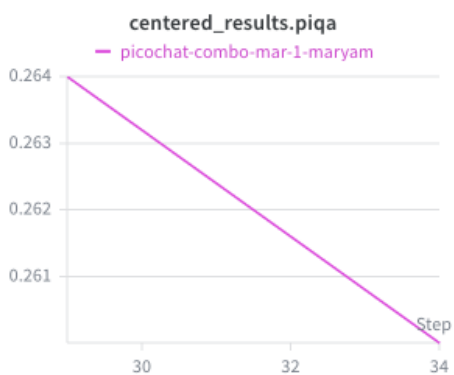


Figure 8:

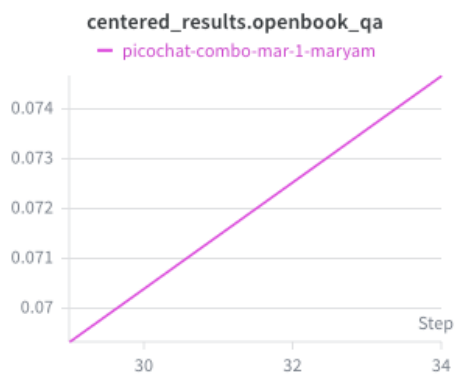


Figure 9:

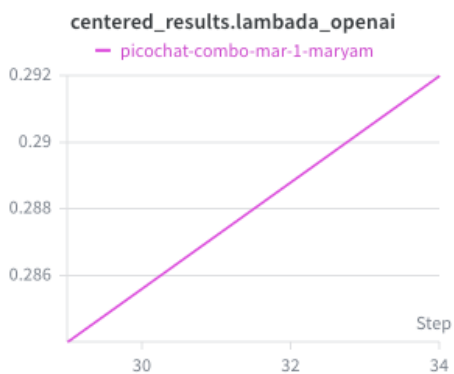


Figure 10:

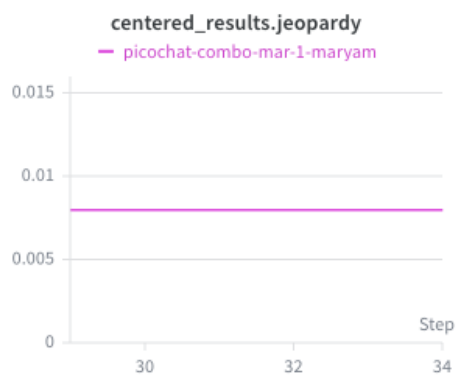


Figure 11:

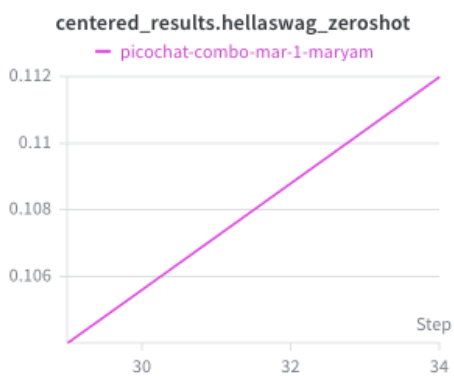


Figure 12:

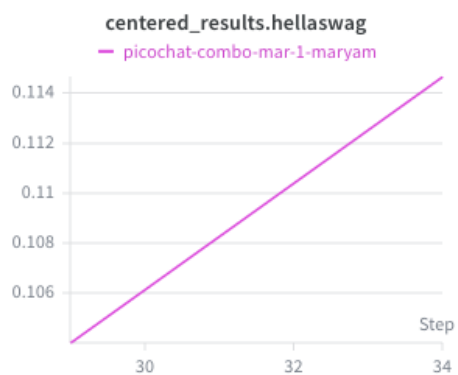


Figure 13:

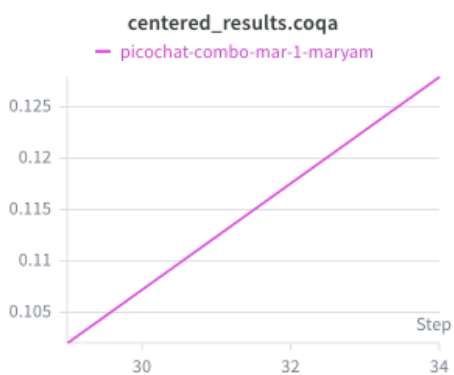


Figure 14:

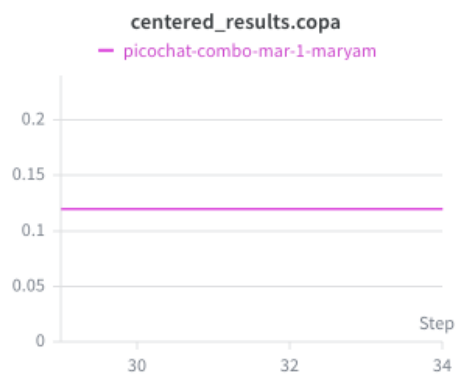


Figure 15:

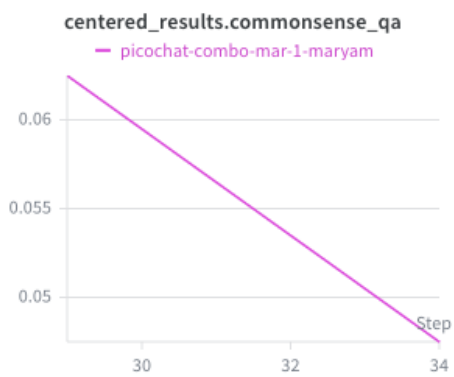


Figure 16:

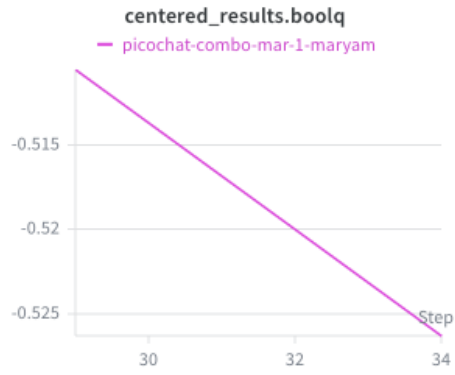


Figure 17:



Figure 18:

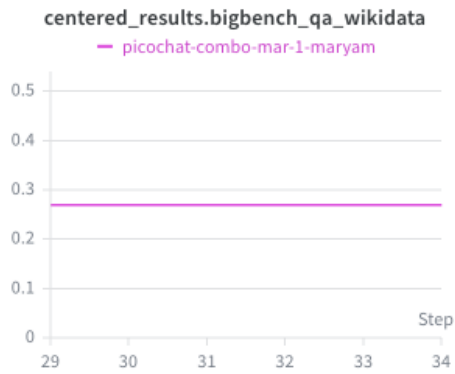


Figure 19:

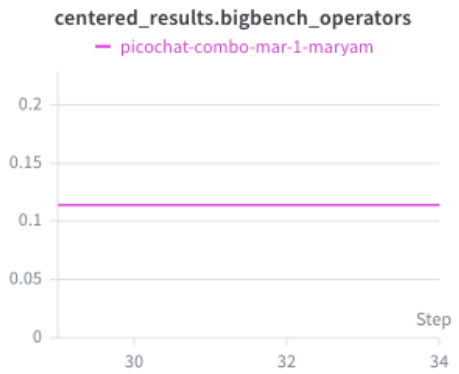


Figure 20:

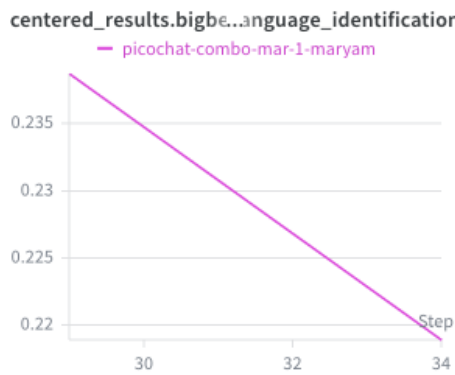


Figure 21:

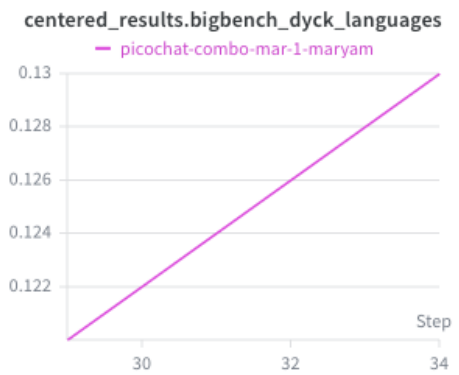


Figure 22:

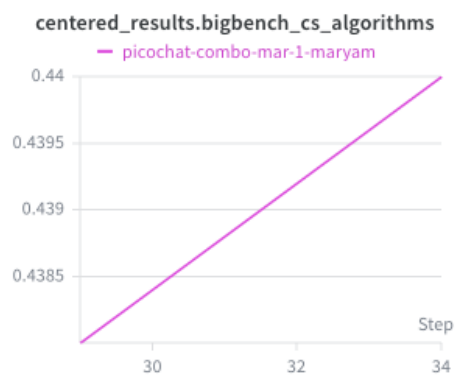


Figure 23:

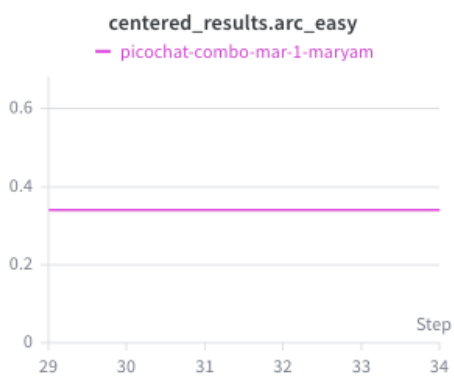


Figure 24:

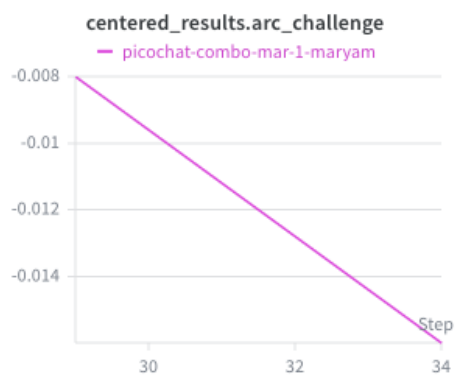


Figure 25:

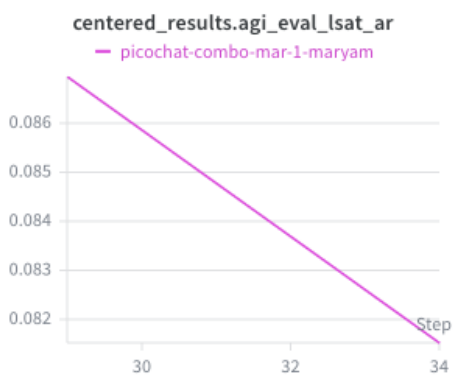


Figure 26:

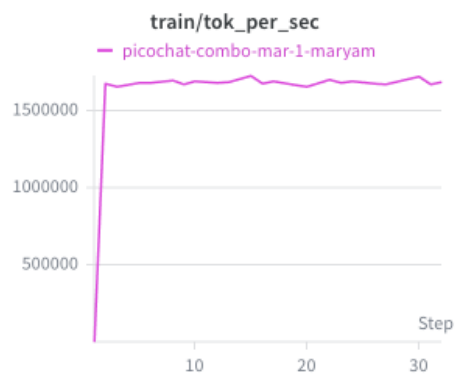


Figure 27:

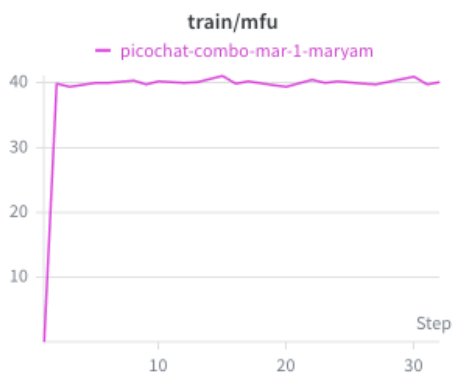


Figure 28:

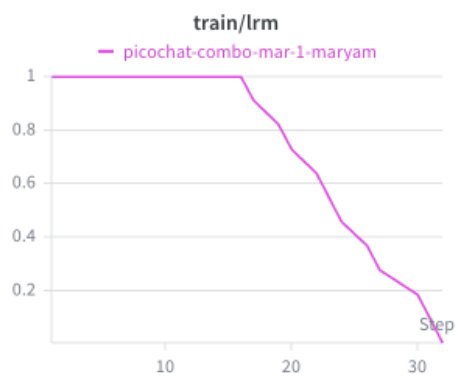


Figure 29:



Figure 30:

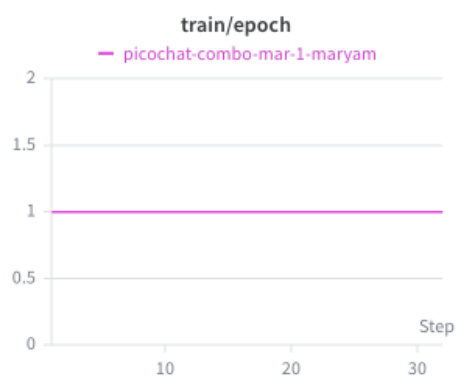


Figure 31:

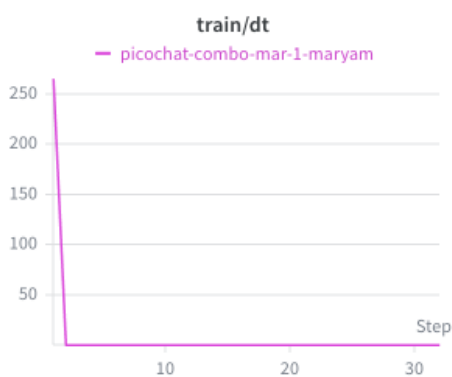


Figure 32:

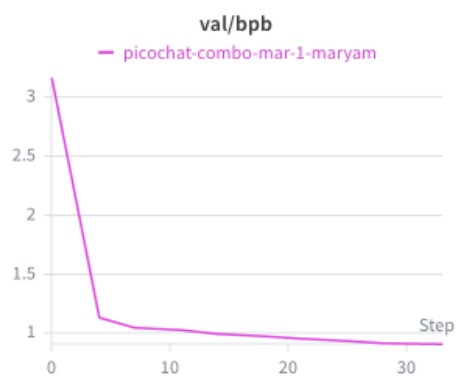


Figure 33:

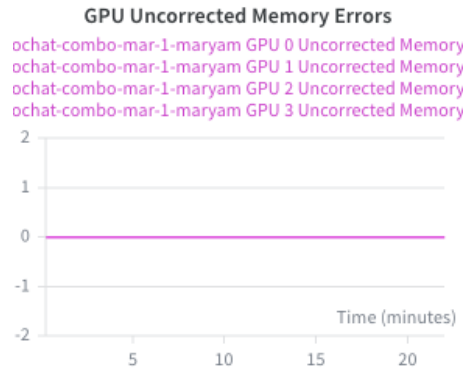


Figure 34:

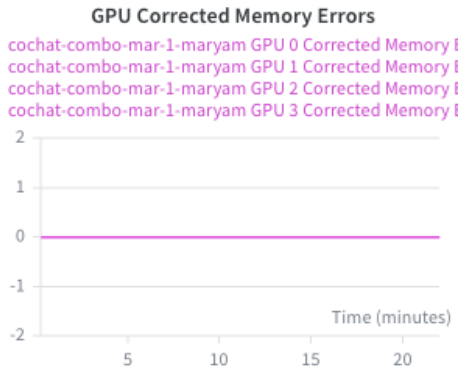


Figure 35:

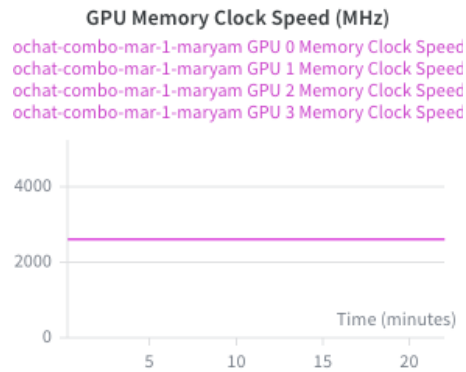


Figure 36:

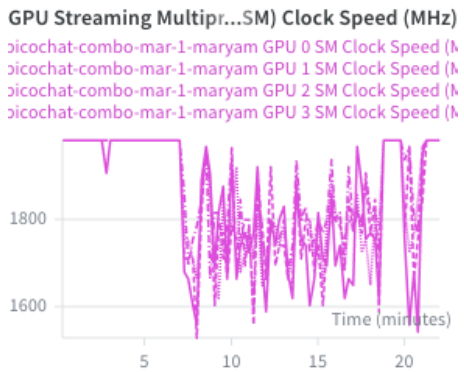


Figure 37:

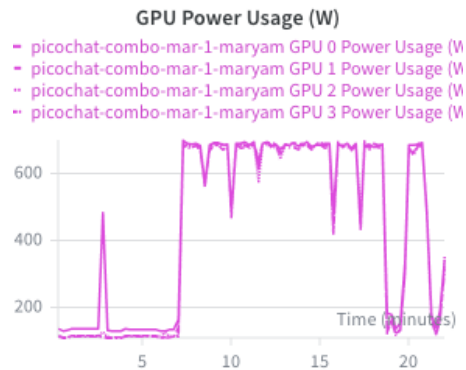


Figure 38:

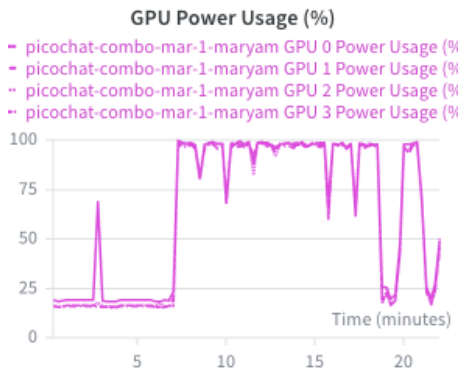


Figure 39:

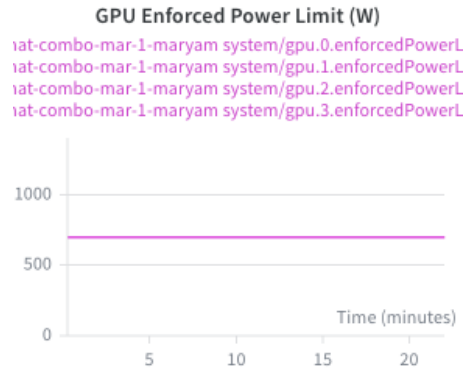


Figure 40:

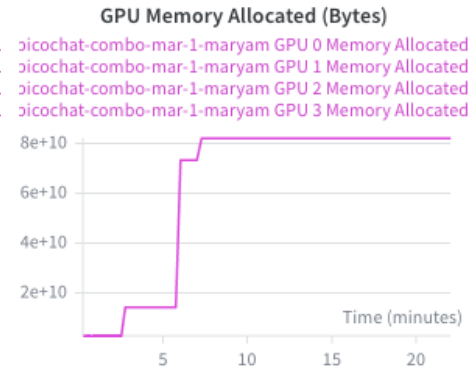


Figure 41:

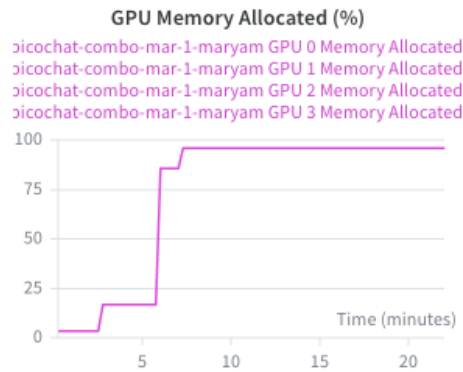


Figure 42:

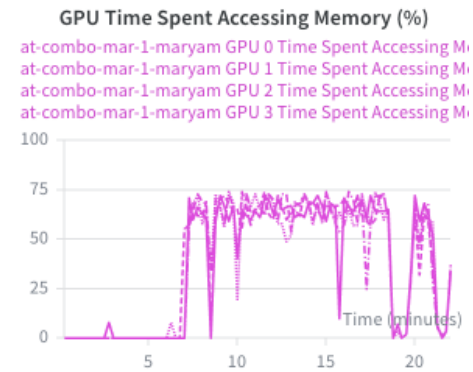


Figure 43:

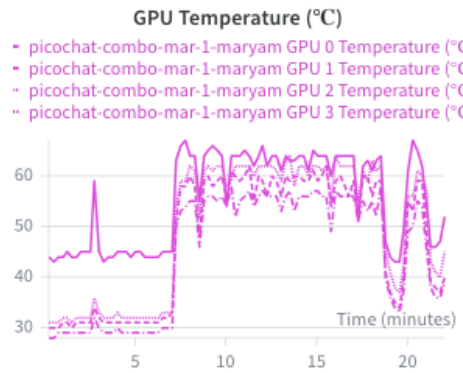


Figure 44:

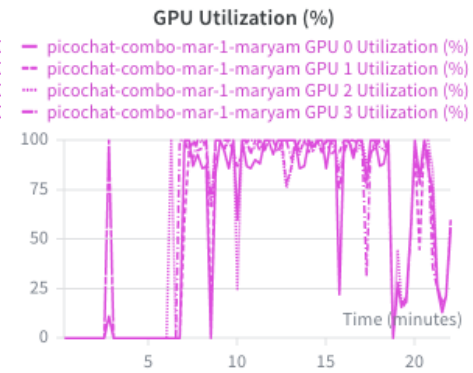


Figure 45:

Extending Context Window Results

March 7, 2026

1 Sequence length 512

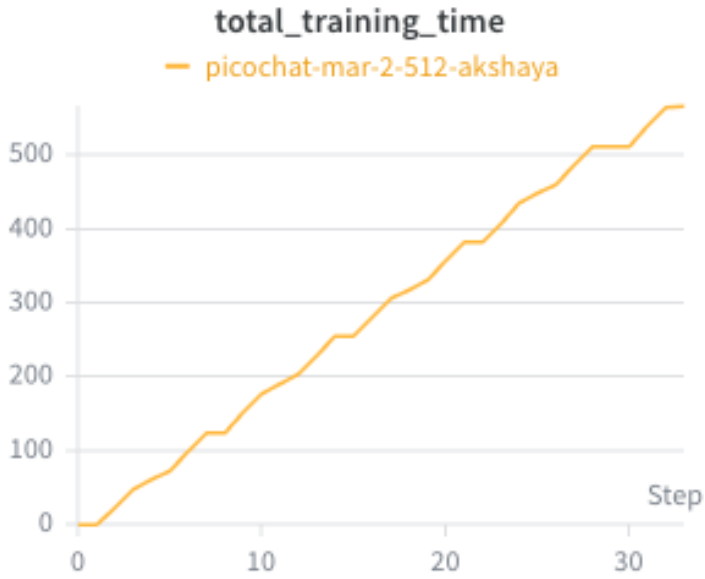


Figure 1:

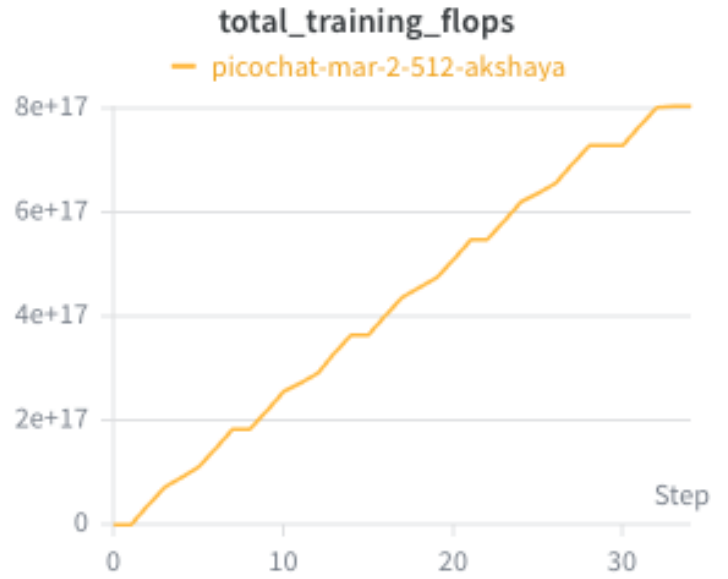


Figure 2:

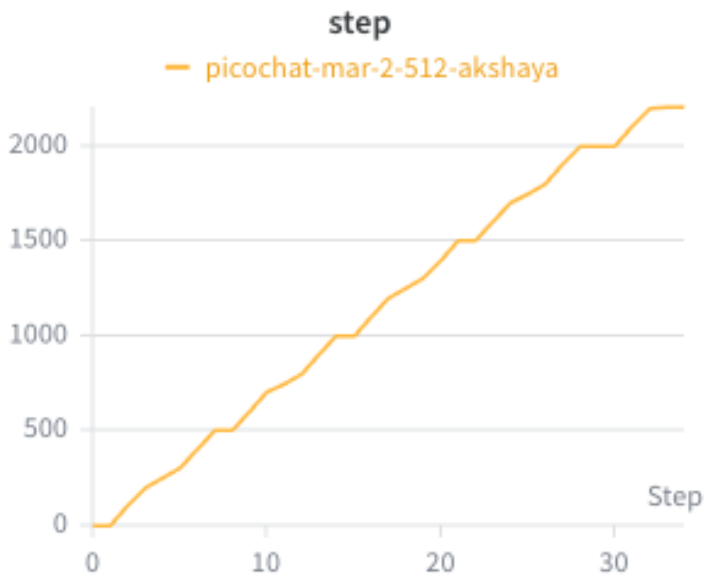


Figure 3:

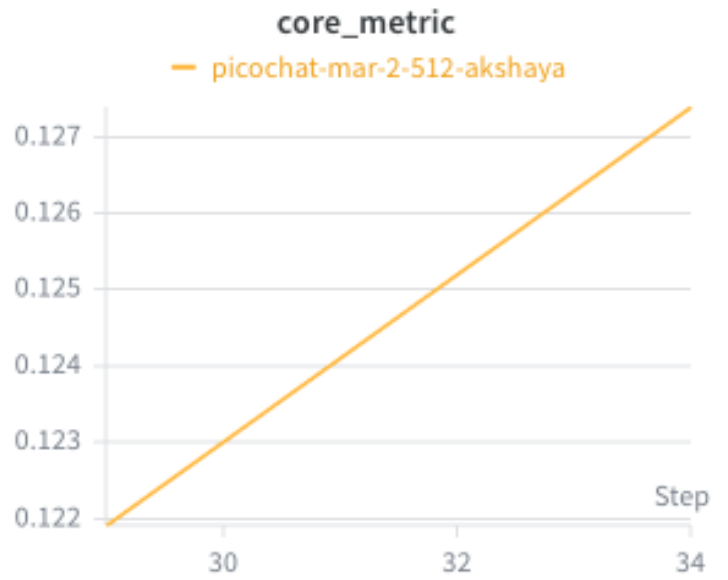


Figure 4:

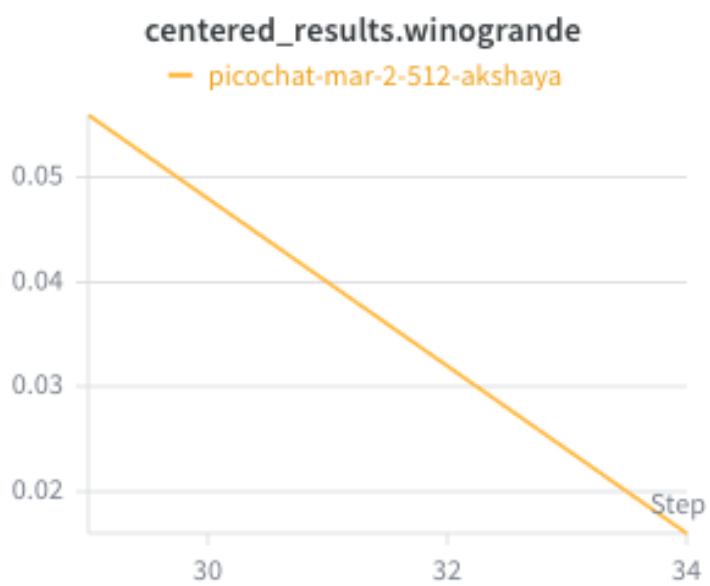


Figure 5:

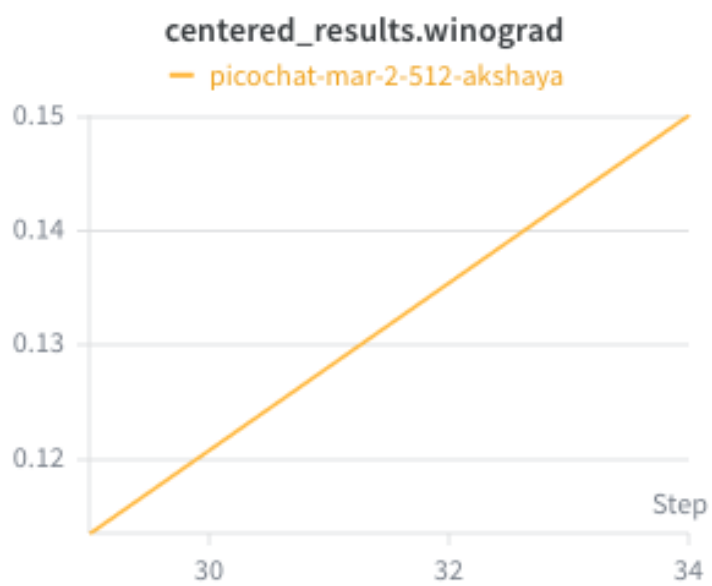


Figure 6:

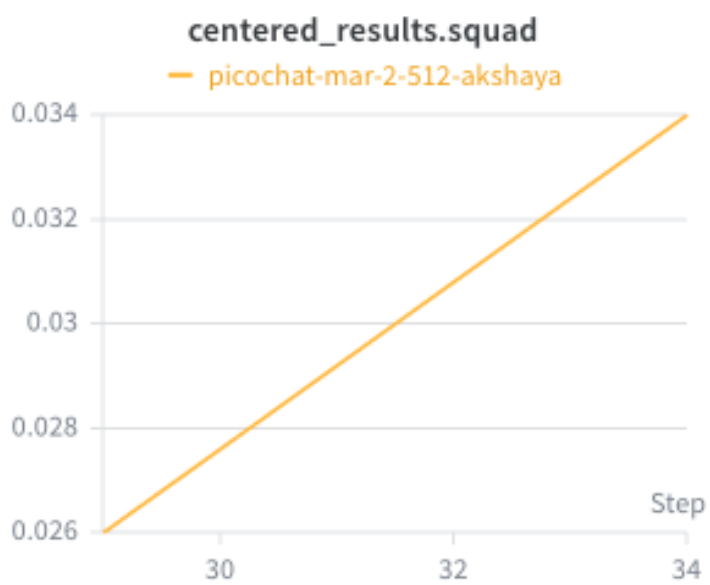


Figure 7:

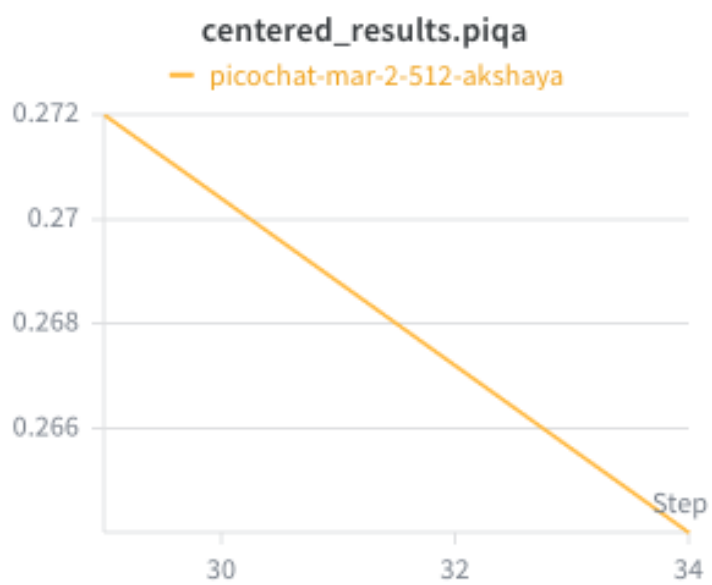


Figure 8:

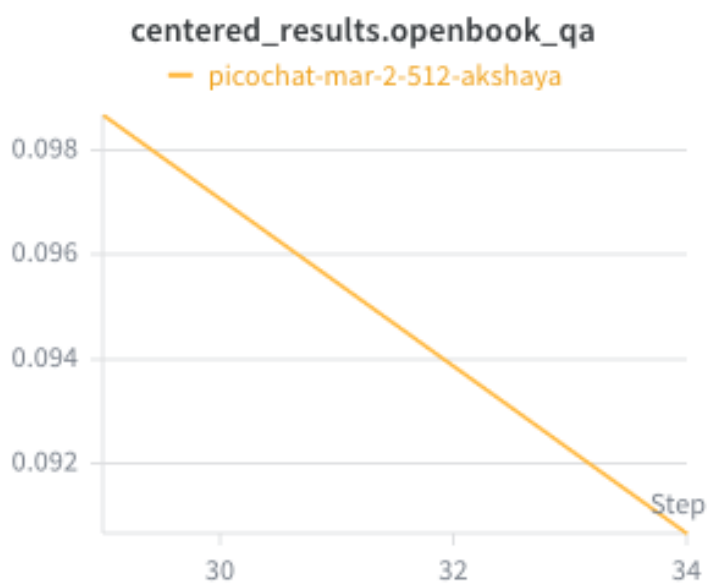


Figure 9:

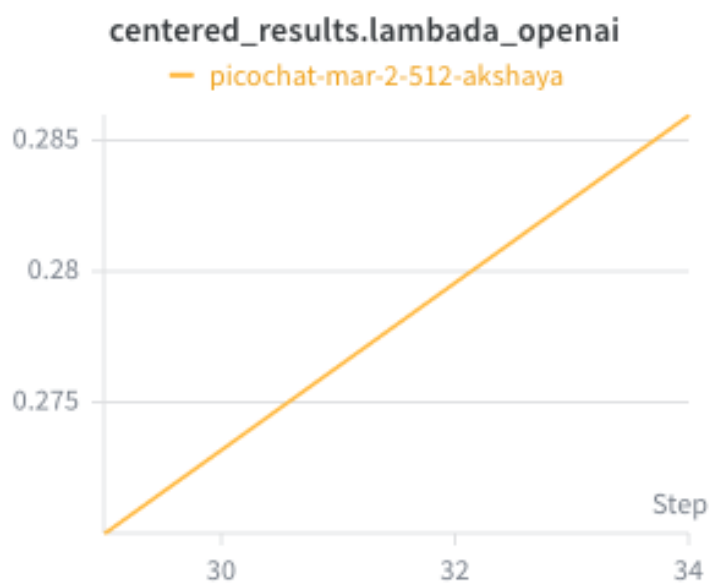


Figure 10:

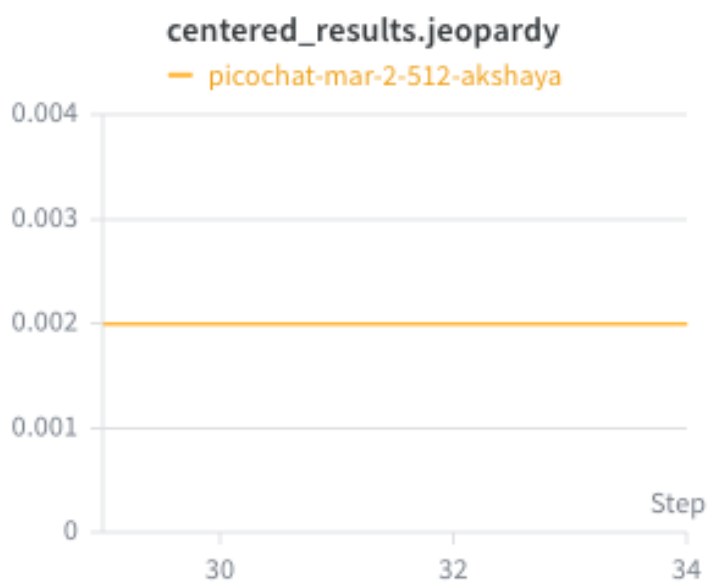


Figure 11:

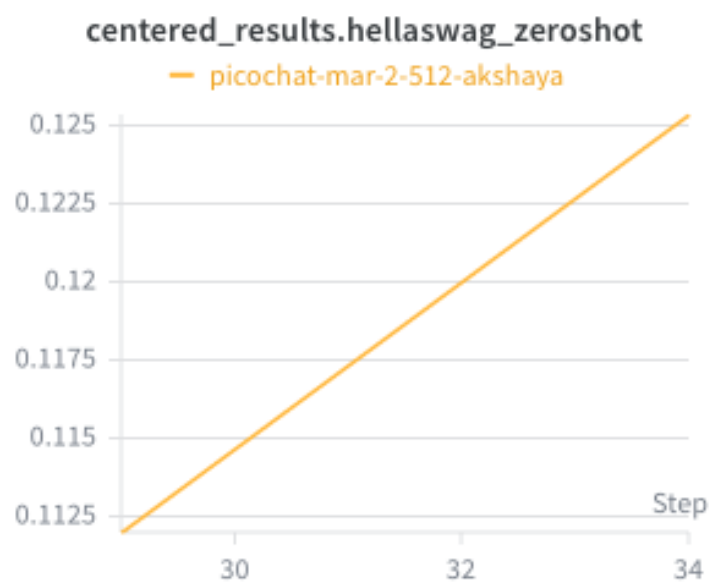


Figure 12:

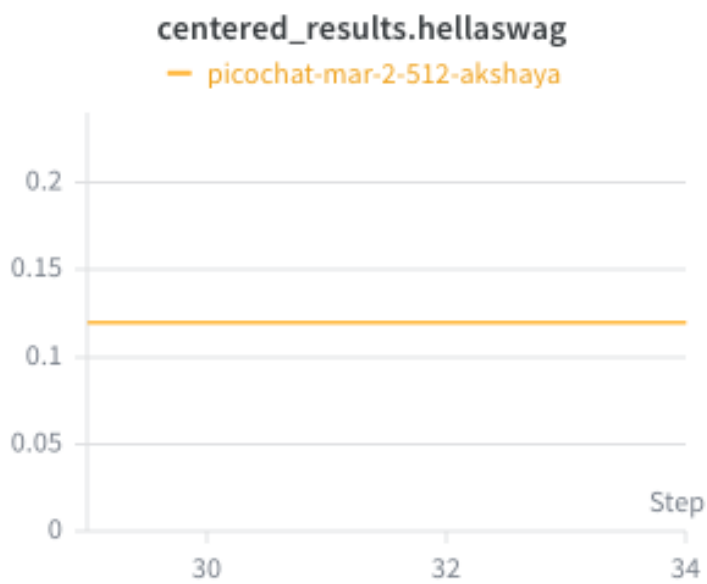


Figure 13:

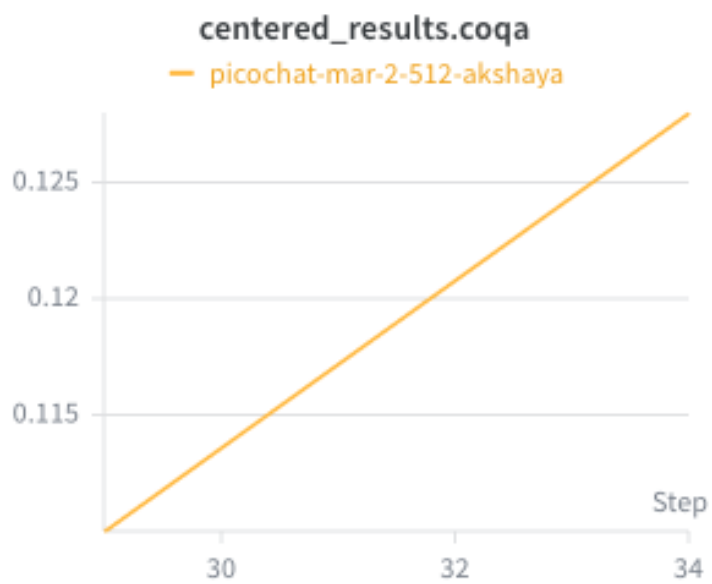


Figure 14:

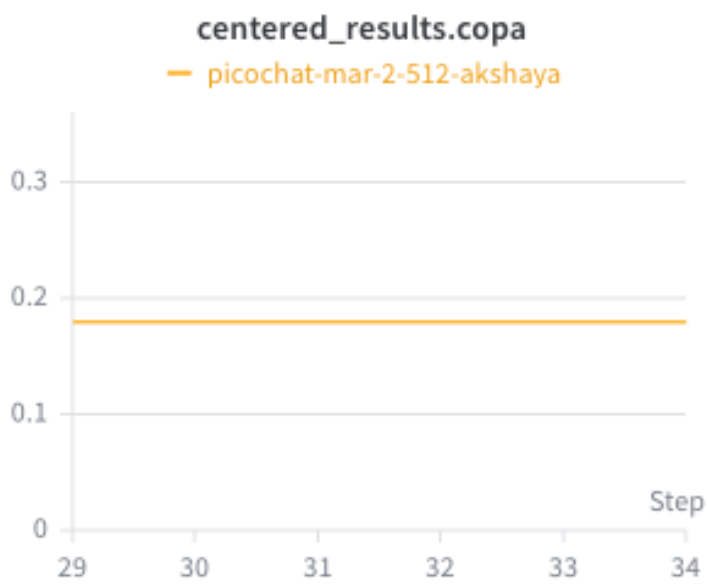


Figure 15:

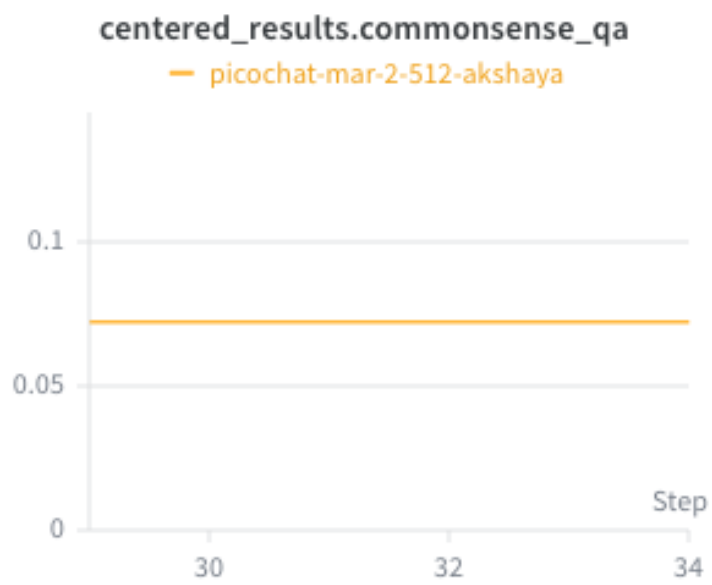


Figure 16:

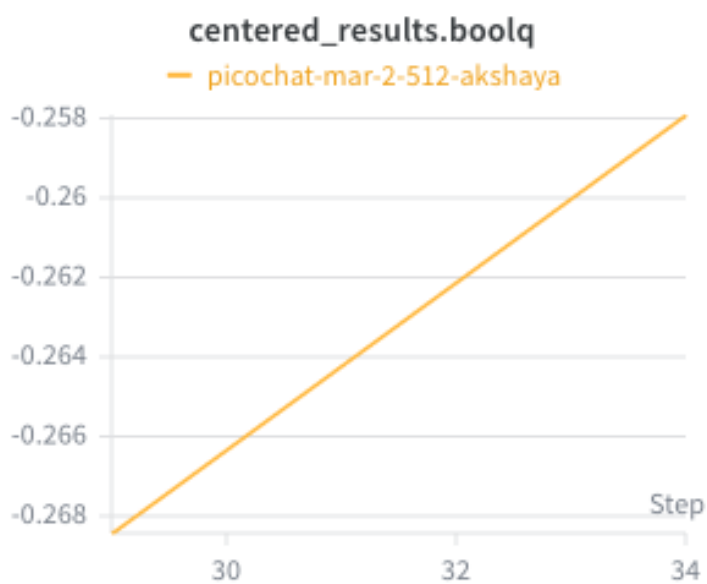


Figure 17:



Figure 18:

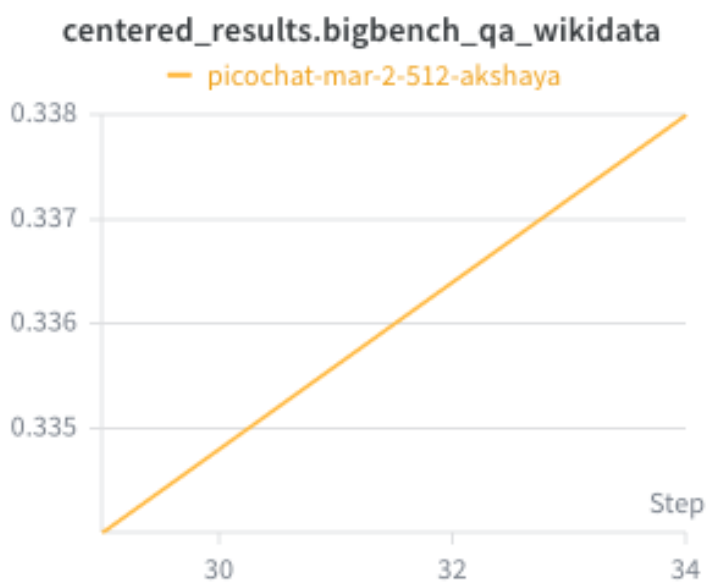


Figure 19:

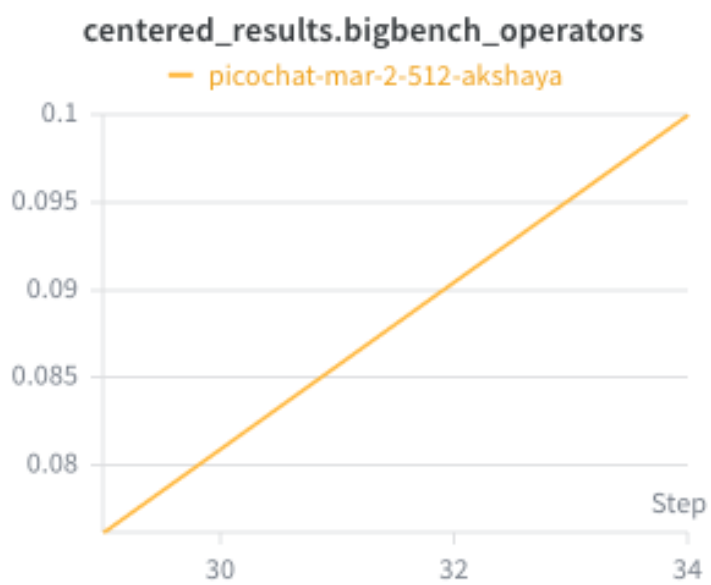


Figure 20:

centered_results.bigbe...anguage_identification

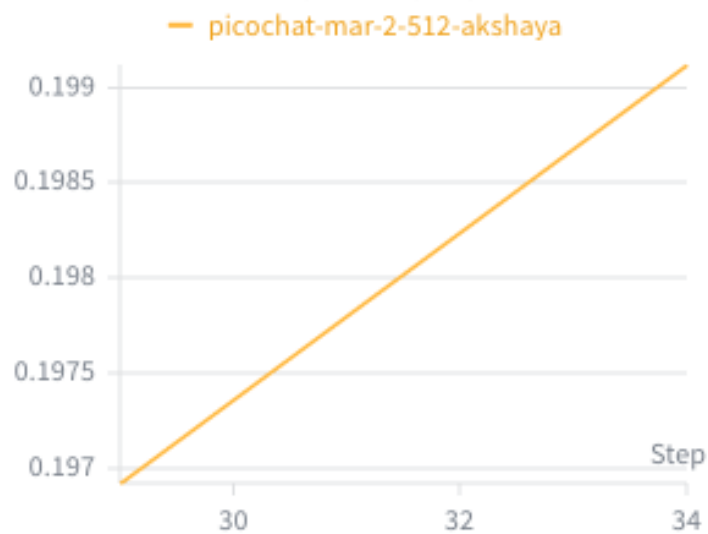


Figure 21:

centered_results.bigbench_dyck_languages

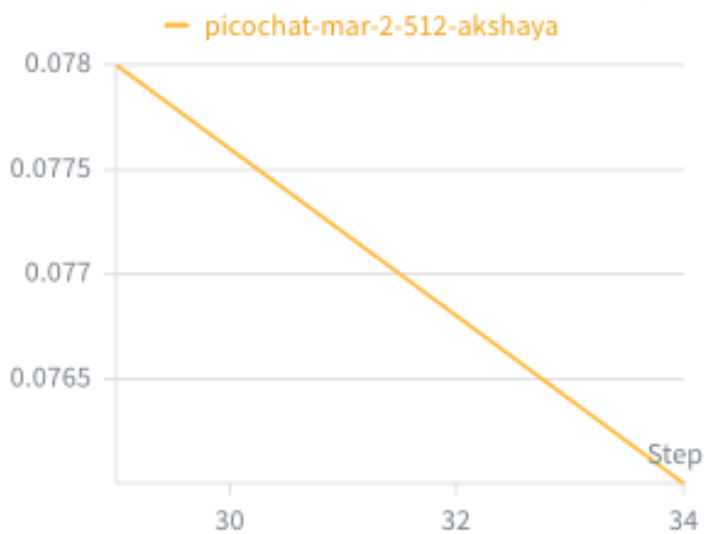


Figure 22:

centered_results.bigbench_cs_algorithms

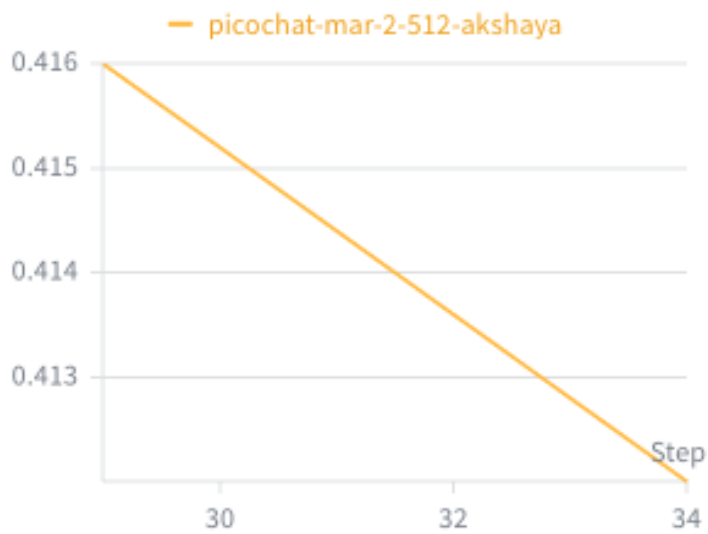


Figure 23:

centered_results.arc_easy

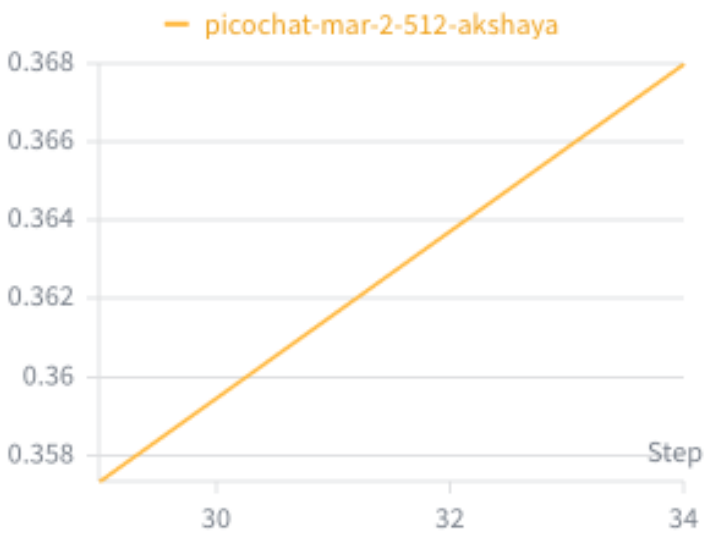


Figure 24:

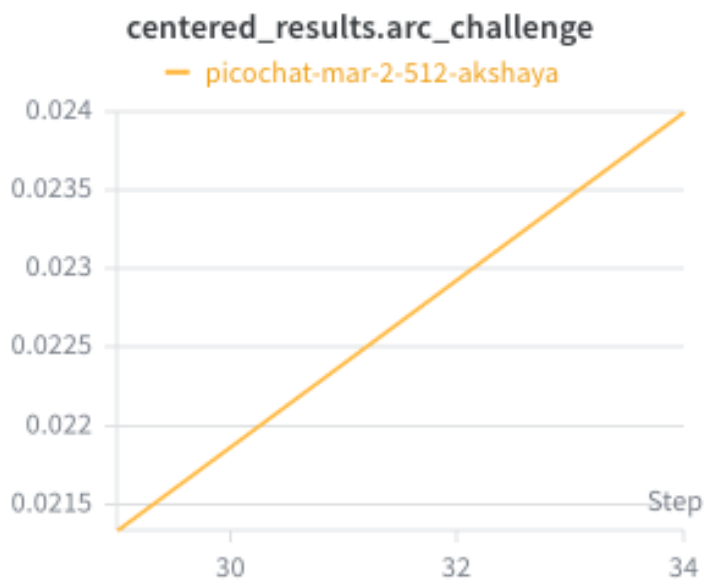


Figure 25:

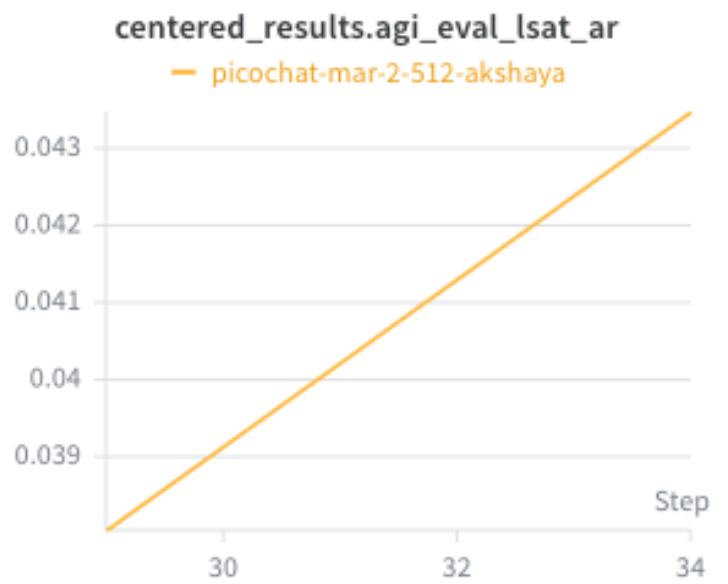


Figure 26:

2 Sequence length 2048 (from checkpoint 1)

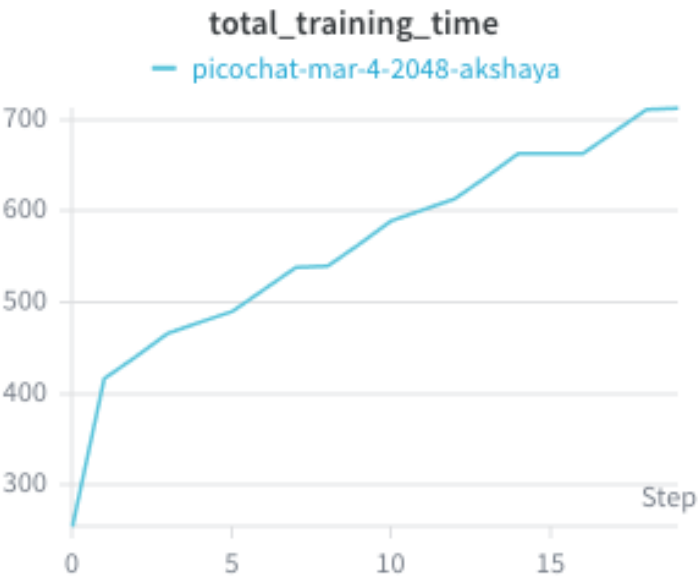


Figure 27:

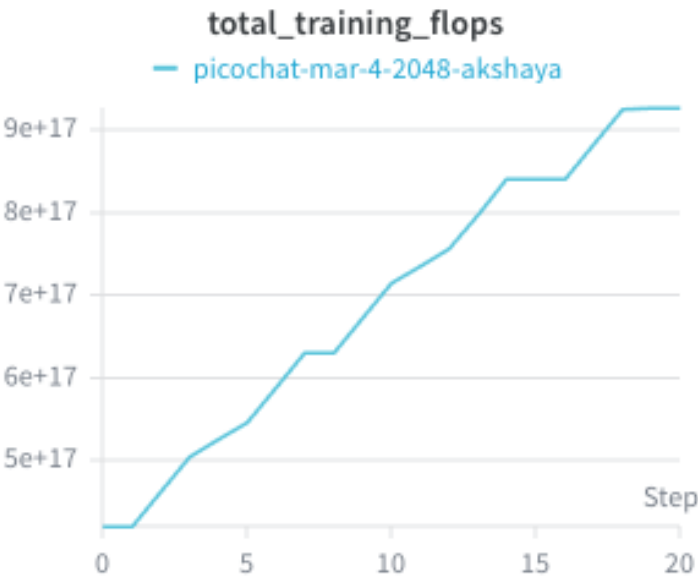


Figure 28:

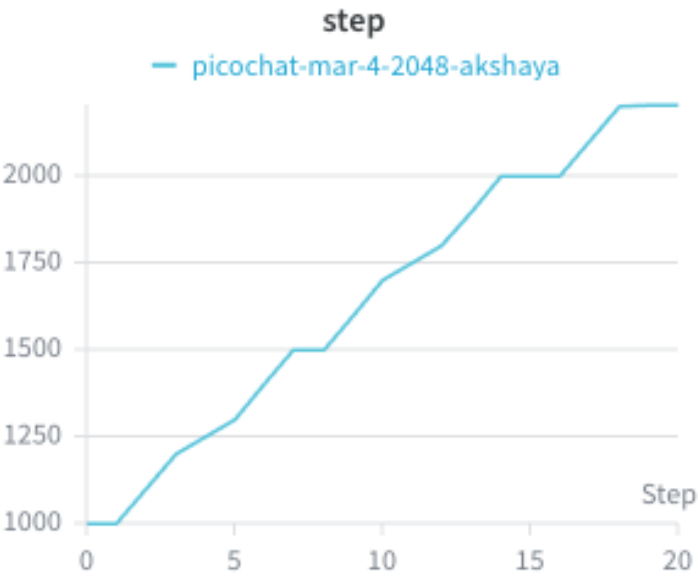


Figure 29:

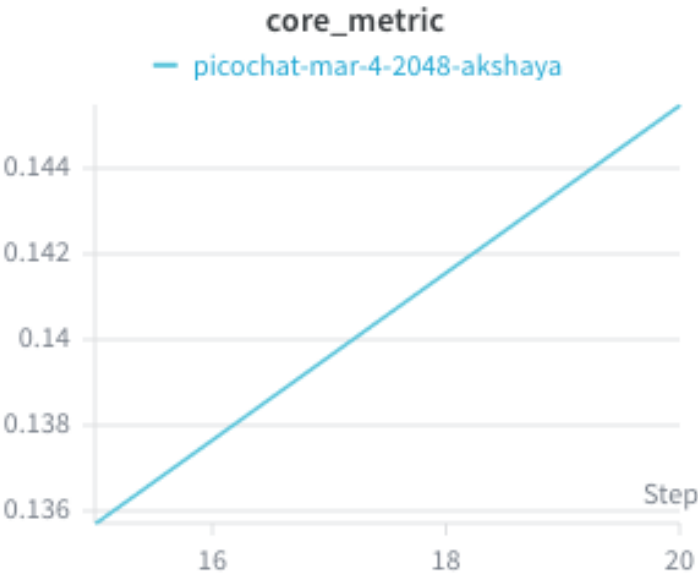


Figure 30:



Figure 31:

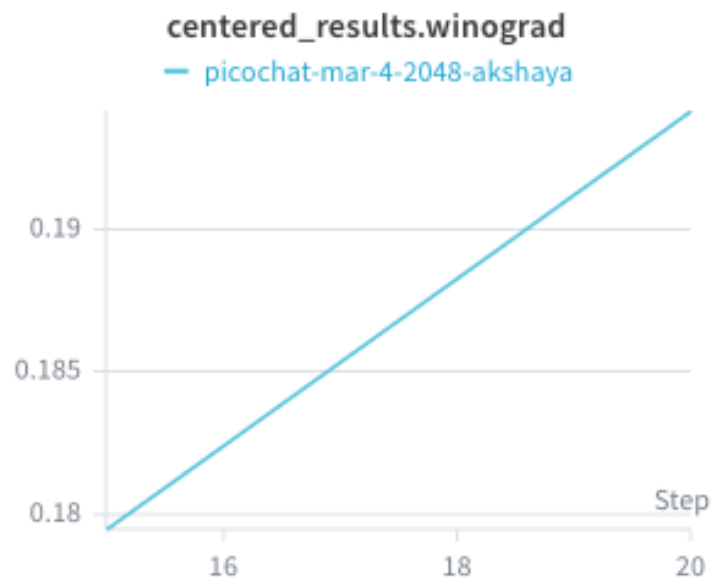


Figure 32:

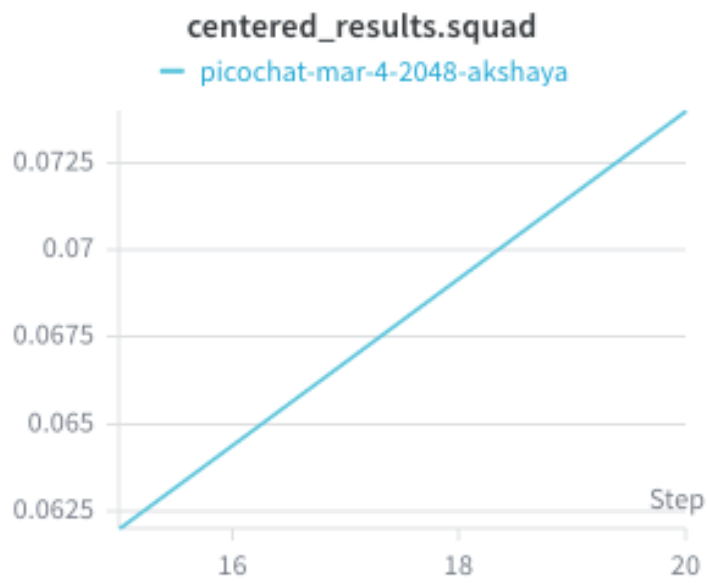


Figure 33:

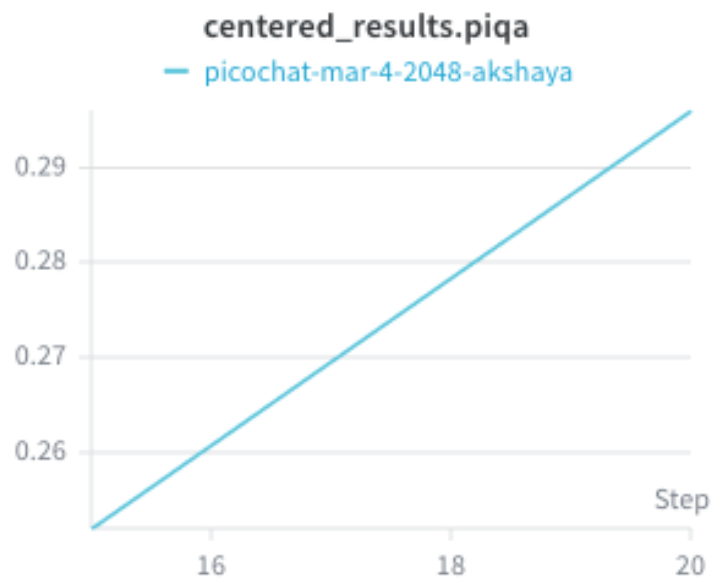


Figure 34:

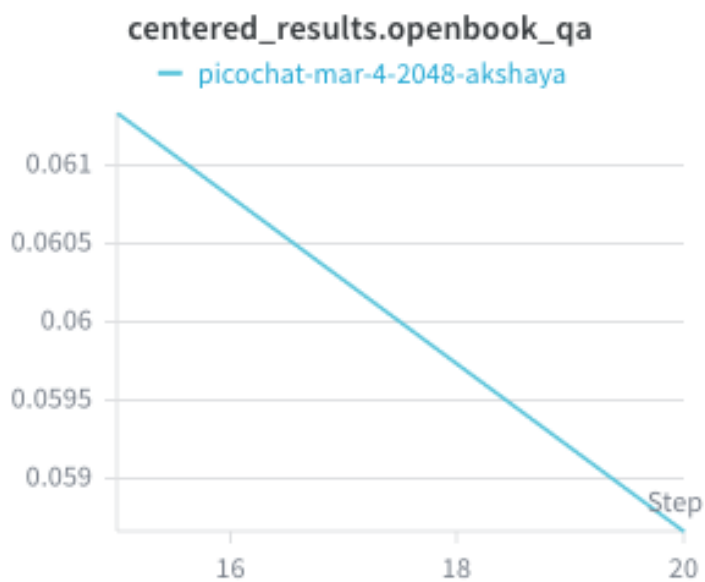


Figure 35:

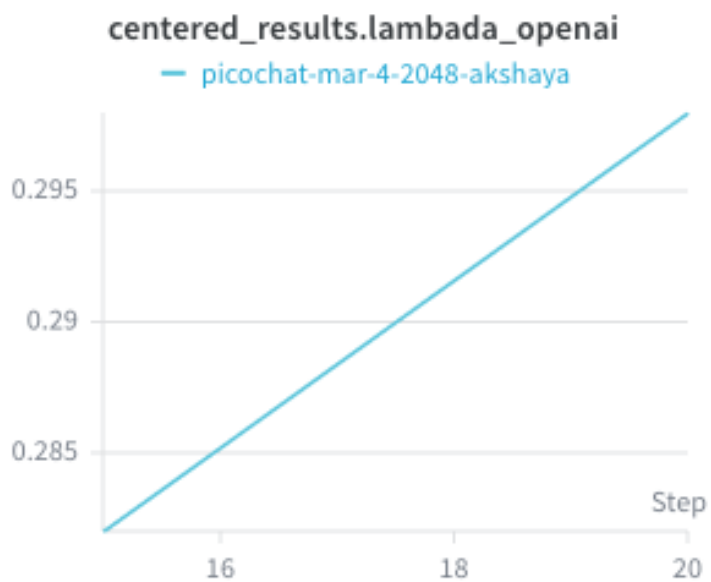


Figure 36:

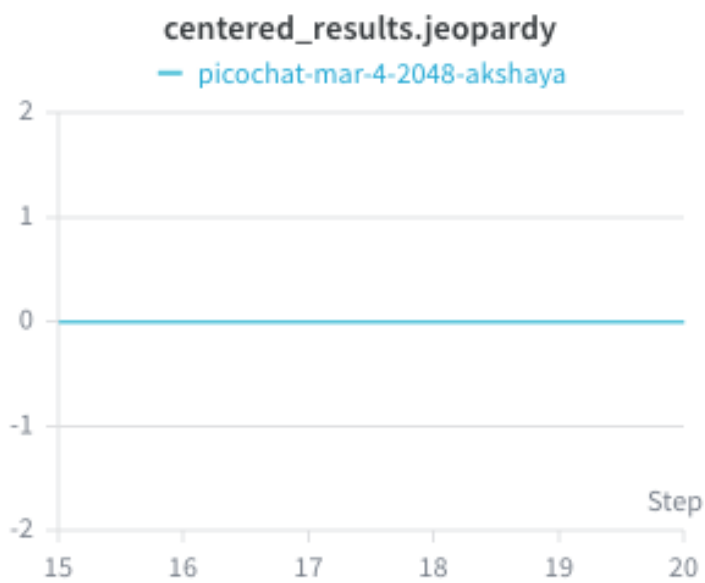


Figure 37:

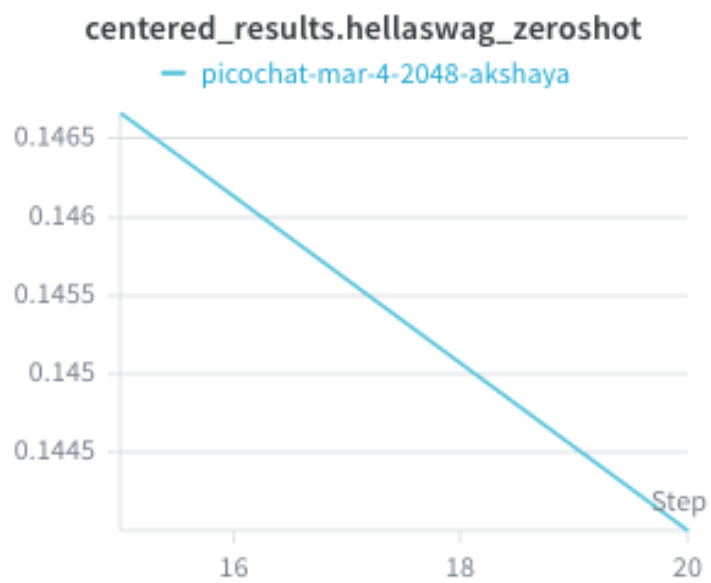


Figure 38:

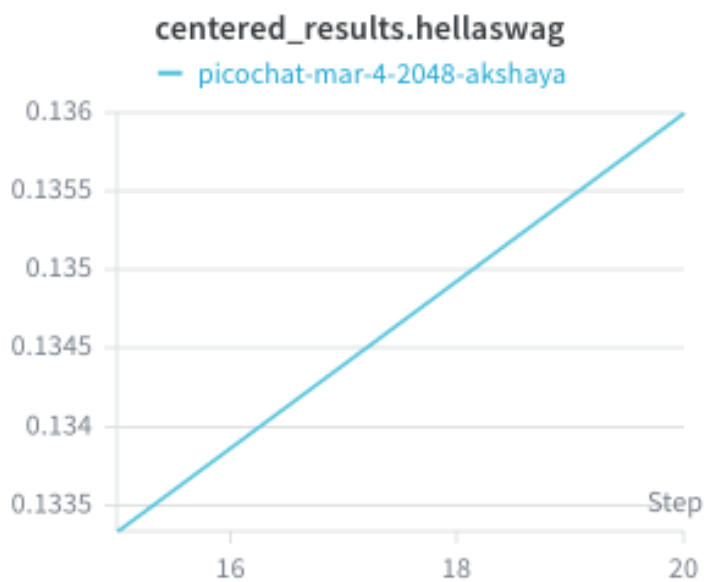


Figure 39:

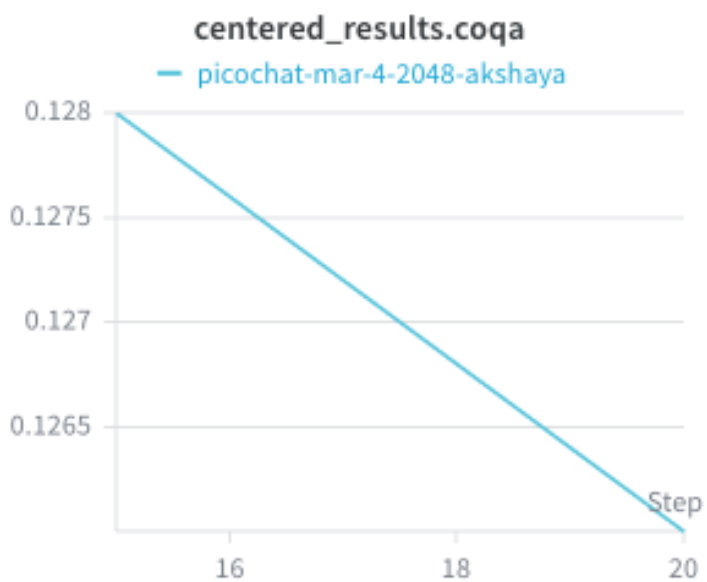


Figure 40:

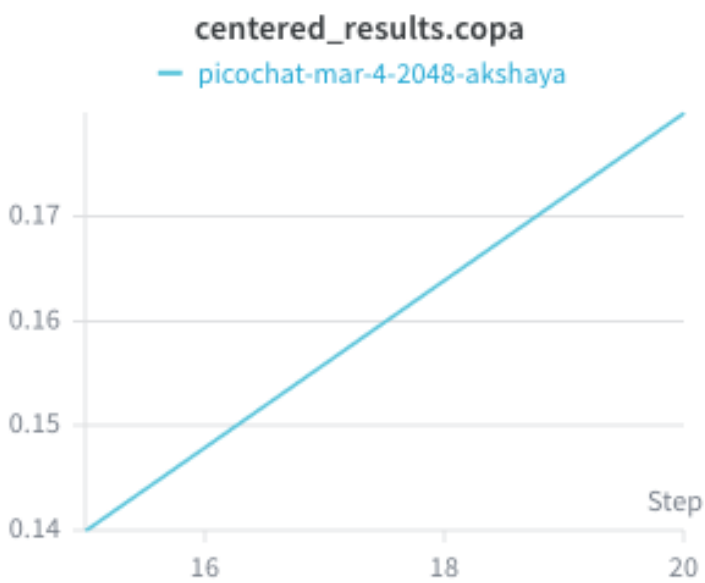


Figure 41:

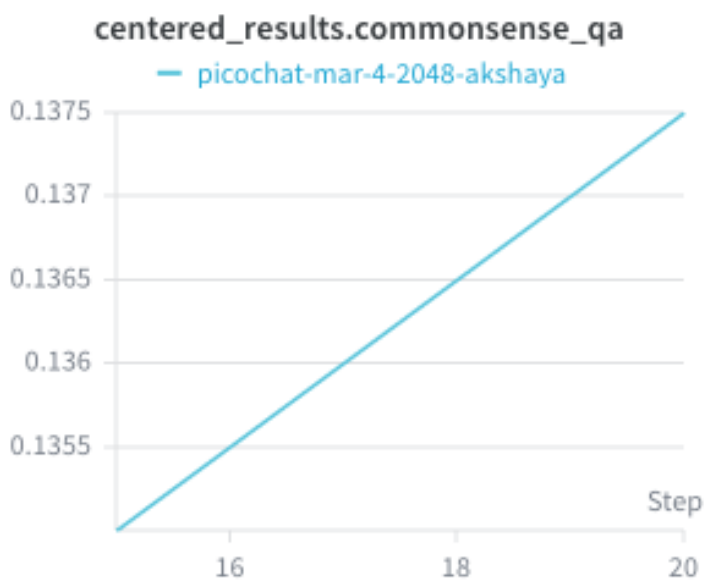


Figure 42:

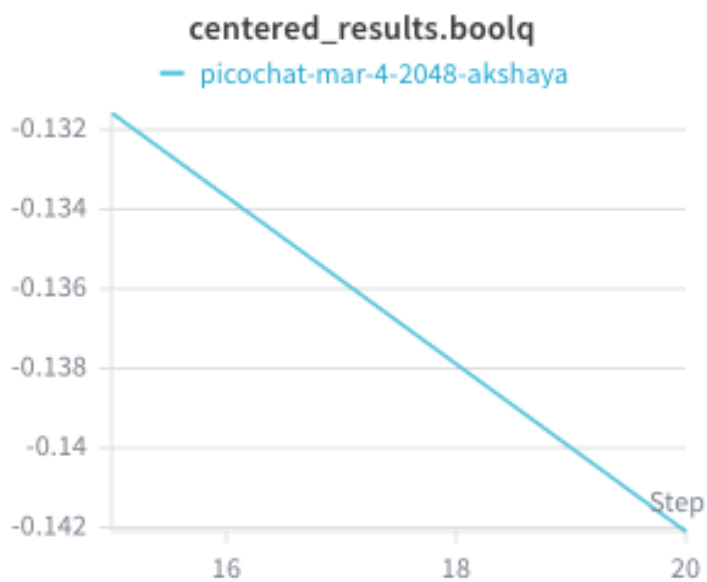


Figure 43:

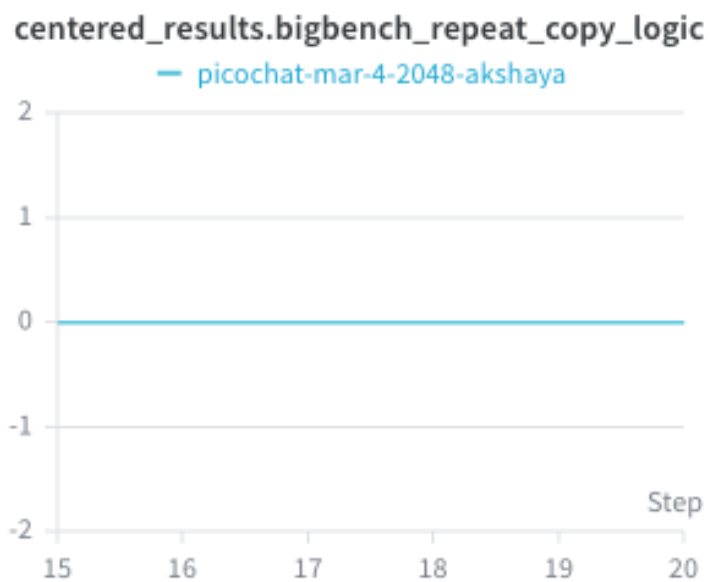


Figure 44:

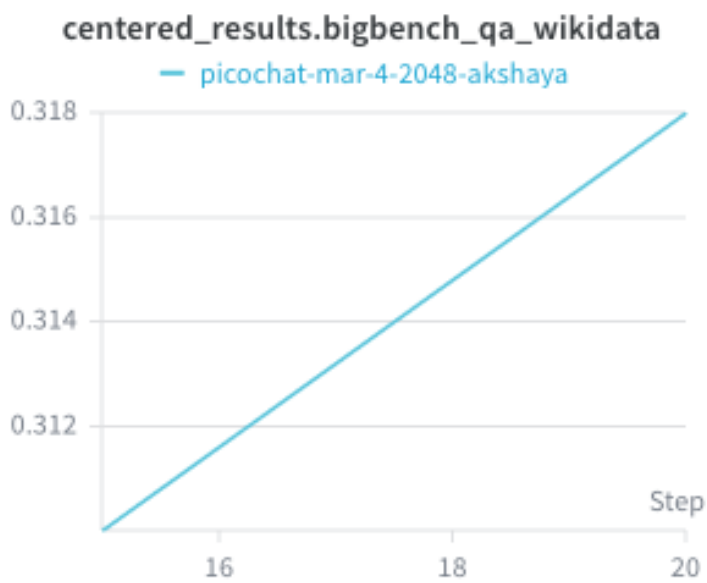


Figure 45:

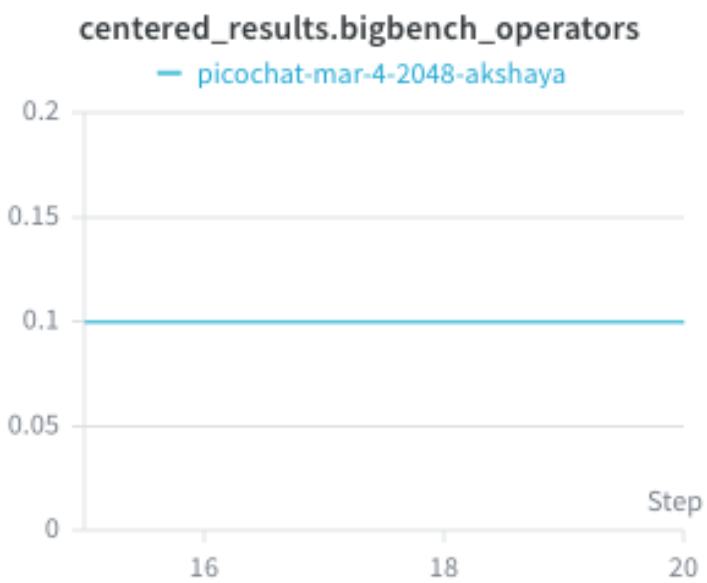


Figure 46:

centered_results.bigbe...anguage_identification

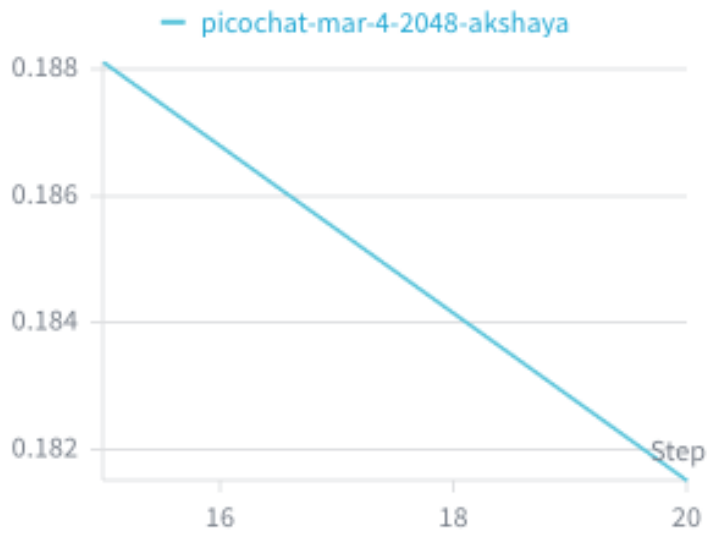


Figure 47:

centered_results.bigbench_dyck_languages

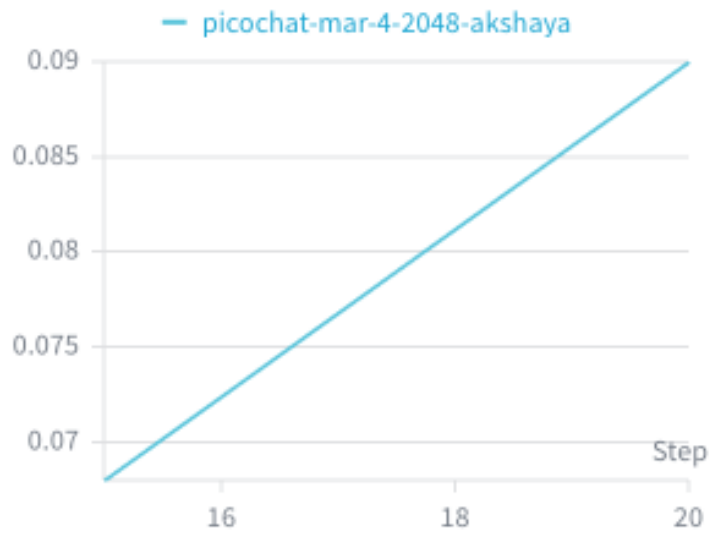


Figure 48:

centered_results.bigbench_cs_algorithms

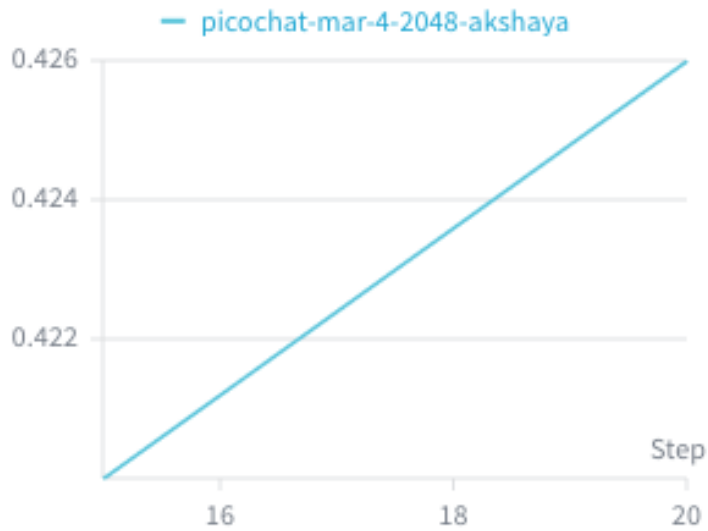


Figure 49:

centered_results.arc_easy

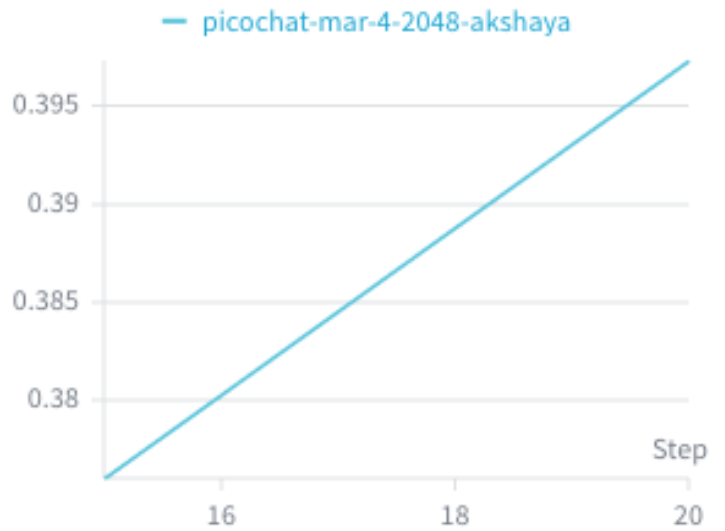


Figure 50:

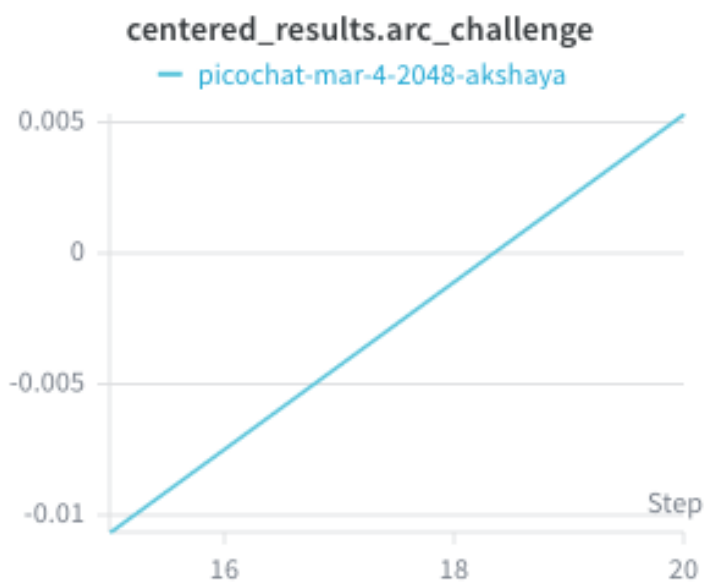


Figure 51:

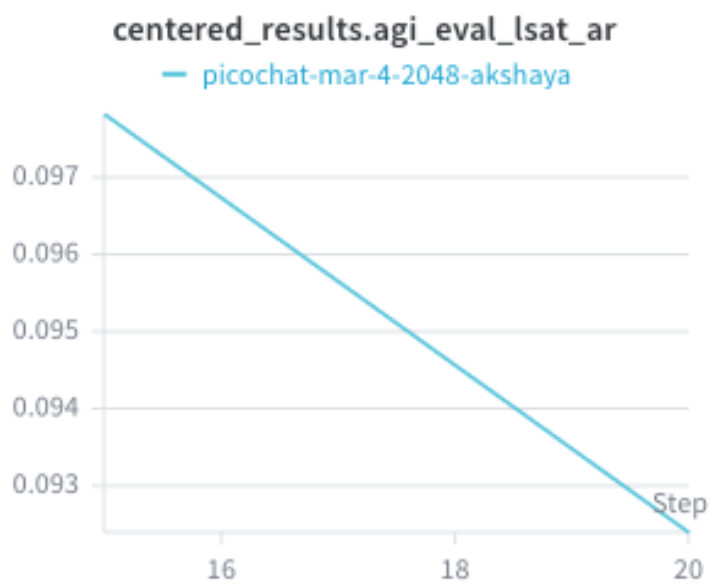


Figure 52:

d20

March 7, 2026

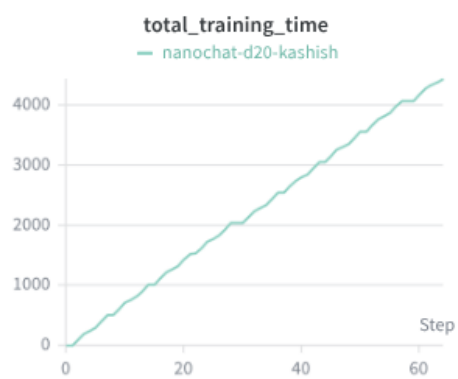


Figure 1:

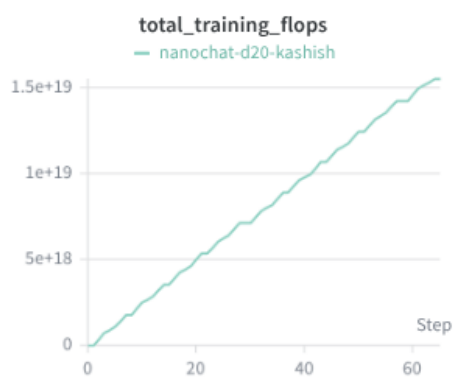


Figure 2:

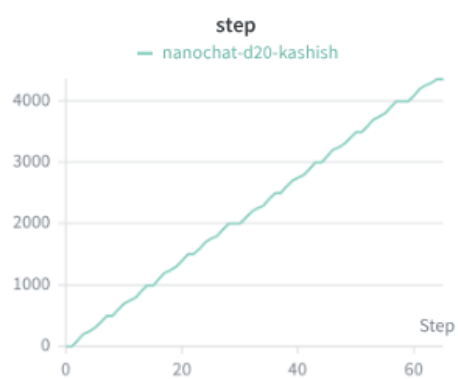


Figure 3:

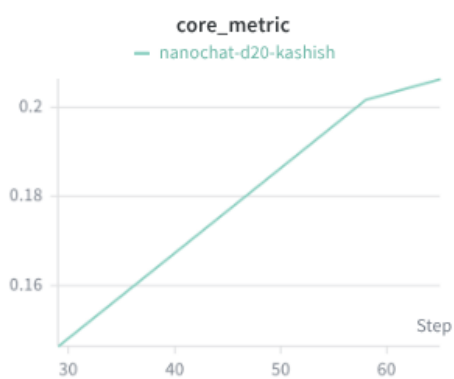


Figure 4:



Figure 5:



Figure 6:

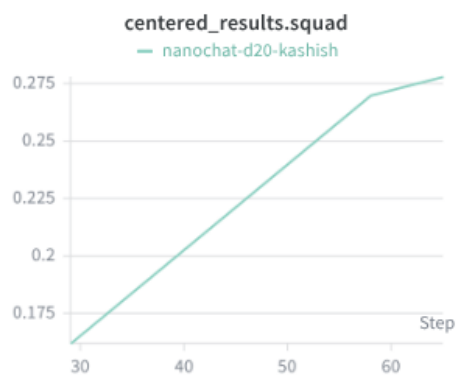


Figure 7:

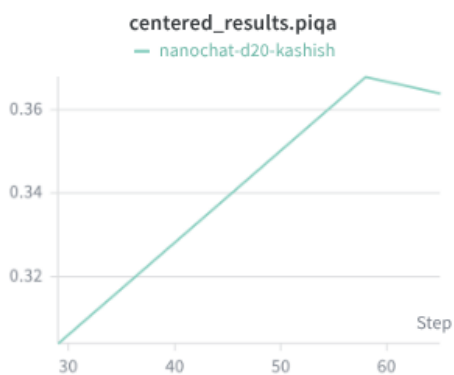


Figure 8:

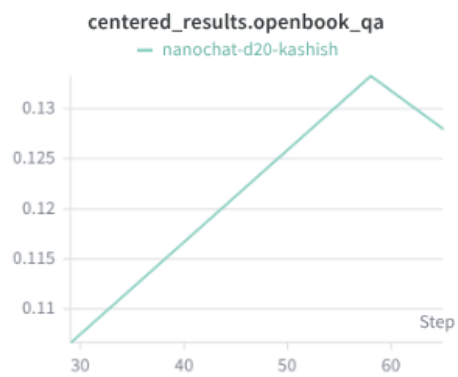


Figure 9:

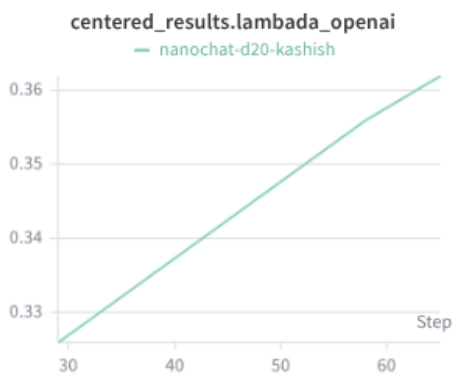


Figure 10:

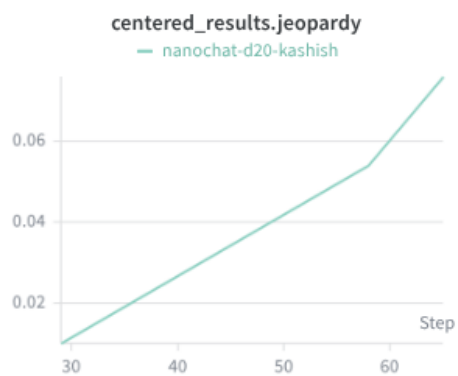


Figure 11:

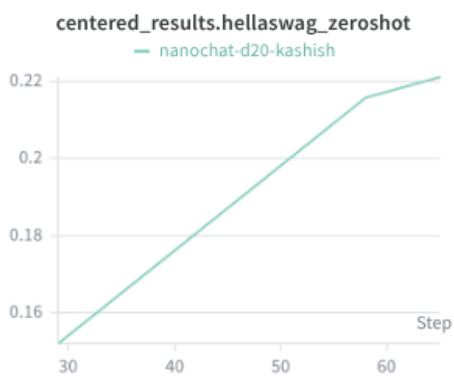


Figure 12:

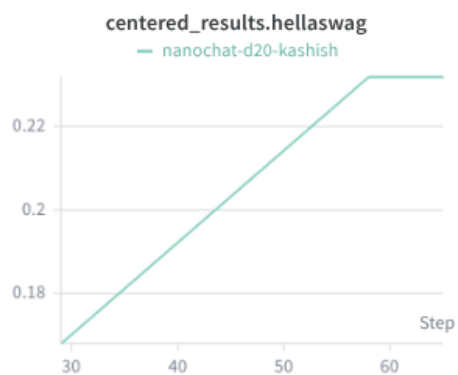


Figure 13:

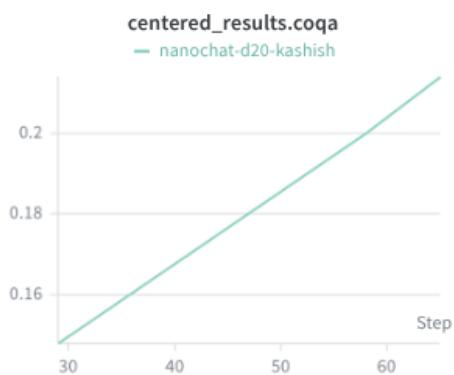


Figure 14:

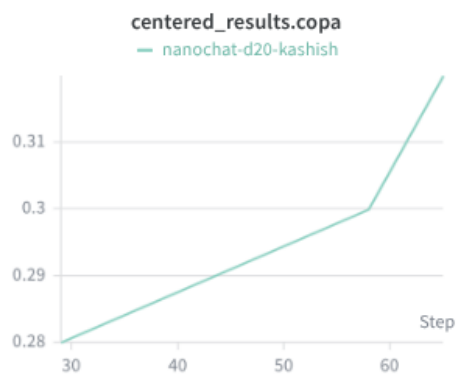


Figure 15:



Figure 16:

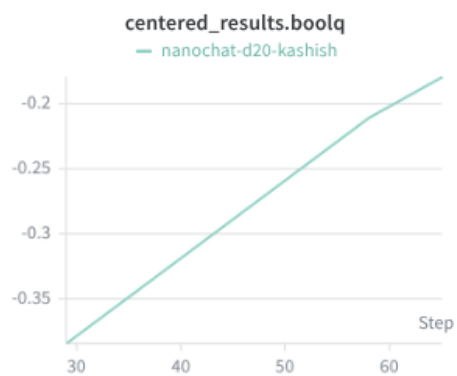


Figure 17:



Figure 18:

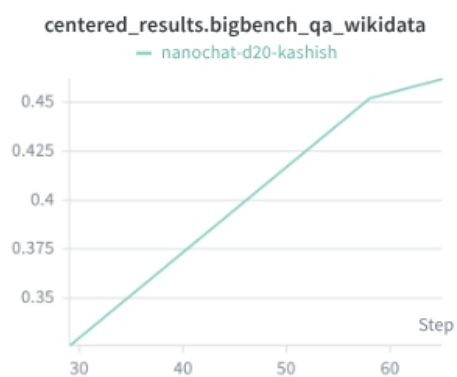


Figure 19:

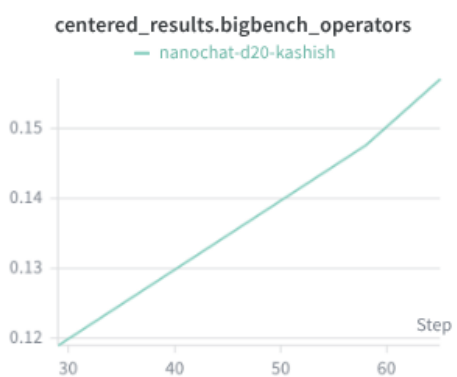


Figure 20:



Figure 21:

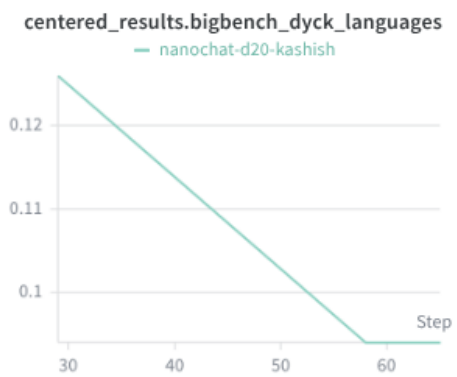


Figure 22:

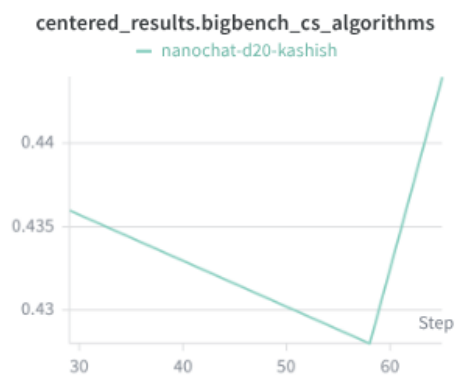


Figure 23:



Figure 24:

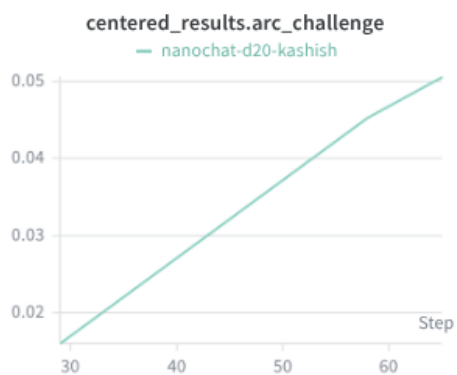


Figure 25:



Figure 26:



Figure 27:



Figure 28:



Figure 29:



Figure 30:



Figure 31:



Figure 32:

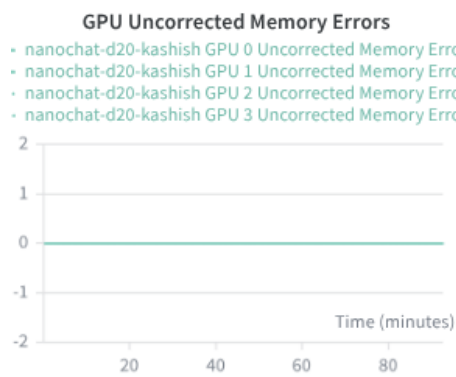


Figure 33:

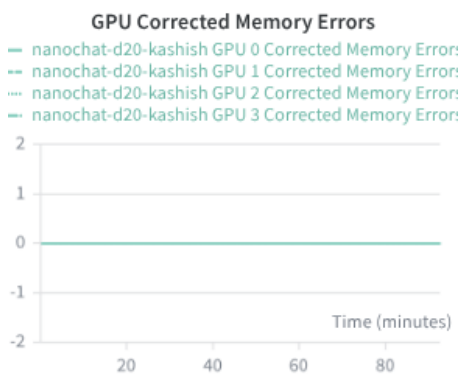


Figure 34:

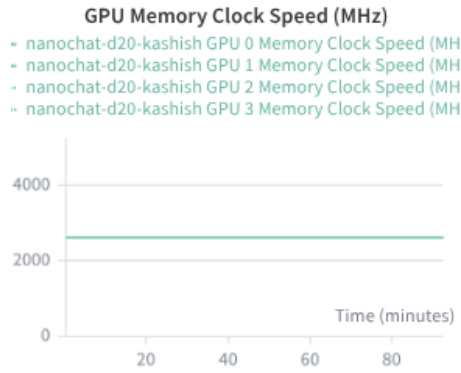


Figure 35:

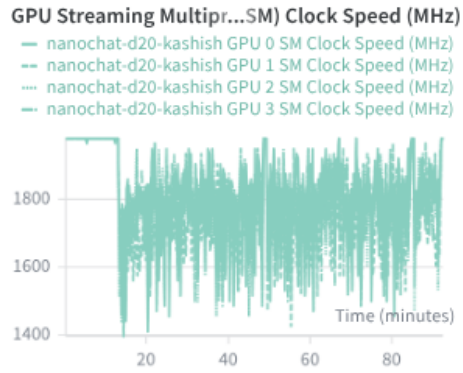


Figure 36:

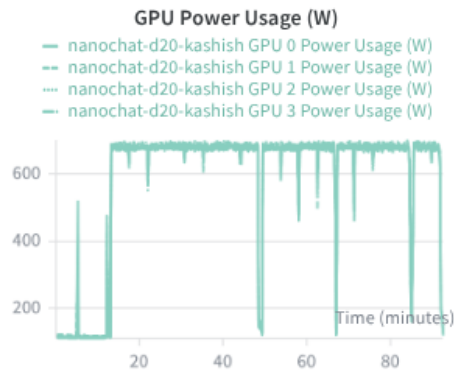


Figure 37:

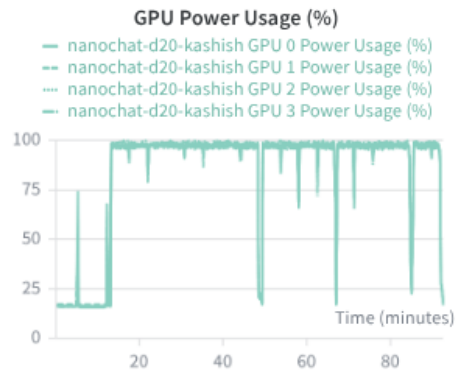


Figure 38:

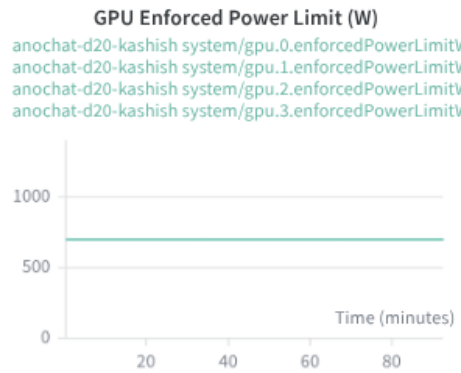


Figure 39:

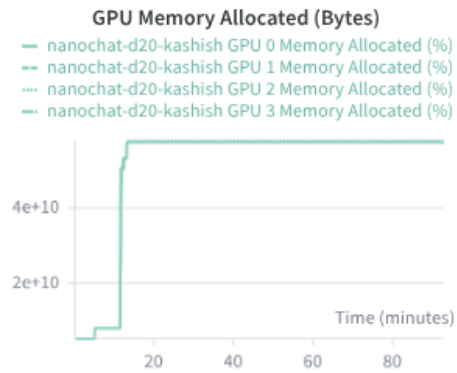


Figure 40:

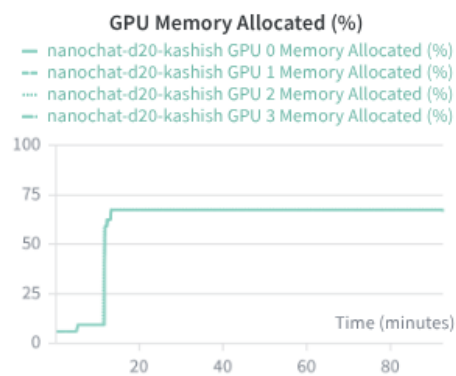


Figure 41:

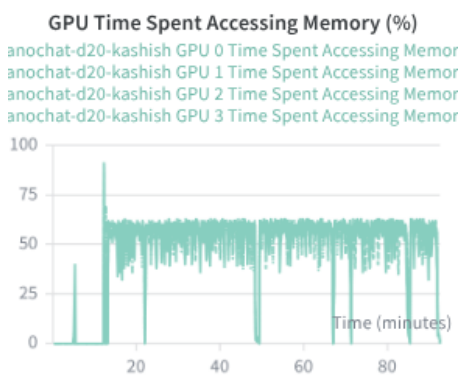


Figure 42:

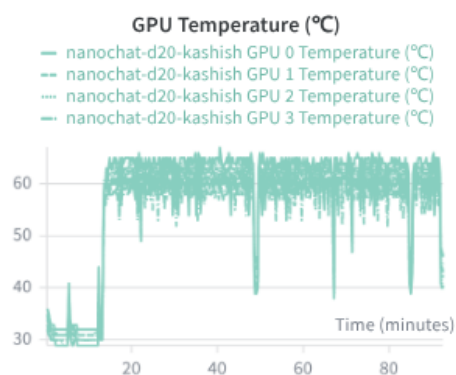


Figure 43:

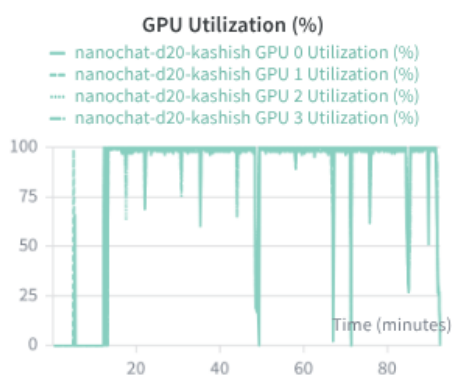


Figure 44:

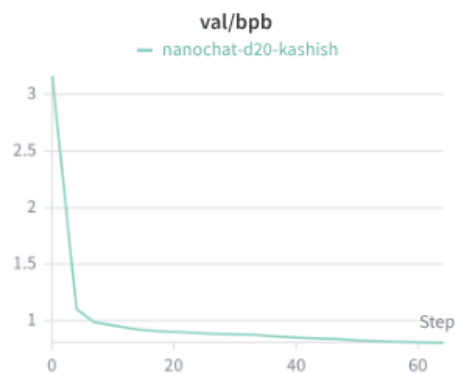


Figure 45: